Agent Skill 표준·등록부

분석모듈을 명세·R wrapper·테스트로 만들고 등록부에 올리는 규약(개발자용)

목차

1 개요 3
1.1 표준화 범위 3
1.2 Skill의 네 구성요소 3
1.3 등록부를 쓰는 곳 4
1.4 현재 규모 4
2 구조와 실행 흐름 6
2.1 저장소 배치 6
2.2 구성요소 간 호출 흐름 6
2.3 실행 틀의 단계 7
2.4 별도 프로세스와 timeout 8
2.5 agent의 2단계 도구 선택 8
3 '종'을 나누는 기준 9
3.1 분석 방법 하나 = 한 종 9
3.2 애매한 경우의 판단 기준 10
3.3 출처(origin)와 연결 함수(r_function) 11
3.4 ID 짓기 11
4 명세(YAML) 필드 레퍼런스 12
4.1 최상위 필드 13
4.2 입력 인자(inputs) 14
4.3 데이터 검증 규칙과 실행 조건 16
4.4 출력(outputs) 17
4.5 오류 처리(errors) 18
4.6 결과 검수 기준(acceptance) 19
4.7 예제(examples) 19
4.8 테스트 파일(tests)과 버전(version) 19
4.9 YAML 1.1 함정 19
5 검증 규칙 레퍼런스 20
5.1 규칙 함수 작성 규약 20
5.2 메시지에 쓰는 값 21
5.3 공통 규칙 22
5.4 계열 전용 규칙 23
5.5 계열 전용 규칙 만들기 26
6 오류 처리 27
6.1 오류 응답 구조 27
6.2 공통 오류 코드 28
6.3 오류 위치별 코드 선택 29
6.4 action별 동작 30
6.5 timeout 30
6.6 오류 메시지 작성 30
7 결과 형식 31
7.1 실제 출력 31
7.2 항목별 설명 33
7.3 JSON 변환 시 주의점 34
7.4 그림 34
8 wrapper 작성 규칙 34
8.1 시그니처와 반환 구조 34

8.2 자료 준비 36
8.3 결과표 36
8.4 그림 37
8.5 경고와 주석 37
8.6 난수 시드 37
8.7 formula 만들기 38
8.8 `[[ ]]`로 읽기 38
8.9 전역 객체를 피할 수 없을 때 38
8.10 금지 사항 39
9 테스트 규칙 39
9.1 테스트 대상 39
9.2 공통 도우미 40
9.3 기준값 대조 요령 41
9.4 실행 42
9.5 현재 테스트 결과 42
10 등록부 생성·점검 42
10.1 실행 42
10.2 점검 항목 43
10.3 출력 3종 43
10.4 성공·실패 시 출력 44
11 Skill 한 종 처음부터 만들어 보기 45
11.1 0단계: 준비 46
11.2 1단계: 명세 46
11.3 2단계: 계열 파일과 전용 규칙 48
11.4 3단계: wrapper 49
11.5 4단계: 한 번 실행해 보기 50
11.6 5단계: 테스트 50
11.7 6단계: 전체 테스트와 등록부 점검 52
11.8 7단계: 반영 52
11.9 새 예제 데이터셋이 필요할 때 52
11.10 마무리 점검표 53
12 버전과 재현성 53
12.1 패키지 버전 고정 53
12.2 기보유 함수 목록(`inventory.csv`) 54
12.3 Skill 버전 55
12.4 실행 기록(provenance) 55
13 기보유 패키지 특이점 대응 55
13.1 대응 원칙 56
13.2 대표 사례 56
14 R 실행 서비스 API 57
14.1 `POST /run` 58
14.2 `POST /validate` 59
14.3 조회 endpoint 60
14.4 동시 처리 60
15 출력 명세와 기계 판정 60
15.1 실행과 출력 형식 판정 60
15.2 입력 오류 판정 61
15.3 명세 작성 시 유의점 61
16 부록 62
16.1 Skill 전체 목록 62
16.2 오류 코드 67
16.3 예제 데이터셋 71

항목	내용
버전	0.1
작성일	2026-10-12
작성	차라투 임창우(초안 작성에 Claude Code 사용)
기준 코드	clinical-agent b66c5f9
등록부	`registry/registry.json` 생성 2026-10-27 13:58:45 UTC, 등록 Skill 106종

- clinical-agent의 분석모듈(Agent Skill)을 새로 만들거나 고치는 개발자와 통계 분석가를 위한 규약서
- 다 읽고 나서 할 수 있어야 하는 일
  - Skill 한 종을 명세(YAML), R wrapper, 테스트, 예제로 만들어 등록부 점검까지 통과
  - 이미 있는 명세를 읽고 그 Skill이 어떤 입력을 받는지, 언제 막히는지, 무엇을 내놓는지 예측
- 읽는 순서
  - 1~3장(개요, 구조와 실행 흐름, '종'을 나누는 기준): 전체 그림
  - 4~10장: 참조 장. 명세 필드, 검증 규칙, 오류, 결과 형식, wrapper, 테스트, 등록부 생성을 하나씩 다룸
  - 11장: 실제 Skill 하나를 처음부터 만드는 순서
  - 처음이면 1~3장과 11장을 먼저 읽고 작업하면서 참조 장을 찾아보는 편이 빠름
- 수치와 예시의 출처
  - Skill 수, 규칙 목록, 오류 코드, 데이터셋, 테스트 건수처럼 코드에서 바로 나오는 표: 문서를 만들 때 등록부와 코드에서 직접 읽음
  - 결과 JSON과 HTTP 응답 예시: 실제로 실행해서 얻은 출력. 실행한 날짜를 함께 적음

1 개요

1.1 표준화 범위

- clinical-agent의 통계 분석은 LLM이 R 코드를 써서 돌리는 방식이 아님. 미리 검증해 등록한 분석 단위만 실행
- 이 단위가 Agent Skill(이하 Skill). 통계 분석 방법 하나가 Skill 하나
- agent는 요청을 읽고 어떤 Skill을 어떤 인자로 부를지만 정함
- 실제 계산은 R 실행 서비스가 등록된 Skill의 R 함수를 불러 수행

표준화한 것:

- Skill을 기술하는 형식
  - Skill마다 YAML 명세 한 개
  - 정해진 필드에 적는 것: 입력 인자의 형식, 데이터 검증 규칙, 실행 조건, 출력 표의 이름과 열, 오류 코드별 처리 방식, 결과 검수 기준, 예제
  - 필드는 JSON Schema(`registry/schema/skill.schema.json`)로 정의 → 형식이 틀린 명세는 등록부 점검에서 걸림
- 실행 방식
  - 모든 Skill이 같은 실행 틀(`r-executor/R/core.R`)을 거침
  - 실행 틀은 명세를 읽어 데이터셋을 허용 목록에서 찾고 인자 검증, 검증 규칙·실행 조건 점검을 마친 뒤에야 wrapper 함수를 부름
  - 결과는 표·그림·주석·경고·오류·출처 정보로 이루어진 같은 구조 → agent는 Skill마다 다른 코드를 둘 필요 없음
- 등록 절차
  - Skill 하나를 추가하려면 명세, wrapper, 테스트, 예제 데이터를 갖춰야 함
  - 등록부 생성기(`registry/build_registry.R`)가 이것들을 대조해 문제가 없을 때만 통과
  - 생성기가 함께 만드는 것: 사람이 읽는 요약(`REGISTRY.md`), 항목별 상세 등록부(`registry.xlsx`), agent가 읽는 기계용 등록부(`registry.json`)

1.2 Skill의 네 구성요소

- Skill 한 종은 아래 네 가지로 이루어짐. 하나라도 빠지면 등록부 점검을 통과하지 못함
- 예제 데이터는 이미 허용 목록에 있는 데이터셋을 쓰면 새로 만들지 않음

구성요소	위치	하는 일
명세	`registry/skills/<ID>.yaml`	입력·검증 규칙·실행 조건·출력·오류·검수 기준·예제를 적음. 실행 틀과 agent가 모두 읽음
R wrapper	`r-executor/R/skills/<계열>.R`의 `skill_<계열>_<방법>(data, p)`	검증을 마친 인자로 실제 분석 함수를 부르고 결과를 표·그림으로 정리
테스트	`r-executor/tests/testthat/test-<ID>.R`	정상 사례: 결과표 값을 독립적으로 계산한 기준값과 대조. 오류 사례 2개 이상: 정의한 오류 코드로 막히는지 확인
예제 데이터	`r-executor/R/datasets*.R`의 허용 목록	R 패키지에 들어 있는 공개 자료를 정해진 전처리로 읽음. 명세의 `examples`가 이 ID를 씀

- 명세와 wrapper는 `implementation` 필드로, 명세와 테스트는 `tests` 필드로 이어짐
- 데이터셋은 ID로만 가리킴. 실행 틀은 허용 목록에 없는 ID나 파일 경로를 받지 않음

1.3 등록부를 쓰는 곳

- 등록부는 문서용 목록이 아님. 시스템 여러 곳이 실제로 읽는 기준

읽는 곳:

- R 실행 서비스: 시작할 때 `registry/skills/*.yaml`을 모두 읽음
  - 명세에 없는 Skill ID는 실행하지 않음
  - 명세에 없는 인자는 거부
  - 명세의 규칙과 오류 정의대로 막거나 경고
- agent: `registry/registry.json`을 읽어 LLM에 줄 도구 정의를 만듦
  - Skill ID → 도구 이름, 기능명·분석 방법·설명 → 도구 설명, 입력 정의 → 도구 인자 schema
  - 실행을 요청할 때 등록부의 버전을 함께 보냄. R 실행 서비스의 버전과 다르면 실행되지 않음(섹션 12)
- 분석계획 검증: 계획에 든 분석마다 R 실행 서비스의 `/validate`를 불러 명세의 입력 정의와 규칙으로 미리 점검
  - 사용자가 직접 낸 계획과 agent가 만든 계획이 같은 검증을 거침
- 시험 실행기(`eval/kaic_suite.py`)
  - 등록부의 예제로 Skill 실행
  - 명세의 출력 표 열 목록과 실제 결과를 기계적으로 대조
  - 입력 정의에서 오류 입력을 자동으로 만들어 명세의 오류 코드와 action대로 막히는지 확인(섹션 15)

결과:

- 명세를 고치면 → 실행 동작, LLM이 보는 도구, 시험 판정에 동시에 반영
- 명세와 wrapper가 어긋나면(예: 명세에 없는 열을 wrapper가 내면) → 테스트는 통과해도 시험 판정에서 걸릴 수 있음
- 이 문서의 규칙 대부분은 이 어긋남을 막기 위한 것

1.4 현재 규모

- 등록부(`registry.json`, 생성 2026-10-27 13:58:45 UTC)에 106종 등록
- Skill ID 접두어는 그 Skill이 바탕으로 삼은 기보유 패키지를 나타냄

접두어	기보유 출처	종 수
JST	jstable	39
JSK	jskm	4
JSM	jsmodule	18
SPL	splineplot	3
STV	statgarten stove	17

접두어	기보유 출처	종 수
SRB	statgarten soroban	5
BRD	statgarten board	4
LKT	statgarten shiny.likert	1
OSA	openstat_app	13
CDS	CDISC ADaM 연계(jstable·jskm 기반)	2
	합계	106

분석 범주별:

분석 범주	종 수
생존분석	12
기술통계·그룹비교	11
서베이·표본설계	10
머신러닝(분류)	9
메타분석	8
머신러닝(회귀)	7
진단·예측성능	7
표본수 계산	5
시각화	4
탐색적 자료분석	4
회귀분석	4
다변량 분석	3
반복측정·GEE	3
반복측정·혼합모형	3
상관분석	3
서브그룹 분석	3
스플라인·용량-반응	3
CDISC 연계	2
인과성분석	2
기술통계	1
머신러닝(군집)	1
설문 분석	1

- 명세 전체 합계: 입력 인자 정의 819개, 데이터 검증 규칙 583개, 실행 조건 234개, 오류 처리 정의 1,301개(서로 다른 오류 코드 137종)
- 그림을 내는 Skill: 56종
- Skill 전체 목록: 부록(섹션 16.1)

2 구조와 실행 흐름

2.1 저장소 배치

- Skill 관련 파일은 세 폴더에 나뉨. 아래는 이 문서가 다루는 범위만 추린 것

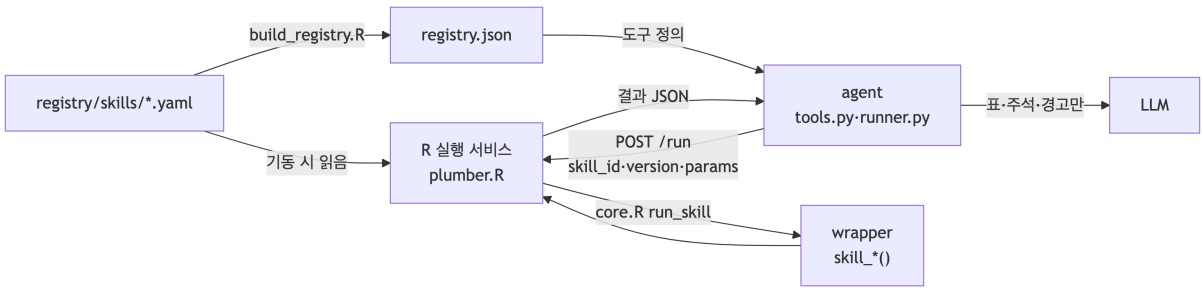
```
registry/
  skills/<ID>.yaml            Skill 명세(Skill마다 1개)
  schema/skill.schema.json    명세 JSON Schema
  build_registry.R            등록부 점검·생성기
  inventory.R, inventory.csv  기보유 함수 목록과 그 생성 스크립트
  REGISTRY.md, registry.xlsx, registry.json   생성물(직접 고치지 않는다)
r-executor/
  R/core.R                    공통 실행 틀(입력 검증, 공통 규칙, 실행, 결과, 그림 저장)
  R/isolate.R                 별도 R 프로세스 실행과 제한 시간
  R/load.R                    로더(실행 틀과 모든 wrapper를 읽는다)
  R/datasets.R, R/datasets_<계열>.R   데이터셋 허용 목록
  R/skills/<계열>.R           wrapper 함수와 계열 전용 규칙
  plumber.R, serve.R          R 실행 서비스 API와 기동 스크립트
  tests/testthat.R            전체 테스트 실행(JUnit 결과 기록)
  tests/testthat/helper-skills.R, test-<ID>.R
  install_packages.R, Dockerfile, DESCRIPTION
agent/clinical_agent/
  tools.py                    등록부 → LLM 도구 정의
  runner.py                   통계실행 agent(2단계 도구 선택)
  loop.py                     도구 호출 루프(오류 전달, 감사 기록)
  executor.py                 R 실행 서비스 클라이언트
  workflow.py                 분석계획 검증·실행
```

- `r-executor/R/datacheck.R`(데이터 검증)와 `upload.R`(업로드 자료 가명처리)도 같은 로더로 읽힘. Skill 작성과는 직접 관계가 없어 이 문서에서 다루지 않음

`load.R`이 파일을 읽는 방식:

- 읽는 순서(모두 전역 환경에): `core.R` → `isolate.R` → `datacheck.R` → `upload.R` → `datasets.R` → `datasets_*.R`(이름순) → `skills/*.R`(이름순)
- 저장소에 `.r-lib` 폴더가 있으면 그 라이브러리를 맨 앞에 둠
- 모든 파일이 같은 전역 환경을 씀
  - 그래서 가능한 것: 계열 파일이 `rule_funs`에 규칙 덧붙이기, 다른 계열 파일의 도우미 함수를 실행 시점에 부르기(예: `km.R`이 `cox.R`의 `cox_surv()`를 씀)
  - 파일을 읽는 순간(최상위 코드)에 다른 계열 파일의 함수를 부르면 그 파일이 아직 읽히지 않았을 수 있음 → 최상위에는 함수·목록 정의만 둠
- 같은 이름을 두 파일이 정의하면 나중 파일이 조용히 덮어씀 → 등록부 생성기가 이름 충돌을 점검(섹션 10)

2.2 구성요소 간 호출 흐름

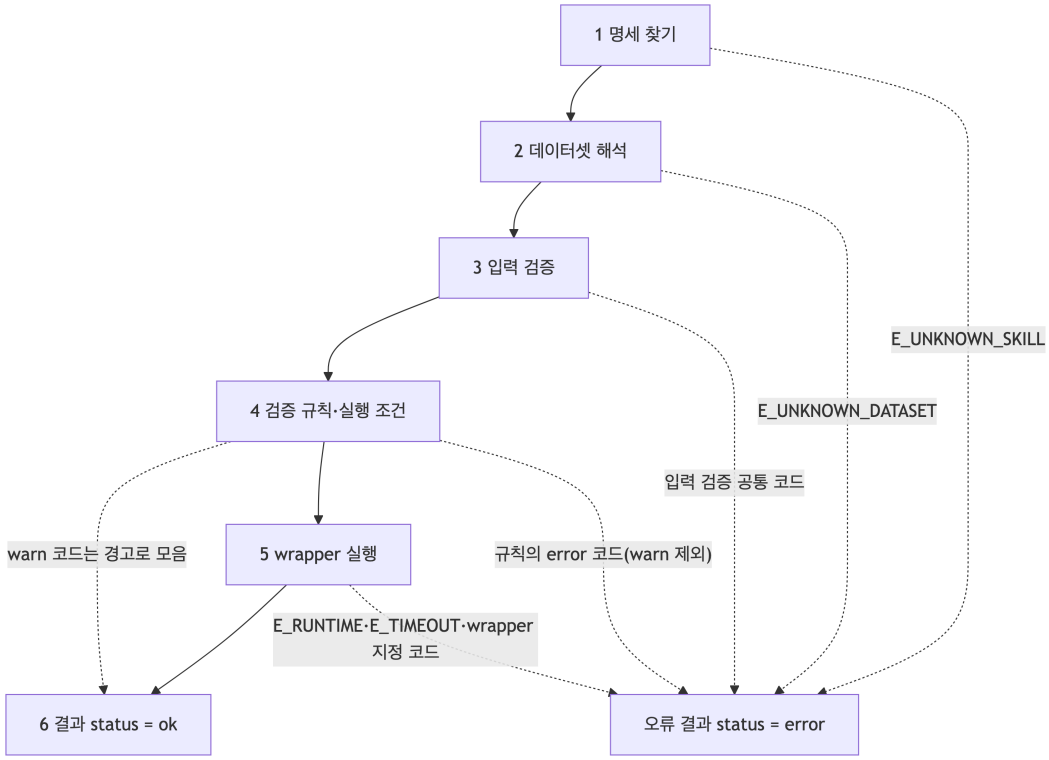


- R 실행 서비스와 agent가 읽는 파일이 다름(주의)
  - R 실행 서비스: YAML 명세를 직접 읽음
  - agent: 생성기가 만든 `registry.json`을 읽음
- 명세를 고친 뒤 생성기를 다시 돌리지 않으면
  - agent는 예전 명세로 도구를 만듦

  - 버전이 바뀐 경우에는 R 실행 서비스가 실행을 거부(`E_VERSION_MISMATCH`, 섹션 12)
- 컨테이너 배포도 같음
  - R 실행 서비스 image에는 `registry/skills`와 `r-executor/R`을, agent image에는 `registry/registry.json`을 복사해 넣음
  - Skill을 추가하거나 고친 뒤에는 두 이미지를 모두 다시 빌드

2.3 실행 틀의 단계

- R 실행 서비스의 `/run`은 `core.R`의 `run_skill(skill_id, dataset_ref, params)`을 부름
- 아래 순서로 진행. 어느 단계든 오류가 나면 그 자리에서 멈추고 구조화된 오류 결과를 돌려줌



1. 명세 찾기
   - 시작할 때 읽은 명세 목록에서 Skill ID를 찾음. 없으면 `E_UNKNOWN_SKILL`
2. 데이터셋 해석
   - `dataset_ref`가 `up_` 뒤에 16~64자리 16진수가 붙은 형식이면 업로드 자료의 정제본을 읽음
   - 그 밖에는 허용 목록(`dataset_catalog`)의 ID여야 함. 목록의 `loader()` 함수로 자료를 만듦
   - 경로나 코드는 받지 않음. 해당이 없으면 `E_UNKNOWN_DATASET`
3. 입력 검증
   - 명세 `inputs`에 없는 인자가 들어오면 `E_UNKNOWN_PARAM`
   - 그다음 명세에 적힌 순서대로 인자마다 필수 여부, 형식, 범위, 열 존재, 변수 형식(`dtype`)을 봄
   - 빠진 선택 인자에는 명세의 `default`를 채움
   - 자세한 판정: 섹션 4.2
4. 데이터 검증 규칙과 실행 조건
   - 명세의 `validation` 목록을 위에서부터, 이어서 `preconditions` 목록을 위에서부터 실행
   - 규칙이 위반을 알리면 그 규칙의 `error` 코드를 명세 `errors`에서 찾아 action대로 처리: `warn`이면 경고로 모으고 다음 규칙으로, `warn`이 아니면(`block`·`ask_user`) 그 자리에서 멈춤
   - 두 목록은 같은 방식으로 실행. 구분은 의미에 따른 것(섹션 4.3)
5. wrapper 실행
   - 명세 `implementation`에 적힌 함수를 `impl(data, p)`로 부름. `p`는 3단계를 통과하고 기본값이 채워진 인자 목록
   - 실행 중 콘솔 출력과 메시지는 버림. 경고는 모아서 결과의 `warnings`에 넣음
   - 단 "was built under R version", "is deprecated", "replacing previous import", "Registered S3 method"가 들어간 환경 메시지는 넣지 않음
   - R 수준의 timeout(`R_EXECUTOR_TIMEOUT_SEC`, 기본 60초)을 넘으면 `E_TIMEOUT`

6. 결과
   - `status = "ok"`와 함께 돌려주는 것: wrapper가 돌려준 표·그림·주석, `warnings`(규칙 경고 + wrapper 경고 + 실행 중 잡힌 경고)
   - 성공이든 오류든 결과에는 출처 정보(`provenance`)가 붙음(섹션 7)

- 같은 단계에서 5번을 빼고 1~4번만 하는 함수가 `validate_skill()`, API로는 `/validate`
- 분석계획을 접수할 때와 데이터 검증 기능이 이것을 씀

2.4 별도 프로세스와 timeout

- `setTimeLimit()`으로 거는 R 수준 제한은 R 코드가 실행되는 동안에만 끊김
- 컴파일된 코드 안에서 멈춘 계산은 끊지 못함(geepack의 반복 추정이 2분 넘게 돌아오지 않은 사례 있음)
- 그래서 `/run`은 기본적으로 `isolate.R`의 `run_skill_isolated()`로 Skill을 별도 R 프로세스(`callr::r_session`)에서 돌림

별도 프로세스의 동작:

- 프로세스는 한 번 띄운 뒤 다음 요청에 다시 씀. 프로세스가 죽었으면 다음 요청 때 새로 띄움
- 프로세스 timeout은 `R_EXECUTOR_TIMEOUT_SEC`보다 5초 김
  - 넘기면 프로세스를 강제 종료하고 `E_TIMEOUT` 오류를 돌려줌
  - 이때 출처 정보에 `isolation: "process_killed"`가 붙고 패키지 버전 같은 항목은 빠짐
- 프로세스 자체에서 R 오류가 새어 나오면 프로세스를 종료하고 `E_RUNTIME`을 돌려줌
- `R_EXECUTOR_ISOLATE=false`면 별도 프로세스 없이 같은 프로세스에서 `run_skill()`을 부름
- 테스트와 등록부 생성기는 `run_skill()`을 직접 부름

동시 실행:

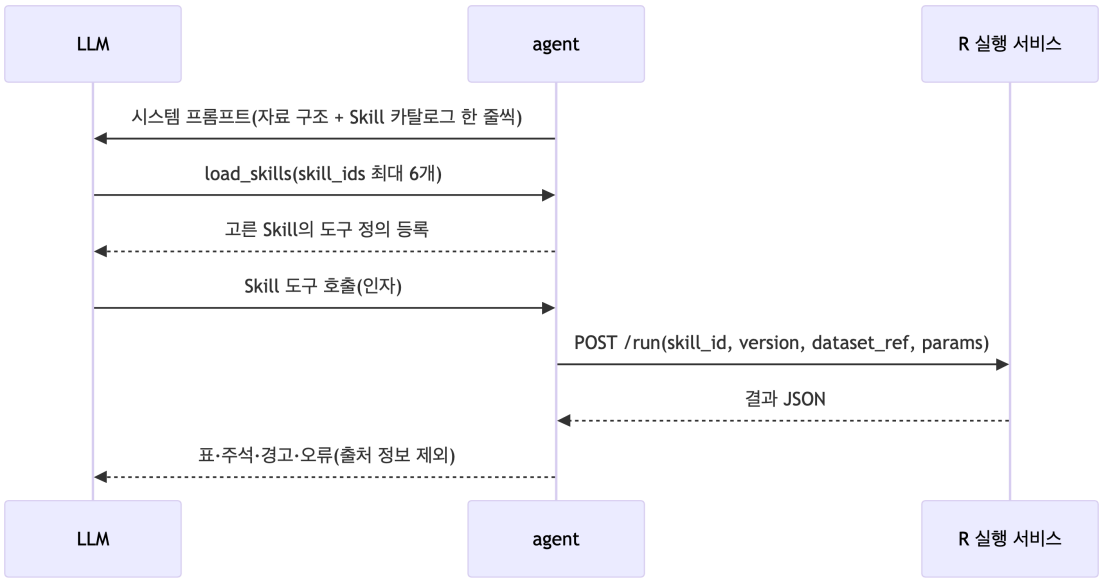
- R 실행 서비스는 요청을 한 번에 하나씩 처리(plumber 단일 스레드)
- 같은 R 프로세스에서 Skill 두 개를 동시에 돌리면 안 됨
  - 일부 wrapper가 기보유 패키지의 전역 데이터 조회 버그를 우회하려고 고정된 이름의 임시 전역 객체를 쓰기 때문(섹션 13)
- 처리량은 서비스 인스턴스 수로 늘림

관련 환경변수:

변수	기본값	읽는 곳
`AGENT_TOOL_MODE`	`routed`	agent/clinical_agent/runner.py
`CLINICAL_AGENT_ROOT`		r-executor/R/core.R, r-executor/R/load.R
`R_EXECUTOR_FIGURE_DIR`		r-executor/R/core.R
`R_EXECUTOR_HOST`	`127.0.0.1`	r-executor/serve.R
`R_EXECUTOR_ISOLATE`	`true`	r-executor/plumber.R
`R_EXECUTOR_PORT`	`8001`	r-executor/serve.R
`R_EXECUTOR_TIMEOUT_SEC`	`60`	r-executor/R/core.R, r-executor/R/isolate.R
`R_EXECUTOR_URL`	`http://localhost:8001`	agent/clinical_agent/config.py

2.5 agent의 2단계 도구 선택

- agent는 등록부의 Skill 하나하나를 function calling 도구로 바꿈(`tools.py`의 `skill_to_tool()`)
  - 도구 이름: Skill ID
  - 도구 설명: "기능명 — 분석 방법. 설명"
  - 도구 인자: 명세 `inputs`에서 `dataset_ref`를 뺀 나머지. 데이터셋은 세션이 정한 값을 실행할 때 붙임
- 그래서 LLM은 열 이름과 옵션만 고름. 어느 자료를 쓸지는 고를 수 없음
- Skill이 많으면 도구 정의 전체가 프롬프트를 크게 차지 → 통계실행 agent의 기본 방식(`AGENT_TOOL_MODE=routed`)은 두 단계로 나눔



- 처음에 LLM이 보는 것: 데이터셋 구조와 Skill 카탈로그뿐
- 카탈로그는 Skill마다 한 줄: `ID | 기능명 | 분석 방법 | 필수 인자 | 선택 인자`
  - 인자는 `이름(dtype)` 꼴. dtype이 없으면 `enum` 값 목록이나 형식 이름을 괄호에 넣음
- LLM은 이 줄들을 보고 `load_skills`로 Skill을 최대 6개까지 불러옴. 그때부터 그 Skill들의 전체 도구 정의(설명과 인자 schema)를 볼 수 있음
- `AGENT_TOOL_MODE=full`이면 처음부터 모든 Skill 도구를 넘김
- 두 방식 비교(2026-10-10, Skill 64종 상태, 시나리오 47개를 3회씩, `runner.py` 주석에 기록된 측정)
  - 정확도·일관성·예외 처리·오류 복구: 같음
  - 입력 토큰 중앙값: routed 17,584, full 63,228

이 구조 때문에 명세를 쓸 때 챙길 것:

- 기능명과 분석 방법이 카탈로그에서 서로 구별되어야 함
  - LLM은 카탈로그 한 줄만 보고 후보를 고름
  - 비슷한 Skill끼리는 `method`에 검정 이름이나 조건을 분명히 적음(예: “Welch two-sample t-test”와 “Mann-Whitney U 검정”)
- 설명(`description`)에 언제 쓰고 언제 다른 Skill을 쓰는지 적음
  - 불러온 뒤의 도구 설명에 그대로 들어감
  - 기존 명세는 “분포가 심하게 치우친 변수는 JST-TB1-MWU를 쓴다”처럼 대안 Skill ID를 적음
- 인자 설명이 곧 LLM이 받는 인자 안내
  - 도구 인자 schema에 들어가는 것: 인자 `description`과 `dtype`(“변수 형식: numeric” 꼴로 덧붙음), `values`, `min`·`max`, `min_items`·`max_items`
  - `default`는 들어가지 않음
  - agent 시스템 프롬프트가 요청하지 않은 선택 인자를 넣지 말라고 지시 → 선택 인자는 대개 R 쪽 기본값으로 실행

LLM에 넘기는 결과:

- Skill 결과는 LLM에 그대로 가지 않음
- privacy guard(`privacy.py`의 `tool_result_for_llm()`)가 `status`, `skill_id`, `tables`, `notes`, `warnings`, `error`만 남김. 그림은 이름과 제목만 넘김
- 출처 정보는 LLM에 넘기지 않고 Audit Trail과 trajectory에만 남김
- 결과표 하나가 200행을 넘으면 행 단위 자료가 섞였을 수 있다고 보고 LLM에 넘기지 않음(섹션 8.3)

3 ‘종’을 나누는 기준

3.1 분석 방법 하나 = 한 종

- Skill은 통계 분석 방법 하나를 단위로 셈. R 함수 개수가 아님
- 한 함수가 여러 분석 방법을 수행하면 Skill도 여러 개

  - 예: `jstable::CreateTableOneJS`는 인자에 따라 Welch t-test, Welch 분산분석, 카이제곱 검정, Fisher 정확검정, 순위합 검정 등을 수행
  - 현재 이 함수를 부르는 Skill은 14종(`CDS-ADSL-BASELINE`, `JSM-PS-IPTW`, `JSM-PS-MATCH`, `JST-TB1-ANOVA`, `JST-TB1-CHISQ`, `JST-TB1-DESC`, `JST-TB1-FISHER`, `JST-TB1-KW`, `JST-TB1-MWU`, `JST-TB1-NORMALITY`, `JST-TB1-PAIRWISE`, `JST-TB1-SMD`, `JST-TB1-STRATA2`, `JST-TB1-TTEST`)
- 반대로 여러 함수를 함께 불러야 하나의 분석 방법이 완성되면 그 묶음이 한 종
  - 그림을 그리는 함수와 그림에 표시한 수치를 계산하는 함수를 함께 부르는 Skill이 대부분 이 경우(`r_function`이 목록인 Skill이 62종)
- 같은 함수에서 나온 Skill이 별개의 종으로 인정되려면 **검증 규칙, 오류 처리, 결과 검수 기준 가운데 적어도 하나가 실제로 달라야 함**
- 인자 하나만 바꾼 복제는 종으로 세지 않음

Table 1 계열의 연속변수 비교 네 종을 명세에서 직접 뽑은 예:

ID	분석 방법(method)	그룹 범주 수 규칙	검수 기준의 기준값(acceptance.reference)
`JST-TB1-TTEST`	Welch two-sample t-test	최소 2, 최대 2 → `E_GROUP_COUNT`	변수마다 stats::t.test(x ~ group, var.equal = FALSE)의 p값을 독립 계산
`JST-TB1-ANOVA`	Welch one-way ANOVA	최소 3, 최대 20 → `E_GROUP_COUNT`	변수마다 stats::oneway.test(x ~ group, var.equal = FALSE)의 p값을 독립 계산
`JST-TB1-MWU`	Mann-Whitney U 검정 (Wilcoxon 순위합 검정)	최소 2, 최대 2 → `E_GROUP_COUNT`	변수마다 stats::wilcox.test(x ~ group, exact = FALSE, correct = FALSE)의 p값을 독립 계산
`JST-TB1-KW`	Kruskal-Wallis 검정	최소 3, 최대 20 → `E_GROUP_COUNT`	변수마다 stats::kruskal.test(x ~ group)의 p값을 독립 계산

- 네 Skill은 같은 함수를 부르지만 아래가 달라 각각 한 종
  - 검정 방법
  - 그룹 범주 수에 대한 규칙과 그 위반 때의 안내(예: t-test에서 3그룹이 들어오면 `JST-TB1-ANOVA`를 쓰라는 hint)
  - 독립 기준값을 계산하는 함수

3.2 애매한 경우의 판단 기준

- 실제로 Skill을 나누다 보면 애매한 경우가 생김

현재 등록부가 따르는 원칙:

- 표시 방식만 다르면 인자로 둠
  - 같은 계산에 그림 모양이나 요약 방식만 바뀌는 선택지는 별도 Skill로 만들지 않고 `enum·boolean·column_list` 인자로 받음
  - 예: `JSM-COR-PEARSON`의 추세선(`line`: 회귀선·LOESS·없음), `JST-TB1-DESC`의 `nonnormal`(중앙값 [IQR]로 요약할 변수 목록)
- 항상 함께 쓰고 검증·출력이 같은 방법은 한 Skill로 묶음
  - Kaplan-Meier 생존곡선과 log-rank 검정은 같은 자료·같은 검증 규칙으로 함께 보고 → `JSK-KM-LOGRANK` 한 종(분석 방법: Kaplan-Meier survival curve with log-rank test)
- 가정과 검증이 다르면 나눔
  - Pearson 상관과 Spearman 상관은 한 화면(jsmodule의 산점도 모듈)에서 나오지만 `JSM-COR-PEARSON`과 `JSM-COR-SPEARMAN` 두 종
  - Pearson: 정규성 점검 규칙(`cor_normality`, 경고), Fisher z 신뢰구간
  - Spearman: 동순위 점검 규칙(`cor_ties`, 경고), p값 계산 방식 열(`p_method`). 신뢰구간은 내지 않음
  - 결과표의 열과 검수 기준도 다름
- 가중·설계 기반 분석은 별도 Skill
  - 표본설계 가중치를 쓰는 분석(`JST-SVY-*`, `JSK-KM-SVY`)은 같은 이름의 일반 분석과 따로 둠
  - 이유: 설계 변수(`weights·strata·cluster·nest`) 입력과 설계 전용 검증 규칙(`svy_*`)이 붙음. 검정 자체도 설계 기반 검정으로 바뀜

- 출처 모듈의 버그는 옮기지 않음
  - Skill은 기보유 기능의 분석 방법을 옮길 뿐 그 구현의 버그까지 옮기지는 않음
  - 결과가 틀리는 동작은 wrapper에서 우회하고 테스트로 고정(섹션 13)
  - 출처 모듈이 임의 코드 실행(`eval(parse())`)으로 처리하던 기능(예: 테마 선택)은 Skill로 가져오지 않음

3.3 출처(`origin`)와 연결 함수(`r_function`)

- Skill마다 함수 두 가지를 적음
  - `origin`: 이 Skill이 바탕으로 삼은 기보유 기능
  - `r_function`: Skill이 실제로 부르는 함수

`origin` 적는 법:

- 기보유 함수 목록 `registry/inventory.csv`에 있는 `패키지::함수`여야 함(섹션 12)
- 분석 로직이 Shiny 모듈 안에 있는 경우(jsmodule 등): 모듈 자체를 출처로 적음(예: `jsmodule::rocModule`, `jsmodule::scatterServer`)
- 사내 Shiny 앱의 기능: `openstat_app::<앱 이름>` 꼴
- 두 기능 이상을 합쳐 만든 Skill: 목록으로 적음(현재 18종)

`r_function` 적는 법:

- wrapper가 부르는 함수를 적음. 출처 모듈이 내부에서 `pROC::roc`를 부른다면 `r_function`은 `pROC::roc`
- 출처 모듈 코드에서 확인한 호출 방식과 기본값을 따름
- 출처와 같은 패키지·같은 방법을 써야 함
- 등록부 생성기가 `r_function`의 함수가 설치된 패키지 안에 실제로 있는지 확인

3.4 ID 짓기

- ID 형식은 `<접두어>-<계열>-<방법>`. 정규식 `^[A-Z]{2,4}-[A-Z0-9]+-[A-Z0-9]+$`를 만족해야 함
- 파일 이름은 `<ID>.yaml`과 정확히 같아야 함
- 접두어는 기보유 출처 패키지를 따름

접두어	기보유 출처
JST	jstable
JSK	jskm
JSM	jsmodule
SPL	splineplot
STV	statgarten stove
SRB	statgarten soroban
BRD	statgarten board
LKT	statgarten shiny.likert
OSA	openstat_app
CDS	CDISC ADaM 연계(jstable·jskm 기반)

- 계열 코드는 같은 계열의 Skill끼리 맞춤

현재 쓰이는 계열 코드와 그 wrapper 파일:

계열 코드	Skill 수	접두어	wrapper 파일
REG	11	JST, SRB, STV	`ml.R`, `reg.R`, `stat2.R`
TB1	11	JST	`tb1.R`
SVY	10	JST	`svy.R`
CLS	9	STV	`ml.R`

계열 코드	Skill 수	접두어	wrapper 파일
META	8	OSA	`meta.R`
COX	6	JST	`cox.R`
ROC	5	JSM	`roc.R`
SS	5	OSA	`samplesize.R`
VIS	5	JSM, LKT	`stat2.R`, `vis.R`
EDA	4	BRD	`stat2.R`
KM	4	JSK	`km.R`
SURV	4	JSM	`km.R`, `survpred.R`
COR	3	JSM	`cor.R`
GEE	3	JST	`gee.R`
MIX	3	JST	`mix.R`
SPL	3	SPL	`spl.R`
SUB	3	JST	`sub.R`
PS	2	JSM	`ps.R`
STAT	2	SRB	`stat2.R`
TREE	2	SRB	`stat2.R`
ADSL	1	CDS	`cdisc.R`
ADTTE	1	CDS	`cdisc.R`
CLU	1	STV	`ml.R`

- wrapper 함수 이름은 `skill_<계열>_<방법>`(소문자). 명세 `implementation`에 적음. 정규식은 `^skill_[a-z0-9_]+$`
- 계열 코드와 wrapper 파일이 꼭 일대일은 아님
  - 위 표처럼 `REG` 계열은 jstable 회귀(`reg.R`), statgarten 머신러닝 회귀(`ml.R`), 회귀나무(`stat2.R`)가 함께 씀
- 새 Skill은 같은 패키지·같은 계열의 기존 파일에 넣음. 새 계열이면 파일을 새로 만듦

4 명세(YAML) 필드 레퍼런스

- 명세 한 개 = YAML 매핑 하나
- schema(`registry/schema/skill.schema.json`, JSON Schema draft-07)가 정한 것
  - 최상위 필드 17개가 모두 필수
  - 정의되지 않은 필드는 허용하지 않음
- 아래는 가장 단순한 명세인 `JST-TB1-TTEST`의 원문. 이 장의 설명은 이 예를 기준으로 함

```yaml
id: JST-TB1-TTEST
name: 연속변수 2그룹 비교표
method: Welch two-sample t-test
category: 기술통계·그룹비교
description: >-
  연속변수의 평균을 두 그룹 간 비교한다. 그룹별 평균 ± 표준편차와 Welch t-test p값을 Table 1 형식으로 제
시한다.
  등분산을 가정하지 않는다. 분포가 심하게 치우친 변수는 JST-TB1-MWU(Mann-Whitney U)를 쓴다.
r_function: jstable::CreateTableOneJS
origin: jstable::CreateTableOneJS
implementation: skill_tb1_ttest
version: 0.1.0
inputs:
  dataset: {type: dataset_ref, required: true, description: "분석 데이터셋 ID"}
```

```
  vars: {type: column_list, dtype: numeric, required: true, min_items: 1, description: "비교할
연속변수"}
  group: {type: column, dtype: categorical, required: true, description: "비교 그룹 변수(범주 2
개)"}
validation:
  - {rule: distinct_columns, columns: [vars, group], error: E_ARG_CONFLICT}
  - {rule: n_levels, column: group, min: 2, max: 2, error: E_GROUP_COUNT}
  - {rule: not_constant, columns: [vars], error: E_CONSTANT_VARIABLE}
  - {rule: max_missing_rate, columns: [vars, group], max: 0.2, error: W_MISSING_HIGH}
preconditions:
  - {rule: min_group_size, column: group, min: 2, error: E_SMALL_GROUP}
  - {rule: min_rows, columns: [vars, group], min: 4, error: E_TOO_FEW_ROWS}
outputs:
  tables:
    table1:
      description: 변수별 그룹 평균 ± SD와 p값
      columns: [variable, level, "<그룹 수준>", p, test]
errors:
  E_COL_NOT_FOUND: {action: ask_user, message: 데이터에 없는 변수, hint: 유사 변수명을 제안하고 사용
자에게 확인}
  E_TYPE_MISMATCH: {action: ask_user, message: 연속변수가 아닌 변수가 포함됨, hint: "범주형 변수는
JST-TB1-CHISQ 또는 JST-TB1-FISHER를 사용"}
  E_ARG_CONFLICT: {action: ask_user, message: 그룹 변수가 비교 변수에 함께 지정됨, hint: 그룹 변수를
비교 변수 목록에서 제외}
  E_GROUP_COUNT: {action: ask_user, message: 그룹 범주가 2개가 아님, hint: "3그룹 이상은 JST-TB1-
ANOVA를 사용"}
  E_CONSTANT_VARIABLE: {action: block, message: 값이 하나뿐인 변수는 비교할 수 없음, hint: 해당 변수
를 제외}
  E_SMALL_GROUP: {action: block, message: 그룹별 관측 수가 2 미만, hint: 다른 그룹 변수를 선택}
  E_TOO_FEW_ROWS: {action: block, message: 분석 가능한 행 수 부족, hint: 결측을 확인하거나 변수 수를
줄임}
  W_MISSING_HIGH: {action: warn, message: 결측 비율이 20%를 넘음, hint: 완전사례 분석임을 결과 해석에
명시}
acceptance:
  reference: 변수마다 stats::t.test(x ~ group, var.equal = FALSE)의 p값을 독립 계산
  criterion: "결과표 p값이 기준 p값을 소수 셋째 자리로 반올림한 값과 일치(기준값 < 0.001이면 '<0.001')"
examples:
  - dataset: colon_recur
    request: 성별에 따라 나이와 양성 림프절 수 평균이 다른지 비교해줘
    params: {vars: [age, nodes], group: sex}
tests: r-executor/tests/testthat/test-JST-TB1-TTEST.R
```

4.1 최상위 필드

필드	형식(schema)	뜻
`id`	문자열, `^[A-Z]{2,4}-[A-Z0-9]+-[A-Z0-9]+$`	고유 ID. 파일 이름(`<id>.yaml`)과 같아야 함
`name`	문자열, 2자 이상	기능명(한국어). 카탈로그와 등록부에 그대로 나옴
`method`	문자열, 2자 이상	통계 분석 방법 이름. 검정·모형 이름을 구체적으로(예: “Welch two-sample t-test”)
`category`	문자열	분석 범주. 값 목록은 schema로 제한하지 않음
`description`	문자열, 10자 이상	언제 쓰는 분석인지, 가정, 해석 때 주의점, 대신 쓸 Skill

필드	형식(schema)	뜻
`r_function`	`패키지::함수` 문자열 또는 그 목록(1개 이상)	실제로 부르는 R 함수. 패턴 `^[A-Za-z0-9.]+::[A-Za-z0-9._]+$`
`origin`	`패키지::함수` 문자열 또는 그 목록(1개 이상)	기보유 기능 출처. `inventory.csv`에 있어야 함. 패턴은 `-`와 `_`도 허용
`implementation`	문자열, `^skill_[a-z0-9_]+$`	wrapper 함수 이름
`version`	문자열, `^[0-9]+\.[0-9]+\.[0-9]+$`	Skill 버전(섹션 12)
`inputs`	매핑, `dataset` 필수	입력 인자 정의(섹션 4.2)
`validation`	규칙 목록	데이터 검증 규칙(섹션 4.3)
`preconditions`	규칙 목록	실행 조건. 규칙이 없으면 빈 목록 `[]`을 적음
`outputs`	매핑, `tables` 필수	출력 표와 그림(섹션 4.4)
`errors`	매핑, 1개 이상	오류 코드별 처리 방식(섹션 4.5)
`acceptance`	`reference`, `criterion`	결과 검수 기준(섹션 4.6)
`examples`	목록, 1개 이상	예제 데이터셋·자연어 요청·인자(섹션 4.7)
`tests`	문자열, `^r-executor/tests/testthat/test-.+\.R$`	테스트 파일 경로(저장소 루트 기준)

- `category`
  - 자유 문자열. 단 등록부 표와 카탈로그에서 묶음 단위로 쓰이므로 기존 값을 그대로 씀
  - 현재 쓰이는 값: “생존분석”, “기술통계·그룹비교”, “서베이·표본설계”, “머신러닝(분류)”, “메타분석”, “머신러닝(회귀)”, “진단·예측성능”, “표본수 계산”, “시각화”, “탐색적 자료분석”, “회귀분석”, “다변량 분석”, “반복측정·GEE”, “반복측정·혼합모형”, “상관분석”, “서브그룹 분석”, “스플라인·용량-반응”, “CDISC 연계”, “인과성분석”, “기술통계”, “머신러닝(군집)”, “설문 분석”
  - 새 범주는 기존 범주로 묶을 수 없을 때만 만듦
- `description`
  - YAML 접힘 블록(`>-`)으로 여러 줄에 걸쳐 쓰면 한 문단으로 합쳐짐
  - agent는 Skill을 불러온 뒤 이 문장을 도구 설명으로 봄 → 쓰임과 함께 “이럴 때는 이 Skill 대신 무엇을 쓴다”를 적음(섹션 2)

4.2 입력 인자(`inputs`)

- `inputs`는 인자 이름을 키로, 인자 정의를 값으로 하는 매핑
- `dataset` 인자는 항상 있어야 함. 정의는 `{type: dataset_ref, required: true, description: "분석 데이터셋 ID"}`로 고정
- 데이터가 필요 없는 Skill(표본수 계산 등)도 `dataset`을 두고 빈 자료 `no_data`를 예제로 씀

인자 정의에 쓸 수 있는 키(이것이 전부. 그 밖의 키는 schema 오류):

키	필수	형식	뜻
`type`	예	`dataset_ref`, `column`, `column_list`, `enum`, `integer`, `number`, `boolean`, `string` 가운데 하나	인자 형식
`required`	예	`true` 또는 `false`	필수 인자 여부

키	필수	형식	뜻
`description`	예	문자열	인자 설명. LLM 도구 인자 설명과 등록부에 그대로 쓰임
`dtype`	아니오	`numeric`, `categorical`, `binary`, `any` 가운데 하나	열 인자가 가리키는 변수의 형식
`default`	아니오	아무 값	인자가 빠졌을 때 채울 값
`min_items`	아니오	0 이상 정수	`column_list`의 최소 개수
`max_items`	아니오	1 이상 정수	`column_list`의 최대 개수
`values`	아니오	목록	`enum`의 허용 값
`min`, `max`	아니오	숫자	`integer`·`number`의 하한·상한(같은 값 포함)

현재 등록부에서 형식별로 쓰인 수:

type	입력 인자 수	쓰는 Skill 수
`dataset_ref`	106	106
`column`	259	95
`column_list`	115	79
`enum`	54	43
`integer`	124	55
`number`	69	44
`boolean`	55	36
`string`	37	28

4.2.a 형식별 판정

실행 틀의 `validate_inputs()` 판정 순서:

1. 명세에 없는 인자가 들어왔는지 봄. 하나라도 있으면 그 이름들을 담아 `E_UNKNOWN_PARAM`으로 막음
2. 명세에 적힌 순서대로 인자를 하나씩 봄
   - 처음 걸린 인자에서 멈춤 → 필수 인자를 앞에 두는 편이 사용자에게 돌아가는 오류가 자연스러움

빠진 인자:

- 값이 없거나(`null`) 길이가 0(빈 목록 `[]` 포함)이면 빠진 것으로 봄
  - `required: true`이면 `E_MISSING_PARAM`
  - 아니면 `default`가 있을 때 그 값을 채움. 없으면 인자를 비워 둔 채 넘어감
- **기본값은 형식 검사를 거치지 않음**
  - `enum`의 기본값이 `values`에 없거나 `integer` 기본값이 범위를 벗어나도 실행 틀과 등록부 점검 어느 쪽도 잡지 못함 → 명세를 쓸 때 직접 맞춤

값이 있을 때 형식별로 확인하는 것:

- `column`: 문자열 하나여야 함(아니면 `E_INVALID_VALUE`). 그다음 열 이름 점검(아래)
- `column_list`
  - 목록을 펼친 결과가 문자열이어야 함(아니면 `E_INVALID_VALUE`)
  - 개수가 `min_items`보다 적으면 `E_MISSING_PARAM`, `max_items`보다 많으면 `E_INVALID_VALUE`
  - 개수가 맞으면 원소마다 열 이름 점검
  - JSON으로 문자열 하나만 와도(`"age"`) 길이 1 목록으로 받아들임
- `enum`: 값이 `values`에 있어야 함. 아니면 `E_INVALID_VALUE`이고 메시지에 허용 값이 나열됨

- integer·number
  - 숫자 하나여야 함. integer는 정수 값이어야 함(JSON의 5와 5.0 모두 통과)
  - min·max를 벗어나면 E_INVALID_VALUE
- boolean: true·false 하나여야 함. 문자열 "true"는 통과하지 못함
- string: 문자열 하나여야 함. 내용은 검사하지 않음
  - 문자열을 숫자 목록 등으로 해석해야 하면 wrapper나 계열 규칙이 직접 파싱
  - 어떤 경우에도 코드로 평가하지 않음(예: roc.R의 roc_parse_cutoffs()는 쉼표로 나눠 as.numeric()만 함)
- dataset_ref: dataset 인자 전용
  - 실행 틀은 이 정의를 인자 검증에서 빼고 agent도 도구 인자에서 뺌
  - 다른 인자에 이 형식을 쓰지 않음

열 이름 점검(column·column_list 공통) 세 단계:

1. 이름이 R 문법에 맞아야 함(make.names(x) == x)
   - 이유: 실행 틀과 wrapper가 모형식을 reformulate()로 만듦
   - 공백이나 괄호가 든 이름은 E_INVALID_VALUE. 안내는 "변수명은 영문으로 시작하고 영문·숫자·.·_만 쓰도록 바꿔서 다시 올림"
2. 데이터에 그 열이 있어야 함
   - 없으면 E_COL_NOT_FOUND
   - 데이터 열 이름 가운데 비슷한 이름(근사 문자열 일치, 거리 0.3 이내, 최대 3개)이 있으면 "유사 변수명: …"을 안내로 붙임
3. 열의 값이 dtype에 맞아야 함. 맞지 않으면 E_TYPE_MISMATCH

- dtype 판정 기준은 아래 표. dtype이 없으면 any와 같음

dtype	통과 조건
numeric	R에서 숫자형(integer·double). factor는 숫자로 코딩되어 있어도 통과하지 못함
categorical	factor·문자·논리형이거나, 숫자형이면서 결측 제외 값이 모두 정수이고 서로 다른 값이 10개 이하
binary	형식과 관계없이 결측 제외 서로 다른 값이 정확히 2개
any	항상 통과

- categorical이 숫자형도 허용하는 이유: 0/1/2처럼 정수로 코딩된 범주 변수를 받아들이려고
  - 그래서 정수 값이 10종류 이하인 숫자 열은 numeric과 categorical을 모두 통과
- 범주 수 자체를 제한하려면 dtype 대신 n_levels·max_levels 규칙을 씀

4.2.b agent 도구 인자로 변환

- agent는 인자 정의를 JSON Schema로 바꿔 LLM 도구 인자로 넘김(tools.py의 _input_schema())
- 형식별 변환
  - column → 문자열
  - column_list → 문자열 배열(minItems·maxItems 포함)
  - enum → values를 허용 값으로 하는 문자열
  - integer·number → minimum·maximum이 붙은 숫자
  - boolean → 불리언
  - 그 밖 → 문자열
- 설명에는 인자 description 뒤에 "(변수 형식: numeric)"처럼 dtype이 덧붙음
- required: true인 인자가 도구의 필수 인자가 됨. 정의에 없는 인자는 허용하지 않음
- default는 넘어가지 않음

4.3 데이터 검증 규칙과 실행 조건

- validation과 preconditions는 규칙 객체의 목록
- 규칙 객체의 필수 키: rule(규칙 이름), error(위반 때의 오류 코드, ^[EW]_[A-Z0-9_]+$). 나머지 키는 규칙마다 정해진 인자

```yaml
validation:
  - {rule: n_levels, column: group, min: 2, max: 2, error: E_GROUP_COUNT}
  - {rule: max_missing_rate, columns: [vars, group], max: 0.2, error: W_MISSING_HIGH}
preconditions:
  - {rule: min_rows, columns: [vars, group], min: 4, error: E_TOO_FEW_ROWS}
```

- 규칙 인자 가운데 `column`, `columns`, `group`은 **데이터 열 이름이 아니라 Skill 입력 인자 이름**
  - 위 예의 `column: group`은 "입력 인자 `group`에 사용자가 지정한 열"을 뜻함
  - `columns`에 `column_list` 인자를 넣으면 그 인자의 열 전체가 대상
  - 선택 인자를 가리키는 규칙은 사용자가 그 인자를 주지 않았으면 대개 아무것도 검사하지 않고 통과(예: 그룹 없이 실행한 상관분석의 `n_levels`)
- 규칙 이름은 `core.R`의 공통 규칙이거나 계열 파일에 정의된 계열 전용 규칙이어야 함. 규칙마다 필수 키가 있음
  - 목록과 동작: 섹션 5
- 등록부 점검이 문제로 잡는 것: 알 수 없는 규칙 이름, 빠진 필수 키, 명세 `errors`와 공통 오류 어디에도 없는 오류 코드

두 목록의 구분:

- 실행 방식은 같음(섹션 2). 구분은 의미에 따름
- 현재 등록부의 구분
  - `validation`: 자료와 인자가 분석에 맞는 형태인지. 열 사이 중복, 코딩(0/1), 범주 수, 결측 비율, 값 범위
  - `preconditions`: 추정이 성립할 만큼 자료가 충분한지. 완전사례 행 수, 그룹별 관측 수, 사건 수, 군집 수

많이 쓰이는 규칙의 배치(등록부에서 셈):

규칙	validation에 쓰인 수	preconditions에 쓰인 수
`distinct_columns`	106	0
`max_missing_rate`	90	0
`min_rows`	0	81
`not_constant`	77	0
`n_levels`	53	0
`max_levels`	33	0
`min_group_size`	0	27
`positive_values`	25	0
`value_in_column`	23	0
`max_factor_levels`	18	0
`subset_columns`	18	0
`binary_01`	17	0

- `validation`이 먼저 실행됨 → 같은 자료에 형식 문제와 표본 부족이 함께 있으면 형식 문제가 먼저 보고됨
- 경고(`warn`) 규칙은 실행을 멈추지 않으므로 목록 어디에 두어도 실행 여부는 같음
  - 단 앞쪽의 차단 규칙이 먼저 걸리면 경고는 결과에 남지 않음(오류 결과에는 `warnings`가 없음)

4.4 출력(`outputs`)

```yaml
outputs:
  tables:
    correlation:
      description: "전체와 그룹별 결측 없는 쌍 수 n, Pearson r, 95% CI, t, 자유도, p값"
      columns: [variable1, variable2, group, "n", r, CI_low, CI_high, t, df, p]
  figures: [scatter]
```

- `tables`: 결과표 이름을 키로 하는 매핑. 1개 이상 있어야 함

  - 표마다 `description`은 필수, `columns`는 선택(현재 모든 표에 적혀 있음)
- `figures`: 그림 이름 목록
- 표 이름과 그림 이름은 wrapper가 돌려주는 결과의 이름과 정확히 같아야 함
- 현재 등록부의 표는 모두 276개, Skill 하나에 최대 5개

`columns` 적는 법:

- 결과표의 열 이름을 순서대로 그대로 적음
- 시험 실행기가 이 목록으로 결과 형식을 기계 판정(섹션 15) → 표기법이 정해져 있음

표기:

- 그대로 적은 이름
  - 그 이름의 열이 꼭 있어야 함
  - `n`처럼 YAML 1.1에서 불리언으로 읽히는 이름은 따옴표로 감쌈(섹션 4.9)
- `<설명>`
  - 자료에 따라 이름이 정해지는 열. 그룹 수준이 열 이름이 되는 Table 1이 대표적
  - 꺾쇠 부분은 아무 문자열과 맞음. 이 표기 하나가 열 1개 이상과 대응
  - 현재 쓰이는 예: `<그룹 수준>`, `actual_<비사건 범주>`, `actual_<사건 범주>`, `events (<결과변수> = <사건 범주>)`, `mean of <결과변수>`, `<Overall 또는 그룹 수준>`
  - 이름 일부만 자리표시로 둘 수도 있음(예: `p(<수준>vs<수준>)`)
- `A | B`
  - 둘(이상) 가운데 하나. 모형 종류에 따라 열 이름이 바뀌는 회귀 결과에 씀
  - 가르는 기호는 세로 막대 앞뒤에 공백을 하나씩 둔 꼴(`A | B`처럼 씀)
  - 현재 쓰이는 예: `OR | Coefficient | RR`, `log_<효과크기> | <효과크기>`
- 끝의 `?`
  - 인자나 자료에 따라 없을 수 있는 조건부 열(예: 신뢰구간을 요청했을 때만 나오는 열)
  - 현재 쓰이는 예: `subgroup?`, `lower 95% CI?`, `upper 95% CI?`, `adj. coeff(95%CI)?`, `adj. P value?`, `coeff(95%CI)?`

판정:

- 기준은 "필수 열이 모두 있고, 명세에 없는 열이 없는가"
  - wrapper가 디버깅용 열 하나를 더 내거나 열 이름 철자가 한 글자만 달라도 부적합
  - 꺾쇠 표기는 판정에서 정규식 `.+`로 바뀜 → 열 이름 자체에 `<`와 `>`가 함께 들어가면 자리표시로 해석되므로 주의
- 조건부로 생기는 표(그룹이 있을 때만 나오는 표 등)는 명세에 이름만 적어 두면 됨. 실제 결과에 없어도 부적합으로 보지 않음
- 반대로 명세에 없는 이름의 표가 결과에 있으면 부적합
- 그림은 PNG로 저장되고 결과에는 메타데이터만 들어감(섹션 7)
- `figures`에 적은 그림은 테스트에서 `expect_figure(res, "<이름>")`으로 확인

4.5 오류 처리(errors)

```yaml
errors:
  E_COL_NOT_FOUND: {action: ask_user, message: 데이터에 없는 변수, hint: 유사 변수명을 제안하고 사용자에게 확인}
  E_GROUP_COUNT: {action: ask_user, message: 그룹 범주가 2개가 아님, hint: "3그룹 이상은 JST-TB1-ANOVA를 사용"}
  E_CONSTANT_VARIABLE: {action: block, message: 값이 하나뿐인 변수는 비교할 수 없음, hint: 해당 변수를 제외}
  W_MISSING_HIGH: {action: warn, message: 결측 비율이 20%를 넘음, hint: 완전사례 분석임을 결과 해석에 명시}
```

- 키: 오류 코드(`E_` 또는 `W_`로 시작, 대문자·숫자·밑줄)
- 값: `action`·`message`·`hint` 세 키가 모두 있는 객체
- `action`은 `block`(차단), `ask_user`(사용자 확인), `warn`(경고 후 진행) 가운데 하나
- 접두어와 `action` 맞추기
  - 코드가 `W_`로 시작하면 `warn`, `E_`로 시작하면 `block`이나 `ask_user`

- 실행 틀은 접두어가 아니라 `action`만 보고 동작을 정함 → 이 대응은 명세 작성자가 지킴(현재 등록부에서 어긋난 정의는 0개)

명세에 적을 코드:

- 이 Skill의 규칙이 쓰는 코드 전부
- 입력 검증에서 나는 공통 코드(`E_COL_NOT_FOUND`, `E_TYPE_MISMATCH`, `E_INVALID_VALUE` 등)도 이 Skill에 맞는 문구로
- 명세에 없는 공통 코드는 실행 틀의 기본 정의가 쓰임
- 공통 코드 목록, `message`·`hint`가 실제 오류에서 조립되는 방식, action별 agent 동작: 섹션 6
- 좋은 `hint`는 사용자가 바로 할 수 있는 일을 적음. 다른 Skill로 가야 하면 그 Skill ID를 적음(위 예의 `E_GROUP_COUNT`)

4.6 결과 검수 기준(`acceptance`)

```yaml
acceptance:
  reference: 변수마다 stats::t.test(x ~ group, var.equal = FALSE)의 p값을 독립 계산
  criterion: "결과표 p값이 기준 p값을 소수 셋째 자리로 반올림한 값과 일치(기준값 < 0.001이면 '<0.001')"
```

- `reference`: 결과값을 **기보유 패키지를 거치지 않고** 계산하는 방법
  - 기보유 패키지 함수를 다시 불러 같은 값이 나오는지 보는 것은 검증이 아님
  - 기존 명세는 `stats`·`survival` 함수를 직접 부르거나 공식을 그대로 계산(예: Pearson 상관의 r, Fisher z 신뢰구간, t 통계량을 공식으로)
- `criterion`: 무엇을 몇 자리에서 비교하는지, p값 경계 표기(`<0.001`)를 어떻게 다루는지, 그림이 있으면 PNG 생성과 해시 일치까지
- 테스트(섹션 9)는 이 기준을 코드로 옮긴 것이어야 함

4.7 예제(`examples`)

```yaml
examples:
  - dataset: colon_recur
    request: 성별에 따라 나이와 양성 림프절 수 평균이 다른지 비교해줘
    params: {vars: [age, nodes], group: sex}
```

- 예제마다 필수: `dataset`(허용 목록의 데이터셋 ID), `request`(자연어 요청 예시), `params`(그대로 실행되는 인자)
- 등록부 생성기: 첫 번째 예제를 실제로 실행해 성공해야 통과
- 시험 실행기: 모든 예제를 실행하고 첫 번째 예제를 바탕으로 오류 입력을 만듦(섹션 15)

첫 예제를 고르는 기준:

- 경고 없이 또는 의도한 경고만 내며 성공
- 필수 인자를 모두 쓰고 열 인자를 하나 이상 씀(없는 변수 주입 시험의 바탕)
- 숫자형 열 인자가 있으면 같은 데이터셋에 그 자리에 넣을 범주형 변수가 있음(형식 불일치 주입 시험)
- 실행 시간이 짧음

`request`:

- 등록부 상세 표(`registry.xlsx`)에 들어감
- 자연어 경로로 같은 Skill이 실행되는지도 이 문장으로 확인
- 사용자가 실제로 쓸 법한 한국어 문장으로 씀

4.8 테스트 파일(`tests`)과 버전(`version`)

- `tests`는 `r-executor/tests/testthat/test-<ID>.R`로 씀
  - 등록부의 '테스트 결과' 칸은 JUnit 결과에서 파일 이름에서 얻은 묶음 이름(`test-`와 `.R`을 뗀 부분)으로 Skill을 찾음
  - 파일 이름이 ID와 다르면 테스트가 통과해도 등록부에는 "미측정"으로 나옴
- `version`은 `0.1.0`처럼 점 두 개가 있는 문자열
  - 새 Skill은 `0.1.0`으로 시작
  - 버전을 올리는 시점과 효과: 섹션 12

4.9 YAML 1.1 함정

- 명세는 R `yaml` 패키지로 읽음. 이 패키지는 YAML 1.1 규칙을 따름 → 몇몇 평문 값이 문자열이 아닌 다른 형식으로 바뀜

- 아래 표: 2026-10-12에 저장소 라이브러리(`.r-lib`)의 `yaml` 2.3.10으로 직접 읽어 확인한 결과

쓴 것	읽힌 값	대처
키 `y:`, `n:`	키 이름이 `"TRUE"`, `"FALSE"`로 바뀜	입력 인자·규칙 키 이름으로 쓰지 않음 (입력 이름 `y` 금지)
키 `n:`과 `no:`를 한 매핑에	둘 다 `FALSE`가 되어 "Duplicate map key" 읽기 오류	같은 이유로 쓰지 않음
목록 값 `[variable, n]`	두 번째 원소가 논리값 `FALSE`	`[variable, "n"]`처럼 따옴표
목록 값 `yes`, `on`, `y` / `off`	`TRUE` / `FALSE`	따옴표
`version: 0.1, 0.10`	숫자 0.1	`0.1.0`처럼 점 두 개(문자열이 됨)
`01`	정수 1	앞자리 0이 의미 있으면 따옴표

`y`를 입력 인자 이름으로 쓴 경우:

- 특히 찾기 어려움. 매핑의 키 `y:`는 문자열 `"TRUE"`로, 규칙 목록 안의 값 `y`는 논리값 `TRUE`로 읽혀 서로 어긋남
- 확인 예(2026-10-12): `JSM-COR-PEARSON` 명세에서 `var2`를 모두 `y`로 바꿔 읽음
  - 입력 정의의 이름이 `"TRUE"`가 됨
  - `distinct_columns` 규칙의 `columns: [var1, y, group]`에서는 논리값 `TRUE`가 인자 목록의 첫 원소(`var1`)를 가리킴
  - 그 결과 예제 인자로 검증하면 엉뚱하게 `E_ARG_CONFLICT`("같은 변수가 중복 지정됨: bili")로 막힘
  - `y`라는 이름으로 인자를 넘기면 `E_UNKNOWN_PARAM`
- 규칙이 그 인자를 참조하지 않으면 등록부 점검도 잡지 못하고 `registry.json`에 인자 이름이 `"TRUE"`로 기록됨(섹션 11의 실패 예시에서 확인)

따옴표가 필요한 문자열:

- 한 줄 매핑(`{...}`) 안에서 문자열에 `[ ] { } , :`가 들어가면 YAML 구조로 해석됨 → 따옴표로 감쌈
  - 예: 기존 명세의 `message: "var1·var2는 숫자형, group은 범주형이어야 함"`은 쉼표 때문에 따옴표를 씀
- `#`(공백 뒤 샵)은 주석 시작 → 문자열 안에 쓰려면 역시 따옴표 필요

`registry.json`의 목록과 값 하나:

- 등록부 생성기는 schema 검증 때와 같이 YAML 목록을 원소가 하나여도 JSON 배열로 기록
  - 예: `figures: [scatter]` → `"figures": ["scatter"]`
- YAML에 목록이 아니라 값 하나로 쓴 항목은 문자열로 기록
  - `r_function`·`origin`은 schema가 문자열과 목록을 모두 허용 → 함수가 하나인 명세는 대개 `r_function: jstable::cox2.display`처럼 값 하나로 씀
  - 그래서 `registry.json`을 읽는 코드는 `r_function`·`origin`을 문자열과 배열 모두 받아야 함
  - 규칙 키(`columns` 등)도 `columns: group`처럼 값 하나로 쓰면 문자열. 규칙은 R 실행 서비스가 YAML을 직접 읽어 쓰므로 실행에는 영향 없음

5 검증 규칙 레퍼런스

5.1 규칙 함수 작성 규약

- 규칙은 `rule_funs` 목록에 이름으로 등록된 R 함수
- 모든 규칙의 시그니처가 같음

```
rule_funs$<이름> <- function(r, data, p) {
  # r    : 명세의 규칙 객체(list). r[["column"]], r[["min"]] 등
  # data : 데이터셋 전체(data.frame). 완전사례로 줄이지 않은 상태
  # p    : 입력 검증을 통과하고 기본값이 채워진 인자 목록
  # 통과하면 NULL, 위반하면 위반 내용을 설명하는 문자열 하나를 돌려준다
}
rule_required_keys$<이름> <- c("column", "min")   # 명세에 꼭 있어야 하는 키
```

- 실행 틀(`run_rules()`)은 규칙이 돌려준 문자열을 오류 메시지 뒤에 붙임
  - 예: `n_levels`가 `"rx의 범주 수 3 (최대 2)"`를 돌려주고 명세의 `E_GROUP_COUNT` 메시지가 "그룹 범주가 2개가 아님"이면 → 사용자가 받는 메시지는 "그룹 범주가 2개가 아님: rx의 범주 수 3 (최대 2)"
- 경고(`warn`) 규칙은 `"W_MISSING_HIGH: 결측 비율 20% 초과: …"`처럼 코드와 내용을 붙여 `warnings`에 넣음

규칙 함수를 쓸 때 알아 둘 점:

- 규칙 객체의 열 관련 키는 입력 인자 이름 → 실제 열 이름은 `p[[r[["column"]]]]`처럼 한 번 더 찾음
  - 여러 인자를 한꺼번에 펼칠 때는 `core.R`의 `param_columns(p, r[["columns"]])`를 씀
  - 지정되지 않은 선택 인자는 `NULL`이라 자연히 빠짐
- `data`는 자료 전체
  - 공통 규칙 가운데 `min_rows`만 지정한 열의 완전사례를 셈. 나머지(`n_levels`, `min_group_size`, `max_missing_rate` 등)는 자료 전체에서 셈
  - 분석은 완전사례로 하는데 완전사례 기준의 사건 수나 쌍 수를 봐야 한다면 계열 전용 규칙을 만듦(예: `cor_pairs`, `km_min_events`)
- 인자는 반드시 `r[["키"]]`, `p[["이름"]]`으로 읽음
  - `$`는 부분 일치 → `column` 키가 없을 때 `r$column`이 `columns`를 집어 옴(실제로 겪은 오류. 섹션 8.8)
- 반환 문자열에 쓰는 값은 정해져 있음(섹션 5.2)
- 규칙은 데이터를 바꾸지 않음. 판정만 함

5.2 메시지에 쓰는 값

- 규칙이 돌려준 문자열은 사용자에게 그대로 보이고 LLM에도 전달됨
  - wrapper가 `skill_error()`에 넣는 상세 문자열도 같음
- 그래서 데이터셋 구조(`GET /datasets/{ref}/schema`, 섹션 14)가 내는 것보다 많은 값을 담지 않음
  - 업로드 자료의 데이터셋 구조는 최솟값·최댓값을 내지 않음. 대개 한 사람의 값이기 때문
  - 메시지에 관측 범위를 넣으면 이 제한을 우회하게 됨
  - 공개 데이터셋에도 같은 기준을 씀 → 같은 위반이면 자료 출처와 관계없이 메시지가 같음

담는 것:

- 변수 이름, 건수, 사용자가 준 인자 값, 규칙 기준값
  - 예: `asah` 자료에서 `JSM-ROC-CUTOFF`에 `cutoff = 5`를 주면 "지정한 절단값이 표지자의 관측 범위 밖: 절단값 5이(가) s100b의 관측 범위 밖"

담지 않는 것:

- 숫자형 변수의 관측값: 최솟값·최댓값·관측 범위·분위수·개별 행의 값
  - 범위를 벗어난 값은 "관측 범위 밖"처럼 판정만 씀. 실제 범위는 LLM이 데이터셋 구조에서 봄(공개 자료는 최솟값·최댓값, 업로드 자료는 사분위수)
  - 분위수로 정하는 값(예: 스플라인 매듭)이 겹치면 값 대신 겹친 분위수 위치를 씀(예: "5%·27.5% 분위수가 같음")
- 대상자 ID, 군집 ID 같은 식별자 값 → 개수만 씀(예: "USUBJID가 2행 이상인 대상자 N명")

범주 값을 보여 줄 때:

- `core.R`의 `msg_levels(x, vals)`를 씀
  - 데이터셋 구조에 범주 목록이 나오는 열(숫자형이 아니고 범주 30개 이하)이면 값을 앞에서 `n`개(기본 5개)까지 붙임
  - 그 밖의 열(숫자형, 범주가 많은 열)이면 빈 문자열 → 호출하는 쪽이 개수를 함께 씀
  - 업로드 자료(`up_…`)를 실행·검증하는 중이면 데이터셋 구조와 같은 기준을 더함: 날짜·시각 열은 빈 문자열, 관측 수가 5 미만인 범주 값은 뺌
    - 실행 틀(`run_skill`·`validate_skill`)이 데이터셋 문맥을 정해 두므로 Skill wrapper는 따로 할 일 없음
  - 길어질 수 있는 목록은 이렇게 앞부분만 보임

```
# 데이터셋 구조가 범주 목록을 내는 범주 수 상한(넘으면 개수만 냄)
schema_max_levels <- 30

# 업로드 자료에서 범주 값을 내는 최소 관측 수. 이보다 적은 행에만 있는 값(이름·ID·자유 텍스트처럼 한 사람을
# 가리킬 수 있는 값)은 구조·메시지에 쓰지 않고 개수만 센다. agent의 소표본 셀 기준(PRIVACY_MIN_CELL, 기본
5)과 같다
upload_min_level_n <- function() max(1L, as.integer(Sys.getenv("PRIVACY_MIN_CELL", "5")))
```

```
# 열 x의 범주 값을 구조·메시지에 낼 수 있는지(FALSE면 개수만 낸다)
levels_exposable <- function(x, upload = FALSE) {
  !(is.numeric(x) || (upload && inherits(x, c("Date", "POSIXt"))) ||
      length(unique(stats::na.omit(x))) > schema_max_levels)
}

# 범주 값 vals 가운데 낼 수 있는 값. 업로드 자료는 관측 수가 upload_min_level_n() 미만인 값을 뺀다
frequent_levels <- function(x, vals, upload = FALSE) {
  vals <- as.character(vals)
  if (!upload) return(vals)
  cnt <- table(as.character(x))[vals]
  vals[!is.na(cnt) & cnt >= upload_min_level_n()]
}

# 열 x의 범주 값 vals를 메시지에 붙일 문자열(fmt 형식)로 만든다. 데이터셋 구조에 범주 목록이 나오지 않는 열
# (숫자형, 범주가 schema_max_levels개를 넘는 열, 업로드 자료의 날짜·시각)이거나 낼 값이 없으면 빈 문자열을
# 돌려준다(호출 측은 개수를 함께 쓴다).
msg_levels <- function(x, vals, n = 5, fmt = ": %s") {
  upload <- isTRUE(run_context$upload)
  if (!levels_exposable(x, upload)) return("")
  vals <- frequent_levels(x, vals, upload)
  if (length(vals) == 0) return("")
  sprintf(fmt, paste0(paste(utils::head(vals, n), collapse = ", "), if (length(vals) > n) "
등" else ""))
}
```

- 쓰는 예: `binary_01`

```
sprintf("%s의 값이 0/1이 아님(서로 다른 값 %d개)%s", col, length(vals), msg_levels(data[[col]],
sort(vals)))
```

- `JSK-KM-LOGRANK`에 0/1이 아닌 사건 변수를 준 경우(2026-10-12 실행)
  - `pbc`의 숫자형 `status`(0/1/2) → "status의 값이 0/1이 아님(서로 다른 값 3개)"
  - `lung`의 범주형 `sex` → "sex의 값이 0/1이 아님(서로 다른 값 2개): Male, Female"
- `test-core.R`의 테스트 "규칙 메시지에 관측값(최솟값·최댓값·분위수·식별자)을 쓰지 않음"이 이 기준을 확인
  - 최솟값 −7.77, 최댓값 987.6, `SUBJ001` 같은 눈에 띄는 값을 넣은 합성 자료로 규칙을 직접 불러 메시지에 그 값이 없는지 봄
  - 값을 다루는 규칙을 새로 만들면 이 테스트에 사례를 더함

5.3 공통 규칙

- `core.R`에 정의된 규칙은 14개
- 필수 키는 `rule_required_keys`에서, 쓰는 Skill 수는 등록부에서 읽음

규칙	필수 키	선택 키	위반 조건	쓰는 Skill 수
`distinct_columns`	`columns`		`columns`에 적은 입력 인자들이 가리키는 열 사이에 같은 열이 두 번 나오면 위반(예: 그룹 변수를 비교 변수에도 지정)	96
`n_levels`	`column`	`min`, `max`	`column` 인자가 가리키는 열의 결측 제외 서로 다른 값 수가 `min` 미만이거나 `max` 초과이면 위반. `min`·`max`는 둘 다 선택. 인자를 지정하지 않았으면 통과	51
`min_rows`	`columns`, `min`		`columns` 인자들이 가리키는 열을 모두 갖춘 완전사례 행 수가 `min` 미만이면 위반	65

규칙	필수 키	선택 키	위반 조건	쓰는 Skill 수
max_rows	max		전체 행 수가 max 초과이면 위반(결측과 무관)	3
min_group_size	column, min		column 열의 범주별 관측 수(table(), 결측 제외) 가운데 min 미만인 범주가 있으면 위반. 인자를 지정하지 않았으면 통과	27
not_constant	columns		columns 인자들이 가리키는 열 가운데 결측 제외 값이 1종류 이하인 열이 있으면 위반	77
max_levels	columns, max		columns 열 가운데 결측 제외 서로 다른 값이 max개를 넘는 열이 있으면 위반	30
max_missing_rate	columns, max		columns 열 가운데 결측 비율이 max(0-1)를 넘는 열이 있으면 위반. 보통 W_ 코드(경고)와 묶음	90
expected_count_min	columns, group, min		group 열 × columns의 각 열 교차표(빈 행·열 제외)에서 기대빈도가 min 미만인 칸이 있으면 위반. 2×2보다 작은 표는 검사하지 않음	2
min_events	column, min		column 열에서 값이 1인 행 수(사건 수)가 min 미만이면 위반	9
binary_01	column		column 열의 결측 제외 값이 0과 1 말고 다른 값을 포함하면 위반	17
nonnegative_values	columns		columns 열에 음수가 있으면 위반(추적 0일을 허용하는 시간 등)	1
positive_values	columns		columns 열에 0 이하 값이 있으면 위반(생존시간 등)	25
min_clusters	column, min		column 열(군집 ID)의 서로 다른 값 수가 min 미만이면 위반	10

세부 동작:

- n_levels와 min_group_size, min_events, binary_01, min_clusters는 column 인자가 가리키는 입력을 사용자가 주지 않았으면 검사 없이 통과 → 선택 인자인 그룹 변수에도 안심하고 걸 수 있음
- max_missing_rate의 max는 비율(0.2 = 20%). 메시지에는 열별 결측 비율이 퍼센트로 나옴
- expected_count_min: Table 1의 범주형 변수 비교처럼 그룹 × 변수 교차표에서 카이제곱 근사가 의심스러운지 볼 때 씀

5.4 계열 전용 규칙

- 공통 규칙으로 표현할 수 없는 조건은 계열 파일에서 규칙을 추가
- 현재 계열 전용 규칙은 71개. 아래에 정의 파일별로 모두 실음
  - 규칙마다 필수 키, 쓰는 Skill 수, 내용
  - 내용은 각 규칙 정의 바로 위의 코드 주석(없으면 파일 머리말의 규칙 설명)을 그대로 옮긴 것

cdisc.R (6개)

- adam_columns_present — 필수 키 columns, 쓰는 Skill 2. 지정한(또는 기본값) ADaM 변수가 데이터에 있는지. 기본값으로 채운 열 이름은 입력 검증을 거치지 않으므로 여기서 본다
- adam_flag_values — 필수 키 column, 쓰는 Skill 2. 분석집단 플래그 값은 Y/N/빈칸만
- adam_one_record — 필수 키 column, 쓰는 Skill 2. 대상자당 1행(by가 있으면 by 값마다, value가 있으면 by == p[[value]]인 행만)
- adam_param_exists — 필수 키 column, value, 쓰는 Skill 1. PARAMCD 값이 데이터에 있는지
- adam_cnsr_01 — 필수 키 column, 쓰는 Skill 1. CNSR은 0/1(해당 PARAMCD 행 기준)
- adam_min_analysis_rows — 필수 키 flag, min, 쓰는 Skill 2. 분석 대상(PARAMCD 행 + 플래그 = Y) 행 수 하한

cor.R (3개)

- `cor_pairs` — 필수 키 `columns`, `min`, 쓰는 Skill 3. 변수 쌍(columns가 3개 이상이면 모든 쌍)마다 둘 다 결측이 없는 관측 수 하한. group 키(입력 인자 이름)의 값이 있으면 그룹 수준별로도 점검하고, 그룹 안에서 값이 하나뿐인 변수도 위반으로 본다
- `cor_normality` — 필수 키 `columns`, `alpha`, 쓰는 Skill 1. 정규성(Pearson 상관의 이변량 정규 가정 점검): 두 변수가 모두 있는 관측에서 Shapiro-Wilk p < alpha인 변수가 있으면 위반. 표본이 3-5000개일 때만 검정한다
- `cor_ties` — 필수 키 `columns`, `max_distinct`, 쓰는 Skill 1. 값 종류가 적은 변수(순서형 점수 등): 두 변수가 모두 있는 관측에서 서로 다른 값이 max_distinct개 이하인 변수가 있으면 위반. 동순위가 매우 많아 rho의 해석과 t 근사 p가 불안정해진다(검사 수치처럼 측정 단위 때문에 생긴 약간의 동순위는 해당 없음)

`cox.R` (3개)

- `min_events_per_group` — 필수 키 `column`, `group`, `min`, 쓰는 Skill 1. 층(group)마다 사건(column == 1) 수 하한. 사건이 없는 층은 층화 부분우도에 기여하지 않는다
- `repeated_clusters` — 필수 키 `column`, 쓰는 Skill 3. 군집 변수에 관측이 2개 이상인 군집이 하나라도 있어야 함(모두 1개면 군집 구조가 없음)
- `min_events_per_state` — 필수 키 `column`, `censor`, `min`, 쓰는 Skill 1. 다중상태 사건 변수: 중도절단(censor 인자 값)을 뺀 사건 상태마다 사건 수 하한. columns를 주면 사건 수를 모형 모수 수로 나눈 값(상태별 EPV)을 본다. 모형 변수의 완전사례 기준.

`gee.R` (3개)

- `rm_time_required` — 필수 키 `column`, `param`, `value`, 쓰는 Skill 3. 특정 인자 값일 때 다른 인자가 필요함(예: corstr = ar1이면 time 필요)
- `rm_integer_values` — 필수 키 `columns`, 쓰는 Skill 3. 정수 값만 허용(GEE waves). param·value를 주면 그 조건일 때만 점검
- `rm_unique_time` — 필수 키 `column`, `time`, 쓰는 Skill 3. 군집 안에서 측정 시점이 겹치면 안 됨

`km.R` (7개)

- `km_group_events` — 필수 키 `column`, `group`, `min`, 쓰는 Skill 4. 그룹(group)별 사건 수 하한. 사건이 없는 그룹은 중앙생존시간·CI를 추정할 수 없다
- `km_min_events` — 필수 키 `column`, `min`, 쓰는 Skill 7. 사건 수 하한(value가 있으면 관심 사건 값 기준). 모형 변수(columns)의 완전사례 기준
- `km_time_in_range` — 필수 키 `column`, `value`, 쓰는 Skill 3. 시점 인자(value: landmark_time, time_point 등)가 관찰된 추적시간 범위 안(최소 초과, 최대 미만)에 있어야 함
- `km_landmark_at_risk` — 필수 키 `column`, `group`, `value`, `min`, 쓰는 Skill 1. landmark 시점 이후(시간 >= L)에도 그룹마다 위험집합이 min명 이상 남아 있어야 함
- `km_interval_events` — 필수 키 `column`, `event`, `value`, `min`, 쓰는 Skill 1. landmark 앞 구간(시간 < L)과 뒤 구간(시간 >= L)에 사건이 각각 min건 이상
- `km_cmprsk_states` — 필수 키 `column`, `censor`, 쓰는 Skill 2. 경쟁위험 상태 변수: 중도절단을 뺀 사건 종류가 2개 이상이어야 함(1개면 경쟁 사건이 없음)
- `km_min_unique` — 필수 키 `column`, `min`, 쓰는 Skill 1. 숫자형 변수의 서로 다른 값 수 하한(절단값 탐색 후보 수)

`meta.R` (3개)

- `meta_counts` — 필수 키 `events`, `totals`, 쓰는 Skill 6. 사건 수 ≤ 총 수, 사건 수 ≥ 0, 총 수 > 0, 정수. events·totals는 같은 길이의 인자 이름 목록(짝지어 비교)
- `meta_ci` — 필수 키 `estimate`, `lower`, `upper`, `measure`, 쓰는 Skill 5. 효과크기·95% CI: 하한 ≤ 추정치 ≤ 상한, 하한 < 상한, 비 척도(HR·OR·RR)면 모두 양수
- `min_studies` — 필수 키 `columns`, `min`, 쓰는 Skill 8. 연구(행) 수 하한: 필요한 열이 모두 있는 행

`mix.R` (4개)

- `rm_repeated_clusters` — 필수 키 `column`, `columns`, `min`, 쓰는 Skill 4. 관측이 2개 이상인 군집(반복측정이 있는 대상자) 수 하한. 분석 변수 전체 완전사례 기준
- `rm_member_of` — 필수 키 `column`, `columns`, 쓰는 Skill 1. 한 인자의 변수가 다른 인자 목록에 들어 있어야 함(예: 무작위 기울기 변수는 공변량이어야 함)
- `rm_count_outcome` — 필수 키 `column`, 쓰는 Skill 2. 횟수 자료: 0 이상의 정수만 허용
- `rm_binary_balance` — 필수 키 `column`, `min`, 쓰는 Skill 2. 이분형 결과: 0과 1이 각각 min개 이상

`ml.R` (1개)

- `max_factor_levels` — 필수 키 `columns`, `max`, 쓰는 Skill 18. 범주형(숫자형이 아닌) 예측변수의 범주 수 상한. 더미 변수가 지나치게 늘어나는 것을 막는다.

`ps.R` (1개)

- `ps_enough_controls` — 필수 키 `column`, `columns`, `value`, `ratio`, 쓰는 Skill 1. 완전사례에서 대조군 수가 처리군 수 × ratio 이상(최근접 이웃 비복원 매칭에서 모든 처리군이 짝을 얻을 수 있는지)

`reg.R` (5개)

- `min_events_per_param` — 필수 키 `columns`, `min`, 쓰는 Skill 13. 모수 1개당 사건 수(EPV). column이 없으면 관측 수, minority: true이면 결과변수에서 적은 쪽 범주의 수, 그 외에는 column == 1인 사건 수를 센다. 모형 변수의 완전사례 기준.
- `nonnegative_integer` — 필수 키 `column`, 쓰는 Skill 1. 값이 0 이상의 정수여야 함(개수 자료)
- `subset_columns` — 필수 키 `columns`, `of`, 쓰는 Skill 18. columns에 지정한 변수가 모두 of 인자의 변수 목록 안에 있어야 함
- `value_in_column` — 필수 키 `column`, `value`, 쓰는 Skill 20. value 인자로 받은 값이 column 변수의 값 가운데 있어야 함(범주형 변수의 범주 이름 확인용)
- `min_columns` — 필수 키 `columns`, `min`, 쓰는 Skill 1. 지정한 변수 개수 하한

`roc.R` (7개)

- `roc_model_columns` — 필수 키 `column`, `columns`, 쓰는 Skill 4. 결과변수가 설명변수 목록에 들어 있거나 한 모형 안에 같은 변수가 중복되면 위반
- `roc_distinct_models` — 필수 키 `columns`, 쓰는 Skill 3. 모형끼리 설명변수 구성이 같으면(순서 무관) 비교할 수 없음
- `roc_min_class` — 필수 키 `column`, `columns`, `min`, 쓰는 Skill 5. 결과변수와 columns(모든 모형 변수)의 완전사례에서 결과변수 두 범주가 각각 min 이상
- `roc_epv` — 필수 키 `column`, `columns`, `min`, 쓰는 Skill 4. 모형마다 (적은 쪽 결과 범주 수) / (모형 모수)가 min 이상. 모든 모형 변수의 완전사례 기준
- `roc_min_unique` — 필수 키 `column`, `min`, 쓰는 Skill 1. 숫자형 변수의 서로 다른 값 개수 하한(절단값을 찾을 만한 연속 표지자인지)
- `roc_cutoff_in_range` — 필수 키 `column`, `value`, 쓰는 Skill 1. 사용자 절단값이 표지자의 관측 범위 안에 있어야 함
- `roc_risk_cutoffs` — 필수 키 `value`, `max`, 쓰는 Skill 1. `value` 인자의 위험 범주 기준 문자열(예: "0.1, 0.2")이 쉼표로 구분한 숫자이고, 모두 0과 1 사이(양 끝 제외)이며, 서로 다른 값의 오름차순이고, `max`개 이하인지 (문서 보충)

`samplesize.R` (3개)

- `params_differ` — 필수 키 `first`, `second`, 쓰는 Skill 4. 두 인자 값이 달라야 함(효과 크기 0이면 표본수를 정의할 수 없음)
- `param_order` — 필수 키 `lower`, `upper`, 쓰는 Skill 1. 인자 크기 순서: lower ≤ upper(strict: true면 lower < upper)
- `prevalence_string` — 필수 키 `param`, 쓰는 Skill 2. 유병률 문자열("0.2" 또는 "0.3, 0.3, 0.4"): 값 1개(이분형 사건 유병률) 또는 2-5개(범주별, 합 1), 각 값은 (0, 1)

`spl.R` (5개)

- `spl_knots_distinct` — 필수 키 `column`, `knots`, `columns`, 쓰는 Skill 2. 분위수 매듭이 모두 달라야 함(같은 값이 많은 변수는 매듭이 겹쳐 스플라인 기저를 만들 수 없음)
- `spl_min_unique` — 필수 키 `column`, `value`, 쓰는 Skill 1. 서로 다른 x 값 수 하한(GAM 기저 차원 k보다 적으면 mgcv가 적합하지 못함)
- `spl_ref_in_range` — 필수 키 `column`, `value`, 쓰는 Skill 3. 기준값이 x의 관측 범위 안에 있어야 함
- `spl_min_events` — 필수 키 `outcome`, `family`, `columns`, `min`, 쓰는 Skill 2. 이분형 결과의 적은 쪽 범주 수 하한(family가 binomial일 때만)
- `spl_epv` — 필수 키 `outcome`, `covariates`, `columns`, `df_param`, `min`, 쓰는 Skill 3. 모수 1개당 사건 수(EPV): 스플라인 자유도(df_param 인자 - 1) + 공변량 모수. 사건 수는 binomial 적은 쪽 범주, Cox(family 키 없음) 사건(=1) 수, gaussian·poisson은 관측 수

`sub.R` (5개)

- `sg_cell_size` — 필수 키 `columns`, `group`, `min`, 쓰는 Skill 3. 서브그룹 변수 수준 × 노출 수준 칸마다 관측 수 하한

- `sg_cell_events` — 필수 키 `columns`, `group`, `event`, `min`, 쓰는 Skill 3. 서브그룹 변수 수준 × 노출 수준 칸마다 사건(=1) 수 하한. both: true면 비사건(=0)도 점검. family 키(입력 인자 이름)를 주면 binomial은 사건·비사건, poisson은 칸별 횟수 합을 점검하고 gaussian은 건너뛴다
- `sg_eventrate_time` — 필수 키 `column`, `value`, 쓰는 Skill 1. 사건률 시점이 최대 추적 시간을 넘으면 안 됨
- `sg_outcome_family` — 필수 키 `column`, `family`, 쓰는 Skill 4. 결과변수 형식이 family와 맞아야 함: binomial 0/1, poisson 0 이상 정수, gaussian 숫자형
- `sg_clusters_per_level` — 필수 키 `columns`, `id`, `min`, 쓰는 Skill 1. 서브그룹 수준마다 군집 수 하한(혼합모형 서브그룹)

`survpred.R` (3개)

- `surv_cmprsk_epv` — 필수 키 `column`, `value`, `columns`, `min`, 쓰는 Skill 1. 경쟁위험 관심 사건 수 / 모형 모수(EPV). 모형 변수 완전사례 기준
- `surv_distinct_models` — 필수 키 `columns`, 쓰는 Skill 2. 비교할 모형(공변량 집합)이 서로 달라야 함
- `surv_cases_controls` — 필수 키 `column`, `event`, `value`, `columns`, `min`, 쓰는 Skill 2. 시점 t의 사례(시간 <= t이고 사건)와 대조(시간 > t) 수 하한. 모형 변수 완전사례 기준

`svy.R` (9개)

- `svy_design_complete` — 필수 키 (없음), 쓰는 Skill 11. 설계 변수(가중치·층·PSU)에 결측 없음
- `svy_positive_weights` — 필수 키 (없음), 쓰는 Skill 11. 가중치가 모두 양수
- `svy_psu_nested` — 필수 키 (없음), 쓰는 Skill 11. nest = FALSE인데 같은 PSU 번호가 여러 층에 나오면 오류(nest = TRUE 필요)
- `svy_lonely_psu` — 필수 키 (없음), 쓰는 Skill 11. 층마다 PSU가 2개 이상(lonely PSU면 분산 추정 불가)
- `svy_design_df` — 필수 키 `min`, 쓰는 Skill 10. 설계 자유도(PSU 수 − 층 수) − 모형 계수 수(절편 제외) ≥ min
- `svy_factor_max_levels` — 필수 키 `columns`, `max`, 쓰는 Skill 4. 숫자형이 아닌(범주형) 설명변수의 범주 수 상한
- `svy_numeric_01` — 필수 키 `column`, 쓰는 Skill 1. 숫자형 결과변수는 0/1 코딩
- `svy_subgroup_events` — 필수 키 `column`, `group`, `columns`, `min`, 쓰는 Skill 1. 서브그룹 수준 × 노출 수준 칸마다 사건 수 ≥ min
- `svy_eventrate_time` — 필수 키 `column`, `value`, 쓰는 Skill 1. 누적 사건률 계산 시점이 관찰된 최대 추적시간 이하

`vis.R` (3개)

- `vis_cell_size` — 필수 키 `columns`, `min`, 쓰는 Skill 4. 그룹 수준(× 선택적 x 수준) 칸마다 분석 열이 모두 있는 관측 수 하한. group 키(입력 인자 이름)의 값이 없으면(선택 인자 미지정) cell 키 변수만으로 칸을 나눈다. 둘 다 없으면 전체 관측 수
- `vis_flag_needs_levels` — 필수 키 `flag`, `column`, `min`, 쓰는 Skill 2. flag 인자(TRUE/FALSE)가 TRUE인데 그룹 범주가 min개 미만이면 위반(쌍별 비교는 3그룹 이상에서만 의미가 있음)
- `vis_min_distinct` — 필수 키 `columns`, `min`, 쓰는 Skill 1. 숫자형 변수의 서로 다른 값 개수 하한(값 종류가 몇 개뿐이면 히스토그램 대신 빈도 막대가 맞음)

5.5 계열 전용 규칙 만들기

- 새 규칙은 그 규칙을 쓰는 계열 파일(`r-executor/R/skills/<계열>.R`)에 둠
- 짧은 예: `cor.R`의 정규성 경고 규칙

```
# 정규성(Pearson 상관의 이변량 정규 가정 점검): 두 변수가 모두 있는 관측에서 Shapiro-Wilk p < alpha인 변수가 있으면 위반.
# 표본이 3–5000개일 때만 검정한다
rule_funs$cor_normality <- function(r, data, p) {
  cols <- param_columns(p, r[["columns"]])
  d <- data[stats::complete.cases(data[, cols, drop = FALSE]), cols, drop = FALSE]
  if (nrow(d) < 3 || nrow(d) > 5000) return(NULL)
  pv <- vapply(cols, function(v) if (length(unique(d[[v]])) < 3) NA_real_ else
stats::shapiro.test(d[[v]])$p.value, numeric(1))
  low <- pv[!is.na(pv) & pv < r[["alpha"]]]
  if (length(low) > 0) sprintf("Shapiro-Wilk p < %g인 변수: %s", r[["alpha"]],
paste(sprintf("%s(p = %.3g)", names(low), low), collapse = ", "))
}
```

```
rule_required_keys$cor_normality <- c("columns", "alpha")
```

만들 때 지킬 것:

1. 이름에 계열 접두어를 붙임(예: `cor_`, `km_`, `svy_`)
   - 모든 계열 파일이 같은 `rule_funs` 목록에 규칙을 덧붙임 → 다른 파일과 이름이 같으면 나중에 읽힌 파일이 앞의 정의를 조용히 덮어씀
   - 등록부 생성기가 이 충돌을 "이름 충돌"로 잡지만 처음부터 접두어로 피함
   - 현재 목록에는 접두어 없는 규칙(`min_events_per_param`, `max_factor_levels` 등)도 있음. 새 규칙에는 접두어를 붙임
2. 필수 키를 `rule_required_keys`에 등록
   - 등록부 생성기가 명세의 규칙 객체에 이 키가 모두 있는지 확인
   - 필수 키가 없는 규칙은 `character(0)`을 등록(`svy.R`의 설계 점검 규칙들처럼 Skill 입력 `weights·strata·cluster`를 직접 읽는 경우)
3. 선택 키는 `NULL`을 고려
   - `r[["group"]]`이 없을 수 있다면 `if (!is.null(r[["group"]])) ...`로 처리
4. 규칙 안에서 오래 걸리는 계산을 하지 않음
   - 규칙은 `/validate`에서도 실행됨. 이 경로는 별도 프로세스·timeout 없이 R 실행 서비스 프로세스에서 돎(섹션 2)
   - 표본이 클 때 검정을 건너뛰는 식으로 상한을 둠(`cor_normality`는 5,000행을 넘으면 검정하지 않음)
5. 같은 판정이 정상 사례와 오류 사례 테스트에 모두 드러나게 함
   - 규칙이 위반을 알리는 자료를 테스트에서 하나 이상 만듦(섹션 9)

명세에 거는 법:

- 규칙을 정의해도 명세에서 쓰지 않으면 아무 일도 하지 않음
- 명세의 `validation`이나 `preconditions`에 `{rule: <이름>, ..., error: <코드>}`로 걸고 그 코드를 명세 `errors`에 정의
- 같은 규칙을 기준값만 바꿔 두 번 걸 수도 있음
  - 예: subgroup Cox 명세는 `sg_cell_events`를 `min: 1`과 `E_SPARSE_SUBGROUP`(차단)으로 한 번, `min: 5`와 `W_FEW_SUBGROUP_EVENTS`(경고)로 한 번 걺
  - 효과: 추정이 불가능한 경우는 막고 가능하지만 불안정한 경우는 경고만 함

6 오류 처리

6.1 오류 응답 구조

- Skill 실행이 실패하면 결과는 아래 모양
- 2026-10-12에 `JST-TB1-TTEST`를 3그룹 변수(`rx`)로 실행해 얻은 실제 출력. provenance는 줄임


```
{
  "status": "error",
  "skill_id": "JST-TB1-TTEST",
  "error": {
    "code": "E_GROUP_COUNT",
    "action": "ask_user",
    "message": "그룹 범주가 2개가 아님: rx의 범주 수 3 (최대 2)",
    "recovery_hint": "3그룹 이상은 JST-TB1-ANOVA를 사용"
  },
  "provenance": {
    "skill_id": "JST-TB1-TTEST",
    "skill_version": "0.1.0",
    "r_function": "jstable::CreateTableOneJS",
    "dataset_ref": "colon_recur",
    "params_hash": "092e7837f199650e34a8236d7f73951cb477efdcf5769b6880698c702c1b1071",
    "r_version": "4.5.1",
    "packages": {
      "jstable": "1.3.25",
```

```
      "jskm": "0.6.3",
      "…": "(38개 더, 줄임)"
    },
    "executed_at": "2026-10-12T03:59:02Z",
    "elapsed_ms": 26
  }
}
```

`error` 객체의 네 항목:

- `code`: 오류를 일으킨 단계가 정한 코드
  - 입력 검증은 공통 코드, 규칙은 명세에 적은 `error` 코드, wrapper는 `skill_error()`로 지정한 코드
  - wrapper가 지정하지 않은 R 오류는 모두 `E_RUNTIME`
- `action`: 그 코드의 명세 `errors` 정의에서 가져옴
  - 명세에 없으면 실행 틀의 공통 정의(`default_errors`), 거기에도 없으면 `block`
- `message`: 정의의 `message` 뒤에 콜론과 상세 내용을 붙임
  - 상세 내용: 규칙이 돌려준 문자열, 입력 검증이 만든 설명(예: "var2: albumn"), `skill_error()`의 `detail`, 또는 R 오류 메시지
- `recovery_hint`: 오류를 낸 쪽이 안내를 직접 지정했으면 그것, 아니면 정의의 `hint`
  - 입력 검증의 `E_COL_NOT_FOUND`는 비슷한 열 이름을 찾으면 "유사 변수명: …"을 직접 지정 → 명세의 `hint`보다 우선
- 아래는 같은 날 `JSM-COR-PEARSON`에 없는 열 `albumn`을 넣어 얻은 출력
- 명세의 `hint`("유사 변수명을 제안하고 사용자에게 확인") 대신 실제 후보가 안내에 들어감

```
{
  "status": "error",
  "skill_id": "JSM-COR-PEARSON",
  "error": {
    "code": "E_COL_NOT_FOUND",
    "action": "ask_user",
    "message": "데이터에 없는 변수: var2: albumn",
    "recovery_hint": "유사 변수명: albumin"
  },
  "provenance": {
    "skill_id": "JSM-COR-PEARSON",
    "skill_version": "0.1.0",
    "r_function": ["ggpubr::ggscatter", "stats::cor.test"],
    "dataset_ref": "pbc",
    "params_hash": "0222d8cdfbf9822fb7b0ff10652a5eefeb7df4c6a56f021c216af9c200f9889b",
    "r_version": "4.5.1",
    "packages": {
      "jstable": "1.3.25",
      "jskm": "0.6.3",
      "…": "(38개 더, 줄임)"
    },
    "executed_at": "2026-10-12T03:59:02Z",
    "elapsed_ms": 20
  }
}
```

- 오류 결과에는 `tables`·`figures`·`notes`·`warnings`가 없음. 앞선 규칙에서 모인 경고도 버려짐
- `provenance`는 성공·실패와 관계없이 붙음

6.2 공통 오류 코드

- 실행 틀이 직접 내는 코드와 그 기본 정의(`core.R`의 `default_errors`)
- Skill 명세에 같은 코드를 적으면 명세의 정의가 우선

오류 코드	action	기본 메시지	기본 안내(hint)
`E_UNKNOWN_SKILL`	block	등록되지 않은 Skill	등록부의 Skill ID를 확인

오류 코드	action	기본 메시지	기본 안내(hint)
E_UNKNOWN_DATASET	block	허용 목록에 없는 데이터셋	등록된 데이터셋만 사용 가능
E_UNKNOWN_PARAM	block	명세에 없는 인자	Skill 입력 스키마를 확인
E_MISSING_PARAM	ask_user	필수 인자 누락	누락된 인자를 지정
E_INVALID_VALUE	ask_user	인자 값이 허용 범위를 벗어남	허용 값을 확인
E_COL_NOT_FOUND	ask_user	데이터에 없는 변수	유사 변수명을 확인
E_TYPE_MISMATCH	ask_user	변수 형식이 분석에 맞지 않음	변수 형식을 확인하거나 다른 분석을 선택
E_RUNTIME	block	분석 실행 중 오류	입력 데이터와 인자를 확인
E_TIMEOUT	block	실행 시간 제한 초과	변수 수나 자료 크기를 줄여 다시 실행

이 밖에 R 실행 서비스 API 자체가 내는 코드 2개:

- HTTP 400과 E_BAD_REQUEST: 요청 본문에 skill_id나 dataset_ref가 없음
- HTTP 409와 E_VERSION_MISMATCH: 요청에 실은 version이 등록된 명세의 버전과 다름
- 둘 다 Skill 실행 전에 발생 → action과 recovery_hint 없이 code와 message만 있음(섹션 14)

agent 쪽에서 덧붙는 코드:

- 도구 호출 루프(loop.py)가 만들어 LLM에 돌려줌
  - E_UNKNOWN_TOOL: 등록되지 않은 도구 이름
  - E_TOOL_RUNTIME: 도구 처리 중 Python 예외
  - E_PRIVACY: privacy guard가 결과를 막음
- 분석계획 API(api.py)는 계획이 막히면 E_PLAN_INVALID로 감싸고 분석별 문제(workflow.py의 check_plan() 결과)를 problems에 담음
- 이 코드들은 Skill 명세에 적지 않음

6.3 오류 위치별 코드 선택

단계	내는 코드	명세에 적을 것
데이터셋 해석	E_UNKNOWN_DATASET	적지 않음(공통 정의로 충분)
입력 검증	E_UNKNOWN_PARAM, E_MISSING_PARAM, E_INVALID_VALUE, E_COL_NOT_FOUND, E_TYPE_MISMATCH	이 Skill에서 생길 수 있는 것을 Skill에 맞는 문구로. 특히 E_TYPE_MISMATCH의 hint에 대신 쓸 Skill을 적음
검증 규칙·실행 조건	규칙 객체의 error	반드시 적음. 빠지면 등록부 점검에서 "정의되지 않은 오류 코드"
wrapper	skill_error()로 지정한 코드	반드시 적음. 등록부 점검은 wrapper 안의 코드를 대조하지 않음
wrapper(예상 못 한 오류)	E_RUNTIME	적기를 권장. 공통 정의보다 구체적인 hint를 줄 수 있음
timeout	E_TIMEOUT	오래 걸릴 수 있는 Skill은 줄일 인자를 hint에 적음

- E_RUNTIME의 메시지에는 R 오류 메시지가 그대로 들어감("분석 실행 중 오류: subscript out of bounds" 같은 식)
  - 사용자도 LLM도 이것으로는 무엇을 고칠지 알기 어려움
- 그래서 예상할 수 있는 실패(사건이 없음, 수렴하지 않음, 그룹이 하나뿐임 등)는 규칙으로 미리 막거나 wrapper 안에서 확인해 구체적인 코드로 바꿈

```
  fit <- do.call(geepack::geeglm, args)
  if (!identical(as.integer(fit$geese$error), 0L)) {
    stop(skill_error("E_NOT_CONVERGED", sprintf("geepack 반복 추정이 수렴하지 않음(geese error =
%s)", fit$geese$error)))
  }
```

- 위는 `gee.R`의 GEE wrapper 일부. 기보유 함수가 수렴 실패를 경고 없이 결과로 내는 문제(섹션 13)를 이렇게 막음
- `skill_error(code, detail, hint)`의 세 번째 인자 `hint`를 주면 명세의 `hint` 대신 그 문장이 안내로 나감
  - 자료에 따라 안내가 달라질 때 사용(예: `meta.R`에서 평균의 비(ROM)를 쓸 수 없을 때 "MD 또는 SMD를 사용")

6.4 action별 동작

- `action`은 그 오류를 받은 쪽이 무엇을 해야 하는지 정함
- `ask_user`(사용자 확인): 사용자에게 되물으면 풀리는 오류
  - 예: 변수 이름이 틀림, 변수 형식이 맞지 않음, 그룹 수가 이 분석에 맞지 않음
  - 통계실행 agent의 시스템 프롬프트는 이 경우 `recovery_hint`를 바탕으로 사용자에게 확인 질문을 하라고 지시
  - 인자를 고쳐 다시 시도할 수 있으면 한 번 다시 시도
- `block`(차단): 같은 자료로는 이 분석을 할 수 없는 오류
  - 예: 값이 하나뿐인 변수, 사건이 없는 자료, 수렴 실패, timeout
  - agent는 실행할 수 없는 이유와 대안을 설명
- `warn`(경고 후 진행): 분석은 할 수 있지만 해석에 주의가 필요한 경우
  - 예: 결측 비율이 높음, 정규성 위반, 희소 칸
  - 실행 틀은 멈추지 않고 `"코드: 내용"` 문자열을 결과의 `warnings`에 넣음
  - 이 경고는 LLM에 결과와 함께 전달됨
  - 이어지는 곳: 보고서 문서의 결과 아래 "경고" 줄, 품질 점검의 경고 항목, 데이터 검증의 판정(경고가 있으면 `warn`)

분석계획 API(`POST /projects/{id}/plan`):

- 계획의 분석마다 `/validate`를 부르고 하나라도 막히면 HTTP 422로 전체를 거부
- 응답의 `problems`에 분석 ID와 함께 위 `error` 객체(`code`, `action`, `message`, `recovery_hint`)가 그대로 들어감
- 사용자가 인자를 고쳐 다시 내면 접수됨. 경고만 있는 계획은 접수됨

6.5 timeout

- timeout은 두 겹
  - R 수준: `run_skill()`이 wrapper를 부르기 직전에 제한을 걺(`R_EXECUTOR_TIMEOUT_SEC`, 기본 60초). 넘기면 "실행 시간 제한 초과: 60초 초과" 같은 `E_TIMEOUT`
  - 프로세스 수준: 컴파일 코드 안에서 멈춰 R 수준 제한이 듣지 않으면 별도 프로세스의 제한(60 + 5초)이 프로세스를 종료. "65초 초과(프로세스 종료)"라는 `E_TIMEOUT`
- 두 경우 모두 `action`은 명세 정의(없으면 `block`)를 따름
- 자료 크기나 인자에 따라 오래 걸릴 수 있는 Skill(머신러닝 하이퍼파라미터 탐색, 순열·bootstrap 등)
  - 명세에 `E_TIMEOUT`을 적고 `hint`에 줄일 인자를 구체적으로 씀
  - 기존 머신러닝 Skill의 문구: "grid_num·iter·folds를 줄이거나 예측변수 수를 줄여 다시 실행"
- timeout은 규칙 점검에는 걸리지 않음 → 오래 걸리는 계산을 규칙에 넣지 않음(섹션 5)

6.6 오류 메시지 작성

- `message`: 무엇이 문제인지 명사형으로 짧게(예: "그룹 범주가 2개가 아님")
  - 상세 내용은 규칙·입력 검증이 뒤에 붙임 → 메시지에 열 이름이나 숫자를 넣지 않음
- 상세 내용(규칙의 반환 문자열, `skill_error()`의 `detail`)에는 관측 범위·개별 값·식별자를 넣지 않음(섹션 5.2)
- `hint`: 사용자가 바로 할 수 있는 일
  - 다른 Skill로 가야 하면 그 ID("3그룹 이상은 JST-TB1-ANOVA를 사용")
  - 자료를 고쳐야 하면 무엇을 어떻게 고칠지("결측을 확인하거나 그룹 범주를 합치거나 그룹 없이 분석")
  - 경고의 `hint`는 결과를 해석할 때 무엇을 적어야 하는지("완전사례 분석임을 결과 해석에 명시")
- 현재 명세에 쓰인 오류 코드 전체와 각 코드를 쓰는 Skill 수: 부록(섹션 16.2)

7 결과 형식

7.1 실제 출력

- JSM-COR-PEARSON을 예제 인자(pbc 자료, 빌리루빈과 알부민, 성별로 나눠)로 실행한 출력
- 실행 방법
  - 저장소의 기준 버전 패키지(.r-lib)를 쓰는 로컬 R에서 테스트와 같은 방법(load.R을 읽고 run_skill() 호출)으로 실행
  - 실행 시각 2026-10-12T03:59:00Z(UTC)
  - JSON은 R 실행 서비스의 /run과 같은 직렬화 옵션(jsonlite::toJSON(auto_unbox = TRUE, dataframe = "rows", na = "null"))으로 만듦
  - 실행 스크립트: techdocs/skill-standard/_examples/capture_examples.R
- provenance의 packages는 앞의 4개만 남기고 줄임


```
{
  "status": "ok",
  "skill_id": "JSM-COR-PEARSON",
  "tables": {
    "correlation": [
      {
        "variable1": "bili",
        "variable2": "albumin",
        "group": "전체",
        "n": 312,
        "r": -0.335,
        "CI_low": -0.43,
        "CI_high": -0.232,
        "t": -6.252,
        "df": 310,
        "p": "<0.001"
      },
      {
        "variable1": "bili",
        "variable2": "albumin",
        "group": "m",
        "n": 36,
        "r": -0.001,
        "CI_low": -0.329,
        "CI_high": 0.328,
        "t": -0.003,
        "df": 34,
        "p": "0.997"
      },
      {
        "variable1": "bili",
        "variable2": "albumin",
        "group": "f",
        "n": 276,
        "r": -0.363,
        "CI_low": -0.461,
        "CI_high": -0.256,
        "t": -6.448,
        "df": 274,
        "p": "<0.001"
      }
    ]
  },
  "figures": [
    {
      "name": "scatter",
      "title": "bili–albumin 산점도와 Pearson 상관계수",
```

```
      "format": "png",
      "file": "1df0e2ee95ba77429abd554a3d6cd87fb1b6b3b95672f5de2a69b392735528d6.png",
      "sha256": "1df0e2ee95ba77429abd554a3d6cd87fb1b6b3b95672f5de2a69b392735528d6",
      "width_in": 6.5,
      "height_in": 5.5,
      "dpi": 150
    }
  ],
  "notes": ["Pearson 적률상관계수 r, 95% CI는 Fisher z 변환, p는 t = r·sqrt((n-2)/(1-r²)) 자유도 n-2 검정(stats::cor.test)", "선형 관계만 잰다. 이상치에 민감하며 치우친 분포·순서형 자료는 JSM-COR-SPEARMAN 사용", "bili·albumin·sex에 결측이 없는 312쌍 사용. 그룹을 지정하면 전체와 그룹별 상관을 함께 제시", "그림의 선: 단순 선형회귀선(lm)과 95% 신뢰대", "p값 표기: 소수 셋째 자리, 0.001 미만은 <0.001. 그림 라벨의 계수는 유효숫자 2자리"],
  "warnings": ["W_NONNORMAL: Shapiro-Wilk p < 0.05인 변수: bili(p = 1.16e-25), albumin(p = 0.00023)"],
  "provenance": {
    "skill_id": "JSM-COR-PEARSON",
    "skill_version": "0.1.0",
    "r_function": ["ggpubr::ggscatter", "stats::cor.test"],
    "dataset_ref": "pbc",
    "params_hash": "44f484e92f8206521402ff9dbbceda4e99d9ee66cd70104c5d19333d6084c1a7",
    "r_version": "4.5.1",
    "packages": {
      "jstable": "1.3.25",
      "jskm": "0.6.3",
      "jsmodule": "2.0.1",
      "splineplot": "0.3.0",
      "…": "(36개 더, 줄임)"
    },
    "executed_at": "2026-10-12T03:59:00Z",
    "elapsed_ms": 1722
  }
}
```


- Table 1 계열은 기보유 패키지가 돌려주는 문자 행렬을 표로 바꿈 → 셀이 서식이 적용된 문자열
- 아래는 같은 방법으로 `JST-TB1-TTEST`를 예제 인자로 실행한 결과의 표 부분
  - 그룹 변수 `sex`의 수준 `0`, `1`이 열 이름이 됨(명세의 `<그룹 수준>`)
  - 첫 행은 그룹별 관측 수


```
{
  "status": "ok",
  "tables": {
    "table1": [
      {
        "variable": "n",
        "level": "",
        "0": "445",
        "1": "484",
        "p": "",
        "test": ""
      },
      {
        "variable": "age",
        "level": "",
        "0": "59.49 ± 12.28",
        "1": "60.00 ± 11.64",
        "p": "0.512",
        "test": ""
      },
      {
        "variable": "nodes",
```

```
        "level": "",
        "0": "3.73 ± 3.55",
        "1": "3.59 ± 3.59",
        "p": "0.565",
        "test": ""
      }
    ]
  },
  "figures": [],
  "notes": [
    "Stratified by sex",
    "p값: Welch two-sample t-test (oneway.test, var.equal = FALSE)"
  ],
  "warnings": []
}
```


7.2 항목별 설명

항목	형식	내용
`status`	`"ok"` 또는 `"error"`	실행 성공 여부
`skill_id`	문자열	실행한 Skill ID
`tables`	객체: 표 이름 → 행 객체의 배열	wrapper가 돌려준 `data.frame`을 행 단위 객체로 바꾼 것. 표 이름과 열은 명세 `outputs.tables`와 같아야 함
`figures`	그림 메타데이터의 배열	`name`, `title`, `format("png")`, `file`, `sha256`, `width_in`, `height_in`, `dpi`. 그림 파일 자체는 들어 있지 않음
`notes`	문자열 배열	사용한 검정·모형과 해석 정보(한국어)
`warnings`	문자열 배열	규칙 경고(`"W_코드: 내용"`), wrapper가 낸 경고, 실행 중 잡힌 R 경고
`error`	객체	실패했을 때만. `code`, `action`, `message`, `recovery_hint`(섹션 6)
`provenance`	객체	출처 정보. 성공·실패 모두

`provenance`의 항목:

항목	내용
`skill_id`, `skill_version`	실행한 Skill과 명세의 `version`
`r_function`	명세의 `r_function`(문자열 또는 배열)
`dataset_ref`	요청에 실린 데이터셋 ID
`params_hash`	요청 인자(기본값을 채우기 전)를 `jsonlite::toJSON(auto_unbox = TRUE)`로 직렬화한 문자열의 SHA-256
`r_version`	R 버전
`packages`	고정된 패키지 목록의 설치 버전. GitHub에서 commit을 고정해 설치한 패키지는 `버전@커밋 앞 7자리`(예: `1.2.0@396f7a4`). 설치되지 않은 패키지는 `null`
`executed_at`	실행 시작 시각(UTC, ISO 8601)

항목	내용
`elapsed_ms`	`run_skill()` 안에서 잰 소요시간(밀리초)

- provenance는 LLM에 넘어가지 않음. agent의 Audit Trail(`tool_result` 이벤트)과 trajectory에 남음
- 별도 프로세스가 timeout으로 종료된 경우 provenance가 줄어들고 `isolation: "process_killed"`가 붙음(섹션 2)

7.3 JSON 변환 시 주의점

- R 결과가 JSON이 될 때 몇 가지가 바뀜
- 결과를 읽는 코드(agent, 시험 실행기, 외부 클라이언트)와 wrapper를 쓰는 사람 모두 알아야 함

바뀌는 것:

- 길이 1인 벡터는 따로 표시하지 않으면 배열이 아니라 값 하나가 됨
  - `notes·warnings`는 실행 틀이 `I()`로 표시해 원소가 하나여도 배열로 냄(`core.R`의 `json_array()`). 위 출력의 `warnings`도 경고 하나짜리 배열
  - 데이터셋 구조(섹션 14)의 `levels`도 같은 방식으로 범주가 하나여도 배열
  - provenance의 `r_function`은 명세에 쓴 형태를 따름 → 함수 하나를 값 하나로 쓴 명세는 문자열, 여럿이면 배열. 결과를 읽는 쪽은 둘 다 받게 처리
  - 빈 벡터(`character(0)`)와 빈 목록은 `[]`
- 2026-10-12 이전의 R 실행 서비스는 `notes·warnings`가 하나면 문자열로 보냈음
  - agent의 클라이언트(`executor.py`)는 둘 다 목록으로 맞춰 받음
  - 그 무렵 저장된 결과를 읽는 보고서·품질검증도 문자열을 한 항목으로 다룸
- 숫자는 소수 넷째 자리까지만 나감
  - `jsonlite::toJSON()`의 기본 자릿수(`digits = 4`)를 그대로 쓰기 때문. 0.123456은 0.1235가 됨
  - 결과표의 숫자는 wrapper에서 보고할 자릿수로 반올림해 둠
  - p값처럼 경계 표기(`<0.001`)가 필요한 값은 문자열로 만듦
  - 예: 상관 계열 wrapper는 r·신뢰구간·t를 소수 셋째 자리로 반올림하고 p값은 문자열로 냄
- `NA`는 `null`이 됨
  - 결측 칸이 있는 표도 행마다 열 집합이 같게 유지됨
  - 시험 실행기는 행마다 열이 다르면 부적합으로 봄
- 행 이름은 사라짐
  - `data.frame`의 행 이름은 JSON에 들어가지 않음 → 변수 이름처럼 의미 있는 행 이름은 `matrix_to_df()` 등으로 열로 옮김(섹션 8)

7.4 그림

- wrapper가 `save_figure()`로 저장한 그림은 PNG 파일로 남음
  - 파일 이름: 내용의 SHA-256 해시에 `.png`를 붙인 것
  - 저장 위치: `R_EXECUTOR_FIGURE_DIR`. 설정되지 않았으면 R 임시 폴더 아래 `figures`
  - R 실행 서비스는 기동할 때 이 폴더를 정해 별도 실행 프로세스에도 넘김
- 클라이언트는 결과의 `file`로 `GET /figures/<file>`을 불러 PNG를 받음
  - API는 64자리 16진수 + `.png` 형식이 아닌 이름을 400으로 거부
- agent의 분석계획 실행(`run-plan`)
  - 결과의 그림마다 이 API로 PNG를 받아 프로젝트 산출물로 저장하고 결과에 산출물 ID를 붙임
  - LLM에는 그림의 `name`과 `title`만 전달됨
- 그림에 표시한 수치는 반드시 표로도 냄
  - LLM은 그림을 보지 못하고 표만 인용
  - 보고서의 수치 검증도 표가 기준

8 wrapper 작성 규칙

8.1 시그니처와 반환 구조

- wrapper는 인자 두 개를 받는 함수

```
skill_<계열>_<방법> <- function(data, p) {
  # data : 데이터셋 전체(data.frame). 허용 목록의 loader()가 만든 그대로
  # p    : 입력 검증과 규칙 점검을 통과한 인자 목록. 빠진 선택 인자에는 명세의 default가 채워져 있다
  list(
    tables   = list(<표 이름> = <data.frame>, ...),   # 필수. 이름은 명세 outputs.tables와 같게
    figures  = list(save_figure(...), ...),            # 선택. 이름은 명세 outputs.figures와 같게
    notes    = c("사용한 검정: ...", ...),             # 선택. 한국어
    warnings = c("W_코드: 내용", ...)                  # 선택
  )
}
```

- wrapper가 불릴 때는 이미 열 이름, 형식, 범위, 규칙 점검이 끝난 상태
  - 같은 검사를 반복하지 않고 분석에 집중
  - 단 완전사례로 줄인 뒤에야 알 수 있는 문제(사건이 하나도 남지 않음, 그룹이 하나만 남음 등)는 wrapper가 확인하고 `skill_error()`로 막음(섹션 6)
- p에 없는 선택 인자는 `p[["이름"]]`이 `NULL` → `if (!is.null(p[["group"]]))`처럼 분기
- 아래는 가장 단순한 wrapper인 `JST-TB1-TTEST`의 실행 경로 전체를 `tb1.R`에서 옮긴 것
  - 계열 공통 도우미 세 개(`tb1_prepare`, `tb1_run`, `tb1_output`)와 Skill 함수 하나로 구성

```
# 분석에 필요한 열만 남기고 범주형 변수를 factor로 맞춘다.
# (jstable의 normalityTest는 data의 모든 숫자형 열을 검정하므로 열을 줄여서 넘겨야 한다)
# factor_vars를 주지 않으면 숫자형이 아닌 변수만 범주형으로 본다.
tb1_prepare <- function(data, vars, groups = NULL, factor_vars = NULL) {
  d <- data[, unique(c(vars, groups)), drop = FALSE]
  for (g in groups) d[[g]] <- droplevels(factor(d[[g]]))
  if (is.null(factor_vars)) factor_vars <- vars[!vapply(vars, function(v) is.numeric(d[[v]]),
logical(1))]
  for (v in factor_vars) d[[v]] <- droplevels(factor(d[[v]]))
  list(data = d, factor_vars = if (length(factor_vars) > 0) factor_vars else NULL)
}

tb1_run <- function(data, vars, group = NULL, ..., factor_vars = NULL) {
  prep <- tb1_prepare(data, vars, group, factor_vars)
  args <- list(vars = vars, data = prep$data, factorVars = prep$factor_vars,
               showAllLevels = TRUE, printToggle = FALSE, ...)
  if (!is.null(group)) args$strata <- group
  res <- do.call(jstable::CreateTableOneJS, args)
  df <- matrix_to_df(res$table)
  df$sig <- NULL
  list(table = df, caption = res$caption)
}

tb1_output <- function(run, notes = character()) {
  list(tables = list(table1 = run$table), notes = c(run$caption, notes))
}

# 연속변수 2그룹 비교: Welch t-test
skill_tb1_ttest <- function(data, p) {
  run <- tb1_run(data, p$vars, p$group, test = TRUE, nonnormal = NULL)
  tb1_output(run, "p값: Welch two-sample t-test (oneway.test, var.equal = FALSE)")
}
```

이 코드에 이 장의 규칙 대부분이 들어 있음:

- 필요한 열만 잘라서 넘김(`tb1_prepare`)
- 그룹 변수를 factor로 맞춤
- 기보유 함수의 행렬 출력을 `matrix_to_df()`로 표로 바꿈
- 사용한 검정을 `notes`에 적음

8.2 자료 준비

- 필요한 열만 넘김
  - 기보유 함수 가운데는 넘겨받은 자료의 모든 열을 처리하는 것이 있음
  - 예: `jstable::CreateTableOneJS(normalityTest = TRUE)`는 `data`의 모든 숫자형 열에 정규성 검정을 함 → 분석과 무관한 상수 열이 하나만 있어도 실패(섹션 13)
  - 그래서 wrapper는 분석에 쓰는 열만 남긴 자료를 넘김
- 완전사례와 형식 변환은 wrapper에서 명시적으로 함
  - 범주형으로 쓸 변수는 `droplevels(factor(x))`로 바꿈
  - 결측 행을 뺄 때는 어떤 열 기준인지 분명히 함
  - 기보유 함수가 내부에서 결측을 다르게 처리하면(예: 단변량과 다변량 모형의 표본이 달라지는 경우) 결과의 의미가 바뀜 → 모형 변수의 완전사례만 넘기는 식으로 표본을 고정
  - 실제로 쓴 표본 수를 `notes`에 적음(예: "bili·albumin·sex에 결측이 없는 312쌍 사용")
- 자료를 고치지 않음
  - `data`는 그 실행에서만 쓰는 사본이지만 전역 환경이나 다른 객체를 바꾸지 않음
  - 파일은 `save_figure()` 말고는 쓰지 않음
  - 네트워크에 접근하지 않음

8.3 결과표

- 집계값만 담음
  - 넣는 것: 요약통계, 검정 통계량, 추정치, 건수
  - 넣지 않는 것: 개인 단위의 값(행 단위 자료, 개별 예측값, 잔차 목록 등). 결과표는 LLM과 보고서로 가기 때문
  - agent의 privacy guard는 결과표 하나가 200행을 넘으면 행 단위 자료가 섞였을 수 있다고 보고 LLM에 넘기지 않음(`E_PRIVACY`)
  - 시점별 생존율처럼 행이 많아질 수 있는 표는 시점 간격 인자로 행 수를 제한
- 건수 열의 이름을 알아보기 쉽게 지음
  - 업로드 자료(`up_*`)로 실행한 결과는 LLM에 넘기기 전에 소표본 셀(1 ~ k−1건, 기본 k = 5)을 `<5`로 가림
  - 건수로 알아보는 기준: 열 이름이 `n`, `count`, `freq`, `events`, `n_event`, `n.risk`, `cases`, `건수`, `빈도` 같은 이름인지, 셀이 `12 (34.5)` 꼴의 빈도(백분율)인지
  - Table 1의 `variable`이 `n`인 행은 행 전체를 건수로 봄
  - 건수를 담는 열에 다른 이름을 붙이면 가려지지 않음
- 기보유 함수의 행렬 출력은 `matrix_to_df()`로 바꿈
  - jstable과 tableone이 돌려주는 문자 행렬은 행 이름에 변수 이름이 있고 범주 수준 행은 행 이름이 비어 있음
  - `matrix_to_df()`는 비어 있는 행 이름을 바로 위 변수 이름으로 채워 `variable` 열로 옮김
  - 모든 셀을 공백을 다듬은 문자열로 바꾸고 `NA`를 빈 문자열로 바꿈

```r
# jstable/tableone이 돌려주는 문자 행렬을 data.frame으로 바꾼다.
# 행 이름(변수명)이 비어 있는 행은 직전 변수의 하위 범주이므로 변수명을 채운다.
matrix_to_df <- function(m, row_col = "variable") {
  m <- as.matrix(m)
  rn <- rownames(m) %||% rep("", nrow(m))
  rn <- trimws(rn)
  for (i in seq_along(rn)) if (i > 1 && !nzchar(rn[i])) rn[i] <- rn[i - 1]
  df <- as.data.frame(m, stringsAsFactors = FALSE, check.names = FALSE)
  df[] <- lapply(df, function(x) { x <- trimws(as.character(x)); x[is.na(x)] <- ""; x })
  df <- cbind(stats::setNames(data.frame(rn, stringsAsFactors = FALSE), row_col), df)
  rownames(df) <- NULL
  df
}
```

- 열 이름과 표 이름은 명세와 똑같이
  - 시험 실행기는 열 이름을 문자 그대로 대조(섹션 4.4)
  - 기보유 함수가 버전이나 옵션에 따라 다른 열 이름을 내면 wrapper에서 이름을 고정
  - 기보유 함수가 디버깅용·내부용 열(jstable의 `sig` 같은)을 붙이면 지움
- 숫자는 보고 자릿수로 반올림
  - JSON으로 나갈 때 소수 넷째 자리에서 잘림(섹션 7) → 표의 숫자는 wrapper에서 의도한 자릿수로 반올림

  - p값은 "<0.001" 경계 표기를 위해 문자열로 만듦
  - 표기 규칙(소수 셋째 자리, 0.001 미만은 <0.001 등)을 notes에 적음

8.4 그림

- 그림은 save_figure()로 저장하고 그 반환값을 figures 목록에 넣음

```
save_figure <- function(plot, name, title, width = 7, height = 5, dpi = 150) {
  dir <- figure_dir()
  tmp <- tempfile(fileext = ".png", tmpdir = dir)
  ragg::agg_png(tmp, width = width, height = height, units = "in", res = dpi)
  tryCatch({
    if (inherits(plot, c("gtable", "grob", "gTree"))) grid::grid.draw(plot) else print(plot)
  }, finally = grDevices::dev.off())
  hash <- digest::digest(file = tmp, algo = "sha256")
  final <- file.path(dir, paste0(hash, ".png"))
  file.rename(tmp, final)
  list(name = name, title = title, format = "png", file = basename(final), sha256 = hash,
       width_in = width, height_in = height, dpi = dpi)
}
```

- ggplot 객체와 grid 객체(gtable, grob, gTree) 모두 받음
- ragg로 PNG를 그린 뒤 내용 해시를 파일 이름으로 바꿈 → 같은 그림이면 같은 파일 이름
- name은 명세 outputs.figures의 이름과 같게, title은 사람이 읽는 제목(한국어 가능)으로
- 그림 안의 문구(축 이름, 범례, 라벨)는 영문
  - 실행 환경에 한글 글꼴이 없을 수 있음
  - 해설은 표와 notes에 한국어로
- 그림에 표시한 수치(중앙생존시간, log-rank p, AUC, 상관계수 등)는 같은 계산으로 표에도 냄. LLM은 그림을 보지 못함
- 그림을 그리는 함수가 예상된 경고(예: 쌍별 결측 제외로 "Removed n rows")를 내면 그 경고만 골라 흡수할 수 있음 (cor.R의 skill_cor_matrix() 참고)
  - 나머지 경고는 실행 틀이 모음

8.5 경고와 주석

- notes·warnings는 문자 벡터로 돌려주면 됨 → 원소가 하나여도 실행 틀이 JSON 배열로 냄(섹션 7)
- wrapper 실행 중 R 경고는 실행 틀이 모아 warnings에 넣음 → 따로 처리하지 않아도 됨
- 사용자가 의미를 알 수 있게 하려면 알려진 경고를 코드가 붙은 문장으로 바꿔 직접 warnings에 넣음
  - 예: 회귀 계열의 reg_capture()와 reg_code_warnings()가 로지스틱 회귀의 "fitted probabilities numerically 0 or 1"을 W_SEPARATION: …으로 바꿈
  - 이렇게 쓴 W_ 코드도 명세 errors에 action: warn으로 정의해 둠
- notes에는 결과를 해석하는 데 필요한 정보를 한국어로 씀
  - 사용한 검정·모형과 그 옵션(예: "p값: Welch two-sample t-test (oneway.test, var.equal = FALSE)")
  - 표본 수와 결측 처리
  - 다중비교 보정 여부
  - 표기 규칙
  - 기보유 함수와 다르게 계산한 부분이 있으면 그 사실

8.6 난수 시드

- 순열 검정, bootstrap, Monte Carlo, cross-validation처럼 난수를 쓰는 분석은 같은 입력에서 같은 결과가 나와야 함
  - 품질 점검이 같은 분석을 다시 실행해 결과표가 같은지 비교하기 때문
- core.R의 with_seed()로 고정 시드에서 실행 → 실행 전의 전역 난수 상태가 복원됨

```
# 난수를 쓰는 분석(순열·부트스트랩·몬테카를로)은 고정 시드로 실행하고 호출 전 난수 상태를 되돌린다.
# 같은 입력이면 같은 결과가 나와야 재현성 점검(품질검증)과 일관성 지표가 성립한다.
with_seed <- function(seed, expr) {
  had <- exists(".Random.seed", envir = globalenv(), inherits = FALSE)
  old <- if (had) get(".Random.seed", envir = globalenv()) else NULL
  on.exit({
```

```
    if (had) assign(".Random.seed", old, envir = globalenv())
    else if (exists(".Random.seed", envir = globalenv(), inherits = FALSE)) rm(".Random.seed",
envir = globalenv())
  }, add = TRUE)
  set.seed(seed)
  expr
}
```

- 시드를 정하는 두 가지 방식
  - 사용자가 바꿀 수 있게 하려면 `seed` 입력 인자(`type: integer`, `default`, `min: 1`)를 둠
  - 그렇지 않으면 wrapper 안의 상수로 고정
  - 어느 쪽이든 쓴 시드를 `notes`에 적음
- 기존 계열의 예
  - `km.R`의 `km_with_seed()`: 난수 생성기 종류(`kind = "Mersenne-Twister"`, `normal.kind = "Inversion"`, `sample.kind = "Rejection"`)까지 고정
  - `ps.R`의 `ps_with_seed()`: 상수 시드
- 기보유 함수가 내부에서 시드를 다시 고정해 사용자 시드가 결과에 영향을 주지 않는 경우도 있음(`ml.R`의 K-평균 군집 주석 참고) → 시드 인자를 두기 전에 실제로 결과가 바뀌는지 확인

8.7 formula 만들기

- 사용자 입력으로 모형식을 만들 때 문자열을 이어 붙여 `eval(parse())`하지 않음
- 열 이름은 입력 검증에서 R 문법에 맞는 이름만 통과(섹션 4.2) → `stats::reformulate()`에 문자 벡터로 넘기면 됨
- `Surv()`처럼 함수 호출이 응답변수가 되는 경우에는 `as.name()`과 `as.call()`로 호출 객체를 만듦. `cox.R`의 방식:

```
# survival::Surv(time, event) 호출
cox_surv <- function(time, event) as.call(list(quote(survival::Surv), as.name(time),
as.name(event)))

# coxph 적합. data_sym은 데이터 symbol(기본은 이 함수의 d), extra_args는 cluster·id 등 추가 인자
cox_fit <- function(d, p, terms, extra_args = list(), data_sym = quote(d)) {
  env <- environment()
  f <- stats::reformulate(terms, response = cox_surv(p$time, p$event))
  cl <- as.call(c(list(quote(survival::coxph), formula = f, data = data_sym, model = TRUE),
extra_args))
  eval(cl, envir = env)
}
```

- 기보유 함수가 적합 객체의 `call`을 다시 평가하는 경우(crude 모형 재적합 등)
  - 데이터와 모형식이 `call` 안에 기호나 값으로 제대로 박혀 있어야 함
  - 위의 `cox_fit()`처럼 `as.call()`로 호출을 만들어 평가하거나 `do.call()`에 값을 넘김
  - 어떤 방식이 맞는지는 기보유 함수의 코드를 읽고 정함

8.8 `[[ ]]`로 읽기

- 규칙 함수와 wrapper에서 인자는 `r[["column"]]`, `p[["center"]]`처럼 `[[ ]]`로 읽음
- R의 `$`는 목록에서 이름이 정확히 일치하는 원소가 없으면 부분 일치하는 원소를 돌려줌
  - `column` 키가 없을 때 `r$column`은 `columns`를 집어 옴
  - `center` 인자가 없을 때 `p$center`는 `centered`를 집어 옴
  - 두 경우 모두 실제로 겪은 오류
- 기존 코드에는 `p$vars`처럼 `$`를 쓴 곳이 남아 있음
  - 그 인자 이름이 다른 인자 이름의 앞부분이 되지 않는 경우에만 안전
  - 예: 인자 `group`과 `group2`를 함께 두고 `group`을 선택 인자로 하면 `group`을 주지 않았을 때 `p$group`이 `group2`의 값을 돌려줌
- 새 코드에서는 `[[ ]]`만 씀

8.9 전역 객체를 피할 수 없을 때

- 일부 기보유 함수는 적합 객체의 `call`에 적힌 데이터 이름으로 전역 환경에서 자료를 다시 찾음

  - 함수 안에서 적합한 모형을 넘기면 실패
  - 같은 이름의 다른 전역 객체가 있으면 그것으로 조용히 틀린 값을 계산(섹션 13의 A1)
- 이런 경우에만 고정된 임시 이름으로 전역에 자료를 둠
  - 이미 그 이름이 있으면 멈춤
  - 끝나면 반드시 지움

```
cox_with_global_data <- function(d, fn) {
  nm <- cox_global_data_name
  if (exists(nm, envir = globalenv(), inherits = FALSE)) stop("임시 데이터 이름이 이미 사용 중: ",
nm)
  assign(nm, d, envir = globalenv())
  on.exit(rm(list = nm, envir = globalenv()), add = TRUE)
  fn(as.name(nm))
}
```

- 이 방식 때문에 R 실행 서비스는 한 프로세스에서 요청을 하나씩만 처리(섹션 2)

8.10 금지 사항

- 임의 코드 실행: `eval(parse(text = ...))`, `source()`, `system()` 등 사용자 입력을 코드로 해석하는 모든 경로
- 데이터셋을 경로로 읽기. 데이터는 `data` 인자로만 받음
- 패키지 설치나 `library()`로 전역 attach
  - 함수는 `패키지::함수`로 부름
  - 기보유 함수가 attach된 패키지를 전제로 동작하는 경우(섹션 13의 B12, S10)는 필요한 범위에서만 처리하고 원래대로 되돌림
- 콘솔 출력에 의존하기
  - 실행 틀이 콘솔 출력과 메시지를 버림
  - 결과는 반환값으로만 전달

9 테스트 규칙

9.1 테스트 대상

- Skill마다 테스트 파일 `r-executor/tests/testthat/test-<ID>.R`을 하나 둠
- testthat 3판(edition 3)으로 실행(`r-executor/DESCRIPTION`의 `Config/testthat/edition: 3`)

한 파일에 최소한 들어갈 것:

1. 정상 사례 1개 이상
   - 예제와 같은(또는 비슷한) 인자로 실행해 성공하는지 확인
   - 결과표의 값을 독립 기준값과 대조. 독립 기준값은 명세 `acceptance.reference`에 적은 방법으로 기보유 패키지를 거치지 않고 계산한 값
   - 비교는 `acceptance.criterion`에 적은 자릿수로
2. 오류 사례 2개 이상
   - 명세에 정의한 오류 코드로 막히는지, 안내 문구(`recovery_hint`)가 비어 있지 않은지 확인
   - 이 Skill에만 있는 규칙(그룹 수, 사건 수, 계열 전용 규칙 등)이 실제로 위반을 알리는 자료를 만들어 넣음
3. 그림을 내는 Skill이면 그림 확인(`expect_figure()`), 경고 규칙이 있으면 그 경고가 나오는 사례

- 현재 등록부의 Skill 테스트 파일에서 오류 사례 확인 함수(`expect_skill_error`)를 부르는 횟수(최소 2회)

오류 사례 확인(expect_skill_error) 수	Skill 수
2	50
3	47
4	8
5	1

- 가장 짧은 예인 `JST-TB1-TTEST`의 테스트 전체

```r
test_that("JST-TB1-TTEST 정상: Welch t-test p값이 기준값과 일치", {
  res <- run("JST-TB1-TTEST", "colon_recur", list(vars = c("age", "nodes"), group = "sex"))
  expect_skill_ok(res)
  d <- resolve_dataset("colon_recur")
  for (v in c("age", "nodes")) {
    ref <- t.test(d[[v]] ~ d$sex, var.equal = FALSE)$p.value
    expect_equal(cell(res$tables$table1, v, "p"), fmt_p(ref), info = v)
  }
  expect_equal(res$provenance$r_function, "jstable::CreateTableOneJS")
})

test_that("JST-TB1-TTEST 오류: 3그룹 변수는 E_GROUP_COUNT", {
  res <- run("JST-TB1-TTEST", "colon_recur", list(vars = c("age"), group = "rx"))
  expect_skill_error(res, "E_GROUP_COUNT")
  expect_equal(res$error$action, "ask_user")
})

test_that("JST-TB1-TTEST 오류: 범주형 변수는 E_TYPE_MISMATCH", {
  res <- run("JST-TB1-TTEST", "colon_recur", list(vars = c("obstruct"), group = "sex"))
  expect_skill_error(res, "E_TYPE_MISMATCH")
})

test_that("JST-TB1-TTEST 오류: 없는 변수는 E_COL_NOT_FOUND와 유사 변수명 제안", {
  res <- run("JST-TB1-TTEST", "colon_recur", list(vars = c("agee"), group = "sex"))
  expect_skill_error(res, "E_COL_NOT_FOUND")
  expect_match(res$error$recovery_hint, "age")
})
```

- 정상 사례: 같은 자료를 `resolve_dataset()`으로 읽어 `stats::t.test()`로 p값을 직접 계산하고 결과표의 p값과 비교
- 오류 사례 세 가지
  - 3그룹 변수(`E_GROUP_COUNT`, action까지 확인)
  - 범주형 변수(`E_TYPE_MISMATCH`)
  - 없는 변수(`E_COL_NOT_FOUND`와 유사 변수명 안내)

9.2 공통 도우미

- `helper-skills.R`은 testthat이 테스트 파일보다 먼저 읽음
- 실행 틀 전체를 읽고(`load.R`) 명세를 한 번 읽어 둔 뒤 아래 함수를 제공

```r
# 테스트 공통 도구: 실행 틀 로드, 기준값 형식화, 결과표 조회
source(test_path("..", "..", "R", "load.R"))
specs <- load_skill_specs()

run <- function(id, dataset, params) run_skill(id, dataset, params, specs)

# tableone 방식 p값 표기: 소수 셋째 자리, 0.001 미만은 "<0.001"
fmt_p <- function(p) if (p < 0.001) "<0.001" else sprintf("%.3f", round(p, 3))

# 결과표에서 변수의 첫 행(변수명 뒤에 " (%)" 등 표시가 붙을 수 있음). 정규식 대신 고정 문자열로 비교한다
row_of <- function(tab, var) which(tab$variable == var | startsWith(tab$variable, paste0(var,
" ")))[1]
cell <- function(tab, var, col) tab[[col]][row_of(tab, var)]

expect_skill_ok <- function(res) {
  expect_equal(res$status, "ok", info = if (!is.null(res$error)) res$error$message)
}
expect_skill_error <- function(res, code) {
  expect_equal(res$status, "error")
  expect_equal(res$error$code, code, info = res$error$message)
  expect_true(nzchar(res$error$recovery_hint))
}
```

```r
# 그림 결과 확인: 메타데이터가 있고, 파일이 실제 PNG이며 내용 해시가 파일 이름과 같음
expect_figure <- function(res, name) {
  figs <- Filter(function(f) identical(f$name, name), res$figures)
  expect_length(figs, 1)
  f <- figs[[1]]
  path <- file.path(figure_dir(), f$file)
  expect_true(file.exists(path))
  expect_equal(readBin(path, "raw", 8), as.raw(c(0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a,
0x0a)))
  expect_gt(file.info(path)$size, 2000)
  expect_equal(digest::digest(file = path, algo = "sha256"), f$sha256)
  invisible(f)
}
```

함수	하는 일
`run(id, dataset, params)`	`run_skill()`을 같은 프로세스에서 부름(별도 프로세스·HTTP 없이)
`fmt_p(p)`	기준 p값을 결과표 표기로 바꿈. 소수 셋째 자리 반올림, 0.001 미만은 `"<0.001"`
`row_of(tab, var)`, `cell(tab, var, col)`	Table 1 형식 표에서 변수의 첫 행과 그 칸. 변수 이름 뒤에 `" (%)"` 같은 표시가 붙어도 찾음
`expect_skill_ok(res)`	`status`가 `"ok"`인지. 실패하면 오류 메시지를 함께 보여 줌
`expect_skill_error(res, code)`	`status`가 `"error"`이고 코드가 `code`이며 `recovery_hint`가 비어 있지 않은지
`expect_figure(res, name)`	그 이름의 그림 메타데이터가 정확히 하나 있는지, 파일이 있는지, PNG 서명으로 시작하는지, 2,000바이트보다 큰지, 내용 SHA-256이 메타데이터와 같은지

9.3 기준값 대조 요령

- 같은 자료를 같은 방식으로 줄임
  - 기준값 계산도 `resolve_dataset()`으로 같은 전처리를 거친 자료에서 시작
  - Skill이 쓰는 완전사례 기준(어떤 열에 결측이 없는 행인지)을 그대로 따름
  - 예: `JSM-COR-PEARSON` 테스트는 `complete.cases(d[, c("bili", "albumin", "sex")])`로 줄인 뒤 그룹별로 나눔
- 표시 자릿수로 비교
  - 결과표가 소수 셋째 자리로 반올림한 값이면 기준값도 `round(x, 3)`으로 반올림해 `expect_equal()`로 비교
  - 허용오차로 느슨하게 비교하지 않음
  - p값은 `fmt_p()`로 같은 문자열을 만듦
- 기보유 함수가 틀리는 경우를 사례에 넣음
  - wrapper가 기보유 함수의 버그를 우회했다면(섹션 13) 우회하지 않았을 때 값이 달라지는 자료를 정상 사례에 넣음 → 우회가 계속 유지되는지 고정
  - 예: subgroup Cox 명세의 검수 기준에 "노출 변수명이 공변량 이름 안에 들어 있어도 기준값과 일치"가 들어 있음
- 건수와 자유도도 비교
  - p값만 맞고 표본 수가 다르면 다른 분석
  - `n`, `df`, 사건 수처럼 정수인 값은 정확히 일치해야 함
- provenance 한 가지는 확인해 둠
  - `JST-TB1-TTEST` 테스트처럼 `res$provenance$r_function`을 확인하면 명세와 wrapper가 이어져 있는지 함께 봄
- 테스트는 난수나 네트워크에 의존하지 않게 씀
  - 난수를 쓰는 Skill은 시드를 고정했으므로(섹션 8) 같은 인자로 두 번 실행해 결과표가 같은지 확인하는 사례를 넣어도 좋음

9.4 실행

- 저장소 루트에서 실행

```
# Skill 하나만
Rscript -e 'testthat::test_file("r-executor/tests/testthat/test-JSM-COR-PEARSON.R")'

# 전체(결과를 r-executor/tests/results/junit.xml에 기록)
Rscript r-executor/tests/testthat.R
```

- 한 파일만 실행하는 명령으로 `test-JSM-COR-PEARSON.R`을 돌린 결과: `[ FAIL 0 | WARN 0 | SKIP 0 | PASS 49 ]`
  - 2026-10-12, 이 문서의 작성 환경(로컬 macOS, R 4.5.1, 저장소 `.r-lib`)
- 전체 실행(`tests/testthat.R`)
  - 저장소의 `.r-lib`을 라이브러리 경로 맨 앞에 두고 `tests/testthat` 폴더 전체를 돌림
  - 진행 상황을 화면에 보여 주면서 JUnit XML(`r-executor/tests/results/junit.xml`)을 남김
  - 실패나 오류가 하나라도 있으면 종료 코드 1로 끝남
- 등록부 생성기는 이 JUnit 파일을 읽어 Skill별 '테스트 결과' 칸을 채움 → 등록부를 만들기 전에 전체 테스트를 먼저 돌림(섹션 10)
- JUnit 파일 구조
  - 묶음(testsuite): 테스트 파일 하나에 대응. 이름은 파일 이름에서 `test-`와 `.R`을 뗀 것
  - 묶음 안의 항목(testcase): `test_that()` 블록이 아니라 기대 확인(expectation) 하나하나
  - 등록부의 "검증 N건"도 이 기대 확인 수
- 컨테이너 이미지에는 테스트 도구(testthat, withr, xml2)가 들어 있지 않음 → 컨테이너 안에서 테스트를 돌릴 때는 그때만 이 패키지들을 설치

9.5 현재 테스트 결과

- 저장소에 남아 있는 마지막 전체 실행 결과(`r-executor/tests/results/junit.xml`)
- 실행 시각: 2026-10-27T13:25:56Z ~ 2026-10-27T13:43:40Z(UTC)

항목	값
테스트 묶음(파일)	109개(Skill 106개, 공통 틀· 기능 `core`, `datacheck`, `upload`)
기대 확인 수	전체 3,973건(Skill 테스트 3,722건)
실패·오류·건너뜀	0 · 0 · 0

- 이 결과로 판정하면 등록 Skill 106종 가운데 106종이 통과
  - 판정 기준은 등록부 생성기와 같음: 실패·오류 0건, 기대 확인 1건 이상
- 공통 틀 테스트(`test-core.R`)가 확인하는 것
  - 등록되지 않은 Skill·데이터셋 차단
  - provenance 기록
  - 데이터셋 구조에 행 단위 값이 없는지
  - 모든 명세의 wrapper 함수 존재
  - R 수준 timeout과 프로세스 종료에 의한 timeout
  - R 문법에 맞지 않는 변수명 차단
  - 그림 해시
  - `validate_skill()` 동작

10 등록부 생성·점검

10.1 실행

- 등록부 생성기는 저장소 루트에서 실행
- 테스트 결과를 등록부에 넣음 → 전체 테스트를 먼저 돌림

```
Rscript r-executor/tests/testthat.R     # junit.xml을 먼저 만든다
Rscript registry/build_registry.R
```

- 진행 순서: `load.R`로 실행 틀과 모든 wrapper를 읽음 → 명세 점검 → 이름 충돌 점검 → 예제 실행 → 테스트 결과 집계 → 출력
- 실행 틀 외에 필요한 패키지: `jsonvalidate`(JSON Schema 검증, ajv 엔진), `xml2`(JUnit 읽기), `openxlsx`(엑셀 출력)
- 이 셋은 R 실행 서비스 이미지에 들어 있지 않음 → 생성기는 개발 환경에서 돌림

10.2 점검 항목

- 생성기가 문제로 잡는 항목은 아래가 전부
- 문제마다 `[Skill ID] 내용` 한 줄로 출력

단계	점검	출력되는 문구
명세	JSON Schema 검증. YAML 목록은 원소가 하나여도 배열로 유지한 채 검증	`스키마: <위치> <내용>`
명세	파일 이름이 `<id>.yaml`인지	`파일명이 ID와 다름`
명세	`origin`의 각 항목이 `inventory.csv`의 `패키지::함수`에 있는지	`기보유 출처가 inventory에 없음: …`
명세	`r_function`의 각 함수가 설치된 패키지 이름공간에 있는지	`연결 R 함수가 없음: …`
명세	`implementation` 함수가 있는지	`wrapper 함수 없음: …`
명세	`tests` 파일이 있는지	`테스트 파일 없음: …`
명세	규칙마다: 이름이 `rule_funs`에 있는지	`알 수 없는 규칙: …`
명세	규칙마다: `rule_required_keys`의 키가 모두 있는지	`규칙 <이름>의 필수 키 누락: …`
명세	규칙마다: `error` 코드가 명세 `errors`나 공통 정의에 있는지	`정의되지 않은 오류 코드: …`
이름 충돌	`skills/*.R`, `core.R`, `isolate.R`, `datasets*.R`에서 최상위에 대입한 이름과 `rule_funs`, `rule_required_keys`, `dataset_catalog`에 추가한 항목 이름을 두 파일 이상이 정의하는지	`[이름 충돌] <이름> 를 여러 파일이 정의: …`
예제	첫 번째 예제를 `run_skill()`로 실행해 `status`가 `ok`인지	`예제 실행 실패: <코드> <메시지>`

생성기가 점검하지 않는 것(테스트와 시험 실행기, 작성자가 챙김):

- wrapper 안에서 `skill_error()`로 내는 코드가 명세 `errors`에 있는지
- 실제 결과의 표 이름·열이 명세 `outputs`와 맞는지(시험 실행기가 판정, 섹션 15)
- `default`가 형식·범위·`values`에 맞는지
- YAML 1.1 때문에 이름이 바뀐 입력 인자(섹션 4.9)
- 오류 사례 테스트가 2개 이상인지, 기준값이 독립적인지

10.3 출력 3종

- 생성기는 `registry/` 아래에 세 파일을 씀
- 모두 생성물 → 직접 고치지 않음
- `registry.json`(agent용)
  - 구조: `{"generated_at": "<생성 시각>", "skills": [<명세>, …]}`

  - `skills`는 YAML 명세를 읽은 내용 그대로이며 Skill ID 순으로 정렬
  - agent는 이 파일로 도구 정의와 카탈로그를 만들고 실행 요청에 이 파일의 `version`을 실음
  - YAML 목록은 원소가 하나여도 배열로 기록. YAML에 값 하나로 쓴 항목(`r_function` 등)은 문자열(섹션 4.9)
- `registry.xlsx`(모듈별 상세 등록부): 시트 세 개
  - `등록부`: Skill마다 한 행, 열 18개. 열은 "고유 ID", "기능명", "분석 방법", "범주", "연결 R 함수", "기보유 출처", "버전", "필수 인자", "선택 인자", "데이터 검증 규칙", "실행 조건", "출력 형식", "오류 처리 로직", "결과 검수 기준", "예제 데이터", "예제 실행", "테스트 결과", "테스트 파일"
  - `데이터셋`: 허용 목록 전체의 ID, 출처, 설명
  - `생성 정보`: 생성 시각, Skill 수, 테스트 통과 Skill 수, R 버전, 기준 패키지 버전(provenance의 `packages`와 같은 목록)
- `REGISTRY.md`(사람이 읽는 요약)
  - 머리: 생성 시각, 등록 Skill 수와 테스트 통과 수, 기준 패키지 버전
  - 표: Skill마다 ID·기능명·분석 방법·기보유 출처·테스트 결과를 한 줄씩
  - 현재 저장소의 `REGISTRY.md`는 2026-10-27 13:58:45 UTC에 생성. 요약은 "106종 (테스트 통과 106종)"

`등록부` 시트의 칸 형식(여러 항목은 한 칸에 줄바꿈해 넣음):

칸	형식
필수·선택 인자	`이름 [형식, dtype, 최소 개수, 값 목록]: 설명`
규칙	`규칙(키=값, …) → 오류 코드`
오류 처리	`코드 [action] 메시지 → 안내`
'예제 실행'	생성기가 방금 실행한 첫 예제의 결과(`ok`/`error`)
'테스트 결과'	JUnit에서 읽은 "통과 (검증 N건, 실패 0, 오류 0, 실행 시각)" 또는 "실패 (…)". JUnit에 그 Skill이 없으면 "미측정"

10.4 성공·실패 시 출력

- 2026-10-12 09:10(KST)에 이 문서의 작성 환경에서 생성기를 출력 경로만 임시 폴더로 바꿔 돌린 결과(저장소의 생성물은 건드리지 않음)
- 첫 예제 106개를 모두 실행 → 이 환경에서 221초 걸림
- 문제가 없을 때 마지막 두 줄:

```
등록부 생성: 106종, 테스트 통과 106종
명세 점검: 문제 없음
```

- 문제가 하나라도 있으면 문제 목록을 출력하고 종료 코드 1로 끝남
- 단 출력 3종은 문제 목록을 출력하기 **전에** 이미 쓰임
  - 실패한 실행도 `registry.json`과 `registry.xlsx`를 덮어씀
  - 종료 코드를 확인하지 않으면 문제가 있는 명세가 그대로 agent에 반영될 수 있음
- 실패했으면 고친 뒤 다시 돌려 "명세 점검: 문제 없음"을 확인

실패 예시:

- 실패할 때 어떻게 보이는지 확인하려고 일부러 틀린 명세 하나(`JSM-COR-DEMO`)를 정상 명세 하나(`JSM-COR-PEARSON`)와 함께 임시 폴더에 두고 생성기를 돌림(2026-10-12)
- 틀린 명세:

```yaml
id: JSM-COR-DEMO
name: 점검 실패 예시
method: Pearson correlation (demo)
category: 상관분석
description: 등록부 점검이 실패하는 모습을 보이려고 만든 명세(문서용, 저장소에 넣지 않음)
r_function: stats::cor.tst
origin: jsmodule::scatterServer
implementation: skill_cor_demo
version: 0.1
```

```yaml
inputs:
  dataset: {type: dataset_ref, required: true, description: "분석 데이터셋 ID"}
  x: {type: column, dtype: numeric, required: true, description: "첫째 연속변수"}
  y: {type: column, dtype: numeric, required: true, description: "둘째 연속변수"}
validation:
  - {rule: n_levels, min: 2, error: E_GROUP_COUNT}
preconditions: []
outputs:
  tables:
    correlation: {description: "상관계수", columns: [variable1, variable2, n, r]}
errors:
  E_COL_NOT_FOUND: {action: ask_user, message: 데이터에 없는 변수, hint: 유사 변수명을 확인}
acceptance: {reference: "stats::cor.test로 독립 계산", criterion: "소수 셋째 자리 반올림 값 일치"}
examples:
  - dataset: pbc
    request: 빌리루빈과 알부민의 상관을 봐줘
    params: {x: bili, y: albumin}
tests: r-executor/tests/testthat/test-JSM-COR-DEMO.R
```

출력(종료 코드 1):

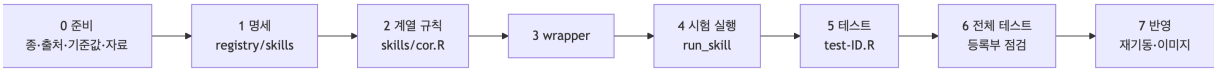
```
등록부 생성: 2종, 테스트 통과 1종
명세 점검 문제:
  [JSM-COR-DEMO] 스키마: /version must be string
  [JSM-COR-DEMO] 스키마: /outputs/tables/correlation/columns/2 must be string
  [JSM-COR-DEMO] 연결 R 함수가 없음: stats::cor.tst
  [JSM-COR-DEMO] wrapper 함수 없음: skill_cor_demo
  [JSM-COR-DEMO] 테스트 파일 없음: r-executor/tests/testthat/test-JSM-COR-DEMO.R
  [JSM-COR-DEMO] 규칙 n_levels의 필수 키 누락: column
  [JSM-COR-DEMO] 정의되지 않은 오류 코드: E_GROUP_COUNT
  [JSM-COR-DEMO] 예제 실행 실패: E_RUNTIME 분석 실행 중 오류: attempt to select less than one
element in get1index
```

각 줄을 읽는 법:

- `/version must be string`: `version: 0.1`이 숫자로 읽힘 → `0.1.0`으로 씀
- `/outputs/tables/correlation/columns/2 must be string`: 열 목록의 세 번째(0부터 세어 2번) 원소 `n`이 논리값 `FALSE`로 읽힘 → `"n"`으로 감쌈
- `연결 R 함수가 없음: stats::cor.tst`: 함수 이름 오타
- `wrapper 함수 없음`, `테스트 파일 없음`: wrapper와 테스트를 아직 만들지 않음
- `규칙 n_levels의 필수 키 누락: column`: 어느 입력 인자를 검사할지 빠짐
- `정의되지 않은 오류 코드: E_GROUP_COUNT`: 규칙이 쓰는 코드를 `errors`에 정의하지 않음
- `예제 실행 실패: E_RUNTIME …`: `column` 키가 없는 `n_levels` 규칙이 실행 중 R 오류를 냄. 앞의 문제를 고치면 함께 없어짐
- 입력 인자 `y`는 어떤 점검에도 걸리지 않음
- 이 실행에서 만들어진 `registry.json`을 열어 보면 이 Skill의 입력 이름은 `["dataset", "x", "TRUE"]`로, 예제 인자는 `{"x": "bili", "TRUE": "albumin"}`으로 기록되어 있었음

11 Skill 한 종 처음부터 만들어 보기

- 등록부에 이미 있는 `JSM-COR-PEARSON`(Pearson 상관분석과 산점도)을 처음부터 만든다고 보고 실제 파일을 따라감
  - 순서: 명세 → 계열 규칙 → wrapper → 시험 실행 → 테스트 → 등록부 점검 → 반영
- Table 1 계열보다 조금 복잡하지만 새 Skill을 만들 때 필요한 요소가 다 들어 있음
  - 계열 전용 규칙, 경고 규칙, 그림, 선택 인자, `enum` 인자
- 아래 명세·코드·테스트는 문서를 만들 때 저장소 파일에서 그대로 읽어 온 것



11.1 0단계: 준비

- 작업 위치: 저장소 루트
- 기준 버전 패키지가 `.r-lib`에 설치되어 있어야 함(`load.R`과 `tests/testthat.R`이 자동으로 그 경로를 씀)
  - 없으면 `r-executor/install_packages.R`로 설치(섹션 12)

코드를 쓰기 전에 정할 네 가지:

1. 이것이 한 종인가
   - Pearson 상관은 Spearman 상관과 같은 jsmodule 화면에서 나옴
   - 그러나 가정(이변량 정규·선형)과 점검(정규성)과 결과(Fisher z 신뢰구간)가 달라 별도 종(섹션 3)
   - 추세선 종류는 표시 방식 → 인자(`line`)로 둠
2. 기보유 출처는 무엇인가
   - 분석 로직이 jsmodule의 산점도 모듈 `scatterServer` 안에 있음 → `origin`은 `jsmodule::scatterServer`
   - `inventory.csv`에 있는지 확인

```
grep '"scatterServer"' registry/inventory.csv
```

   - 모듈 코드를 읽어 실제로 어떤 함수를 어떤 인자로 부르는지 확인
   - 이 모듈은 `ggpubr::ggscatter(cor.coef = TRUE, cor.method = "pearson")`로 그리고 상관계수 라벨은 내부에서 `stats::cor.test()`로 계산 → `r_function`은 `[ggpubr::ggscatter, stats::cor.test]`
   - 확인한 동작은 계열 파일 머리말 주석에 적어 둠(`cor.R` 머리말 참고)
3. 독립 기준값은 무엇인가
   - r, Fisher z 신뢰구간, t 통계량, p값을 공식으로 직접 계산하고 `stats::cor.test()`와도 대조
   - ggpubr를 거치지 않음
4. 예제 자료는 무엇인가
   - 허용 목록의 `pbc`(원발성 담즙성 간경변 임상시험)에 연속변수(빌리루빈 `bili`, 알부민 `albumin`)와 범주 변수(`sex`)가 있음
   - 새 자료가 필요하면 섹션 11.9를 따름

11.2 1단계: 명세

`registry/skills/JSM-COR-PEARSON.yaml`:

```
id: JSM-COR-PEARSON
name: Pearson 상관분석(산점도)
method: Pearson product-moment correlation with Fisher z confidence interval
category: 상관분석
description: >-
  두 연속변수의 선형 상관을 Pearson 상관계수 r로 잰다. 95% 신뢰구간(Fisher z 변환)과 t 검정 p값을 표로 내고,
  산점도에 회귀선(또는 LOESS)과 상관계수 라벨을 그린다(jsmodule scatterServer 방식). 그룹 변수를 주면 전체와 그룹별 상관을 함께 낸다.
  이변량 정규·선형 관계를 가정하며 이상치에 민감하다. 분포가 치우치거나 순서형이면 JSM-COR-SPEARMAN,
  여러 변수의 상관을 한 번에 보려면 JSM-COR-MATRIX를 쓴다. 상관은 인과를 뜻하지 않는다.
r_function: [ggpubr::ggscatter, stats::cor.test]
origin: jsmodule::scatterServer
implementation: skill_cor_pearson
version: 0.1.0
inputs:
  dataset: {type: dataset_ref, required: true, description: "분석 데이터셋 ID"}
  var1: {type: column, dtype: numeric, required: true, description: "첫째 연속변수(산점도 x축)"}
  var2: {type: column, dtype: numeric, required: true, description: "둘째 연속변수(산점도 y축)"}
  group: {type: column, dtype: categorical, required: false, description: "그룹 변수(선택, 범주 2–10개). 지정하면 그룹별 상관도 계산"}
  line: {type: enum, required: false, default: reg.line, values: [reg.line, loess, none],
```

```
description: "그림의 추세선: reg.line = 선형회귀선, loess = LOESS 평활선, none = 없음"}
validation:
  - {rule: distinct_columns, columns: [var1, var2, group], error: E_ARG_CONFLICT}
  - {rule: not_constant, columns: [var1, var2], error: E_CONSTANT_VARIABLE}
  - {rule: n_levels, column: group, min: 2, max: 10, error: E_GROUP_COUNT}
  - {rule: max_missing_rate, columns: [var1, var2, group], max: 0.2, error: W_MISSING_HIGH}
  - {rule: cor_normality, columns: [var1, var2], alpha: 0.05, error: W_NONNORMAL}
preconditions:
  - {rule: cor_pairs, columns: [var1, var2], group: group, min: 4, error: E_TOO_FEW_PAIRS}
outputs:
  tables:
    correlation:
      description: "전체(group = 전체)와 그룹별 결측 없는 쌍 수 n, Pearson r, 95% CI(Fisher z), t
통계량, 자유도(n - 2), p값"
      columns: [variable1, variable2, group, "n", r, CI_low, CI_high, t, df, p]
  figures: [scatter]
errors:
  E_COL_NOT_FOUND: {action: ask_user, message: 데이터에 없는 변수, hint: 유사 변수명을 제안하고 사용
자에게 확인}
  E_TYPE_MISMATCH: {action: ask_user, message: "var1·var2는 숫자형, group은 범주형이어야 함",
hint: "범주형 변수와 연속변수의 관계는 JSM-VIS-BOX 또는 JST-TB1 계열을 사용"}
  E_INVALID_VALUE: {action: ask_user, message: "line 값이 허용 범위를 벗어나거나 쓸 수 없는 변수명",
hint: "line은 reg.line, loess, none 중 하나"}
  E_ARG_CONFLICT: {action: ask_user, message: 같은 변수가 var1·var2·group에 중복 지정됨, hint: 서
로 다른 변수를 지정}
  E_CONSTANT_VARIABLE: {action: block, message: 값이 하나뿐인 변수는 상관을 계산할 수 없음, hint: 다
른 변수를 선택}
  E_GROUP_COUNT: {action: ask_user, message: 그룹 변수의 범주가 2–10개가 아님, hint: "범주를 합친 그
룹 변수를 지정"}
  E_TOO_FEW_PAIRS: {action: block, message: "결측 없는 쌍이 4개 미만(전체 또는 그룹별)이거나 그룹 안에
서 값이 하나뿐인 변수가 있음(신뢰구간 계산 불가)", hint: "결측을 확인하거나 그룹 범주를 합치거나 그룹 없이
분석"}
  E_RUNTIME: {action: block, message: 상관 계산 중 오류, hint: 입력 변수를 확인}
  W_NONNORMAL: {action: warn, message: Shapiro-Wilk 검정에서 정규분포에서 벗어난 변수가 있음, hint:
"이상치·치우침을 확인하고 JSM-COR-SPEARMAN 결과를 함께 보는 것을 검토"}
  W_MISSING_HIGH: {action: warn, message: 결측 비율이 20%를 넘음, hint: 완전사례 분석임을 결과 해석에
명시}
acceptance:
  reference: >-
    var1·var2(·group) 완전사례 자료에서 r = Σ(x - x̄)(y - ȳ) / sqrt(Σ(x - x̄)² Σ(y - ȳ)²),
    95% CI = tanh(atanh(r) ± qnorm(0.975) / sqrt(n - 3)), t = r·sqrt((n - 2) / (1 - r²)), p =
2·pt(-|t|, n - 2)를
    공식으로 직접 계산하고 stats::cor.test(method = "pearson")와도 대조
  criterion: "r·CI·t는 소수 셋째 자리 반올림 값, p값은 소수 셋째 자리 반올림 값(0.001 미만은 '<0.001')
이 일치하고 n·자유도가 일치. 그림 PNG가 생성되고 해시가 메타데이터와 일치"
examples:
  - dataset: pbc
    request: 빌리루빈과 알부민의 상관관계를 성별로 나눠 산점도와 Pearson 상관계수로 보여줘
    params: {var1: bili, var2: albumin, group: sex, line: reg.line}
tests: r-executor/tests/testthat/test-JSM-COR-PEARSON.R
```

블록별로 짚어 볼 점:

- `inputs`
  - 두 연속변수는 `column` + `dtype: numeric`
  - 그룹 변수는 선택(`required: false`)이고 `dtype: categorical`
  - 추세선은 `enum`이고 `default: reg.line`
  - 기본값은 형식 검사를 거치지 않음 → `values`에 있는 값인지 직접 확인
- `validation`(아래 순서)
  - 세 변수 중복(`distinct_columns`)
  - 상수 변수(`not_constant`)

  - 그룹 범주 수 2~10(`n_levels`, 그룹을 주지 않으면 통과)
  - 결측 비율 경고(`max_missing_rate` → `W_MISSING_HIGH`)
  - 정규성 경고(계열 전용 `cor_normality` → `W_NONNORMAL`)
- `preconditions`
  - 결측 없는 쌍이 전체와 그룹별로 4개 이상인지 계열 전용 규칙 `cor_pairs`로 봄
  - Fisher z 신뢰구간은 `n − 3`으로 나눔 → 쌍이 4개 미만이면 계산할 수 없음
  - 전용 규칙을 만든 이유: 공통 규칙 `min_rows`는 그룹별로 세지 못하고 그룹 안의 상수 변수도 보지 못함
- `outputs`
  - 표 `correlation`의 열을 순서대로 적음
  - `n`은 YAML 1.1에서 논리값이 되므로 따옴표로 감쌈(섹션 4.9)
  - 그룹 행과 전체 행이 같은 표에 있음 → 조건부 열 없음
  - 그림은 `scatter` 하나
- `errors`
  - 규칙이 쓰는 코드: `E_ARG_CONFLICT`, `E_CONSTANT_VARIABLE`, `E_GROUP_COUNT`, `E_TOO_FEW_PAIRS`, `W_NONNORMAL`, `W_MISSING_HIGH`
  - 입력 검증에서 날 수 있는 공통 코드: `E_COL_NOT_FOUND`, `E_TYPE_MISMATCH`, `E_INVALID_VALUE`
  - 그리고 `E_RUNTIME`
  - `E_TYPE_MISMATCH`의 `hint`는 범주형 변수가 들어왔을 때 갈 Skill을 알려 줌
- `acceptance`: 공식과 판정 자릿수, 그림 PNG·해시 확인까지 적음
- `examples`
  - 그룹을 쓰는 예를 첫 예제로 둠
  - 필수 인자와 열 인자를 모두 쓰고 같은 자료에 숫자형 인자 자리에 넣을 범주형 변수(`sex`, `trt` 등)가 있음 → 시험 실행기의 오류 주입 시험이 모두 만들어짐(섹션 15)

11.3 2단계: 계열 파일과 전용 규칙

1. 상관 계열 파일은 `r-executor/R/skills/cor.R`. 새 계열이면 파일을 새로 만듦
2. 파일 머리말에 출처 모듈에서 확인한 동작을 적음
3. 전용 규칙 `cor_pairs`를 씀

```
# 변수 쌍(columns가 3개 이상이면 모든 쌍)마다 둘 다 결측이 없는 관측 수 하한.
# group 키(입력 인자 이름)의 값이 있으면 그룹 수준별로도 점검하고, 그룹 안에서 값이 하나뿐인 변수도 위반으로
본다
rule_funs$cor_pairs <- function(r, data, p) {
  cols <- param_columns(p, r[["columns"]])
  grp <- if (!is.null(r[["group"]])) p[[r[["group"]]]]
  if (length(cols) < 2) return(NULL)
  pairs <- utils::combn(cols, 2, simplify = FALSE)
  bad <- character()
  for (pr in pairs) {
    d <- data[stats::complete.cases(data[, c(pr, grp), drop = FALSE]), c(pr, grp), drop =
FALSE]
    if (nrow(d) < r[["min"]]) bad <- c(bad, sprintf("%s–%s 전체 %d쌍", pr[1], pr[2], nrow(d)))
    if (!is.null(grp)) {
      for (g in sort(unique(as.character(d[[grp]])))) {
        dg <- d[as.character(d[[grp]]) == g, , drop = FALSE]
        const <- any(vapply(pr, function(v) length(unique(dg[[v]])) < 2, logical(1)))
        if (nrow(dg) < r[["min"]] || const) {
          bad <- c(bad, sprintf("%s–%s %s=%s %d쌍%s", pr[1], pr[2], grp, g, nrow(dg), if
(const) "(값이 하나뿐인 변수)" else ""))
        }
      }
    }
  }
  if (length(bad) > 0) sprintf("결측 없는 쌍이 %d개 미만이거나 값이 하나뿐: %s", r[["min"]],
paste(utils::head(bad, 3), collapse = "; "))
}

rule_required_keys$cor_pairs <- c("columns", "min")
```

- 규칙 이름에 계열 접두어 `cor_`를 붙임
- 인자는 `r[["…"]]`로 읽음
- 열 이름은 `param_columns()`와 `p[[r[["group"]]]]`로 입력 인자에서 찾음
- 위반 내용은 앞의 세 건만 보여 줌
- 정규성 경고 규칙 `cor_normality`는 섹션 5에 실음

11.4 3단계: wrapper

- Skill 함수는 짧고 계산은 Pearson과 Spearman이 함께 쓰는 계열 도우미가 함

```
# Pearson 상관계수: r, Fisher z 95% CI, t 검정 p
skill_cor_pearson <- function(data, p) {
  out <- cor_skill(data, p, "pearson")
  out$notes <- c("Pearson 적률상관계수 r, 95% CI는 Fisher z 변환, p는 t = r·sqrt((n-2)/(1-r²)) 자
유도 n-2 검정(stats::cor.test)",
                 "선형 관계만 잰다. 이상치에 민감하며 치우친 분포·순서형 자료는 JSM-COR-SPEARMAN 사용",
out$notes)
  out
}

cor_skill <- function(data, p, method) {
  d <- cor_prepare(data, c(p$var1, p$var2), p$group)
  tab <- cor_table(d, p$var1, p$var2, p$group, method)
  plt <- cor_scatter(d, p, method)
  label <- if (method == "pearson") "Pearson" else "Spearman"
  fig <- save_figure(plt, "scatter", sprintf("%s—%s 산점도와 %s 상관계수", p$var1, p$var2,
label), width = 6.5, height = 5.5)
  list(tables = list(correlation = tab), figures = list(fig), notes = c(
    sprintf("%s·%s%s에 결측이 없는 %d쌍 사용. 그룹을 지정하면 전체와 그룹별 상관을 함께 제시", p$var1,
p$var2,
            if (is.null(p$group)) "" else paste0("·", p$group), nrow(d)),
    cor_line_note(p$line),
    "p값 표기: 소수 셋째 자리, 0.001 미만은 <0.001. 그림 라벨의 계수는 유효숫자 2자리"
  ))
}

cor_prepare <- function(data, cols, group = NULL) {
  d <- data[, unique(c(cols, group)), drop = FALSE]
  d <- d[stats::complete.cases(d), , drop = FALSE]
  if (!is.null(group)) d[[group]] <- droplevels(factor(d[[group]]))
  rownames(d) <- NULL
  d
}

# 전체와 그룹 수준별 상관 표
cor_table <- function(d, v1, v2, group, method) {
  sets <- c(list(`전체` = d), if (!is.null(group)) split(d, d[[group]]))
  rows <- lapply(names(sets), function(g) {
    x <- sets[[g]][[v1]]
    y <- sets[[g]][[v2]]
    ct <- suppressWarnings(stats::cor.test(x, y, method = method))
    if (method == "pearson") {
      data.frame(group = g, n = length(x), r = round(unname(ct$estimate), 3),
                 CI_low = round(ct$conf.int[1], 3), CI_high = round(ct$conf.int[2], 3),
                 t = round(unname(ct$statistic), 3), df = unname(ct$parameter), p =
cor_fmt_p(ct$p.value),
                 stringsAsFactors = FALSE)
    } else {
      data.frame(group = g, n = length(x), rho = round(unname(ct$estimate), 3), S =
round(unname(ct$statistic), 1),
                 p = cor_fmt_p(ct$p.value), p_method = cor_spearman_p_method(x, y),
stringsAsFactors = FALSE)
```

```
    }
  })
  out <- do.call(rbind, rows)
  out$variable1 <- v1
  out$variable2 <- v2
  out[, c("variable1", "variable2", setdiff(names(out), c("variable1", "variable2")))]
}

cor_fmt_p <- function(x) ifelse(is.na(x), "", ifelse(x < 0.001, "<0.001", sprintf("%.3f",
round(x, 3))))
```

- `cor_prepare()`: 필요한 열만 남기고 완전사례로 줄이며 그룹을 factor로 맞춤
- `cor_table()`
  - 전체와 그룹 수준별로 `stats::cor.test()`를 부름
  - 통계량을 소수 셋째 자리로 반올림하고 p값을 `cor_fmt_p()`로 문자열로 만듦
  - 열 순서는 명세의 `columns`와 같음
- `cor_skill()`
  - 산점도를 `save_figure()`로 저장(이름 `scatter`)
  - 사용한 쌍 수와 추세선, p값 표기 규칙을 `notes`에 적음
  - 그림 라벨은 영문
- `skill_cor_pearson()`: 계산 방법과 해석상 주의를 `notes` 앞에 덧붙임

11.5 4단계: 한 번 실행해 보기

- 테스트를 쓰기 전에 R에서 직접 불러 결과 모양을 확인. 테스트와 같은 방식

```
Rscript -e '
source("r-executor/R/load.R"); specs <- load_skill_specs()
res <- run_skill("JSM-COR-PEARSON", "pbc", list(var1 = "bili", var2 = "albumin", group =
"sex"), specs)
cat(jsonlite::toJSON(res, auto_unbox = TRUE, dataframe = "rows", na = "null", pretty = TRUE))'
```

- 출력은 섹션 7의 첫 예시와 같음
- 확인할 것
  - `status`가 `ok`인지
  - 표 이름과 열이 명세와 같은지
  - 그림 메타데이터가 있는지
  - 경고가 의도한 것인지(여기서는 빌리루빈의 치우친 분포 때문에 `W_NONNORMAL`)
- 규칙만 빠르게 확인할 때는 `run_skill` 대신 `validate_skill`을 부름
  - 같은 인자로 부르면 `{"status": "ok", "skill_id": "JSM-COR-PEARSON", "warnings": ["W_NONNORMAL: …"]}`가 나옴
- 오류 경로도 직접 넣어 봄
  - 예: `group = "ph.ecog"`로 `lung` 자료에서 실행 → ph.ecog = 3인 사람이 1명이라 `E_TOO_FEW_PAIRS`로 막혀야 함

11.6 5단계: 테스트

`r-executor/tests/testthat/test-JSM-COR-PEARSON.R`:

```
# JSM-COR-PEARSON: 기준값은 Pearson 공식으로 직접 계산하고 stats::cor.test와도 대조(jsmodule·ggpubr
를 거치지 않음)

pearson_ref <- function(x, y) {
  n <- length(x)
  r <- sum((x - mean(x)) * (y - mean(y))) / sqrt(sum((x - mean(x))^2) * sum((y - mean(y))^2))
  ci <- tanh(atanh(r) + c(-1, 1) * qnorm(0.975) / sqrt(n - 3))
  t <- r * sqrt((n - 2) / (1 - r^2))
  list(n = n, r = r, ci = ci, t = t, p = 2 * pt(-abs(t), n - 2))
}
```

```r
test_that("JSM-COR-PEARSON 정상: 전체·그룹별 r·Fisher z CI·t·p가 공식 계산과 일치", {
  res <- run("JSM-COR-PEARSON", "pbc", list(var1 = "bili", var2 = "albumin", group = "sex"))
  expect_skill_ok(res)
  tab <- res$tables$correlation
  d <- resolve_dataset("pbc")
  d <- d[complete.cases(d[, c("bili", "albumin", "sex")]), ]
  sets <- c(list(`전체` = d), split(d, d$sex))
  expect_equal(tab$group, names(sets))
  for (g in names(sets)) {
    ref <- pearson_ref(sets[[g]]$bili, sets[[g]]$albumin)
    i <- tab$group == g
    expect_equal(tab$n[i], ref$n, info = g)
    expect_equal(tab$df[i], ref$n - 2, info = g)
    expect_equal(tab$r[i], round(ref$r, 3), info = g)
    expect_equal(c(tab$CI_low[i], tab$CI_high[i]), round(ref$ci, 3), info = g)
    expect_equal(tab$t[i], round(ref$t, 3), info = g)
    expect_equal(tab$p[i], fmt_p(ref$p), info = g)
    expect_equal(tab$p[i], fmt_p(cor.test(sets[[g]]$bili, sets[[g]]$albumin)$p.value), info =
g)
  }
  # 빌리루빈은 심하게 치우친 분포 → 정규성 경고
  expect_true(any(startsWith(res$warnings, "W_NONNORMAL")))
  expect_figure(res, "scatter")
})

test_that("JSM-COR-PEARSON 정상: 그룹 없이 결측 쌍을 뺀 완전사례로 계산", {
  res <- run("JSM-COR-PEARSON", "lung", list(var1 = "age", var2 = "meal.cal", line = "none"))
  expect_skill_ok(res)
  d <- resolve_dataset("lung")
  d <- d[!is.na(d$age) & !is.na(d$meal.cal), ]
  ref <- pearson_ref(d$age, d$meal.cal)
  tab <- res$tables$correlation
  expect_equal(nrow(tab), 1)
  expect_equal(tab$n, nrow(d))
  expect_equal(c(tab$r, tab$CI_low, tab$CI_high), round(c(ref$r, ref$ci), 3))
  expect_equal(tab$p, fmt_p(ref$p))
  expect_figure(res, "scatter")
})

test_that("JSM-COR-PEARSON 오류: 그룹 안의 결측 없는 쌍이 4개 미만이면 E_TOO_FEW_PAIRS", {
  # lung의 ph.ecog = 3은 1명
  res <- run("JSM-COR-PEARSON", "lung", list(var1 = "age", var2 = "wt.loss", group =
"ph.ecog"))
  expect_skill_error(res, "E_TOO_FEW_PAIRS")
  expect_match(res$error$message, "ph.ecog=3", fixed = TRUE)
})

test_that("JSM-COR-PEARSON 오류: 범주형 변수는 E_TYPE_MISMATCH, 같은 변수 두 번은 E_ARG_CONFLICT",
{
  res <- run("JSM-COR-PEARSON", "pbc", list(var1 = "bili", var2 = "sex"))
  expect_skill_error(res, "E_TYPE_MISMATCH")
  res2 <- run("JSM-COR-PEARSON", "pbc", list(var1 = "bili", var2 = "bili"))
  expect_skill_error(res2, "E_ARG_CONFLICT")
})
```

- 기준값 함수 `pearson_ref()`는 공식을 그대로 옮김. 기보유 패키지를 부르지 않음
- 첫 정상 사례: 그룹이 있는 경우
  - 전체와 그룹별로 n, 자유도, r, 신뢰구간, t, p를 모두 비교
  - `stats::cor.test()`의 p값과도 비교
  - 정규성 경고가 나오는지와 그림도 확인

- 두 번째 정상 사례: 그룹 없이 결측이 있는 변수(`meal.cal`)를 써서 완전사례 처리가 맞는지 봄
- 오류 사례
  - 그룹 안의 쌍 부족(`E_TOO_FEW_PAIRS`, 메시지에 해당 그룹이 나오는지까지)
  - 형식 불일치(`E_TYPE_MISMATCH`)
  - 같은 변수 중복(`E_ARG_CONFLICT`)

이 파일만 돌림:

```
Rscript -e 'testthat::test_file("r-executor/tests/testthat/test-JSM-COR-PEARSON.R")'
```

- 2026-10-12에 이 문서의 작성 환경에서 돌린 결과: `[ FAIL 0 | WARN 0 | SKIP 0 | PASS 49 ]`

11.7 6단계: 전체 테스트와 등록부 점검

```
Rscript r-executor/tests/testthat.R
Rscript registry/build_registry.R
echo $?    # 0이어야 한다
```

1. 전체 테스트가 실패 없이 끝나야 함
2. 생성기가 "명세 점검: 문제 없음"으로 끝나야 함(섹션 10)
3. 생성기가 다시 쓴 세 파일의 변경 확인
   - `REGISTRY.md`에 새 Skill의 행이 생기고 '테스트 결과'가 "통과 (검증 N건, …)"인지
   - `registry.json`의 `skills`에 새 명세가 들어갔는지

- '테스트 결과'가 "미측정"이면 테스트 파일 이름이 `test-<ID>.R`이 아니거나 전체 테스트를 돌리기 전에 생성기를 돌린 것
- 점검에서 무엇이 어떻게 걸리는지: 섹션 10의 실패 예시

11.8 7단계: 반영

- R 실행 서비스는 기동할 때 명세를, agent는 기동할 때 `registry.json`을 읽음 → 둘 다 다시 띄워야 새 Skill이 보임
- 컨테이너로 배포한다면 두 이미지를 모두 다시 만듦
  - R 실행 서비스 이미지(`registry/skills`, `r-executor/R` 포함)
  - agent 이미지(`registry/registry.json` 포함)
- 한쪽만 바꾸면
  - agent는 모르는 Skill이 R 실행 서비스에 있음
  - 또는 agent가 부르는 Skill을 R 실행 서비스가 `E_UNKNOWN_SKILL`이나 `E_VERSION_MISMATCH`로 거부

11.9 새 예제 데이터셋이 필요할 때

- 기존 허용 목록(부록 섹션 16.3)에 맞는 자료가 없으면 계열 전용 데이터셋 파일 `r-executor/R/datasets_<계열>.R`에 추가

```
dataset_catalog$<id> <- list(
  source = "<패키지>::<데이터 이름>",
  description = paste(
    "자료가 무엇인지(연구·대상·행 수).",
    "변수마다 뜻과 코딩(예: status 0 중도절단/1 사망, sex Male/Female)"
  ),
  loader = function() {
    d <- <패키지>::<데이터 이름>
    # 전처리를 모두 여기에 적는다(재코딩, factor 변환, 행 선택)
    rownames(d) <- NULL
    d
  }
)
```

기존 데이터셋 파일이 지키는 규칙:

- R 패키지에 들어 있는 공개 자료만 씀
  - 내려받지 않고 난수로 만들지 않음

  - 같은 코드면 항상 같은 자료가 나옴
- 전처리는 `loader()` 안에 모두 적음
  - 패키지 데이터를 읽을 때 전역 환경을 건드리지 않음(`utils::data(list = name, package = pkg, envir = new.env())` 방식)
- `description`은 agent가 데이터셋 구조와 함께 받아 봄
  - 범주 코드의 의미를 여기에 적지 않으면 LLM은 코드값을 해석하지 못함
  - 표본설계 자료처럼 지정 방법이 따로 있으면 그것도 적음
- 열 이름은 R 문법에 맞게 둠(입력 검증이 그렇지 않은 이름을 거부)
- 데이터가 필요 없는 Skill은 새 자료를 만들지 않고 `no_data`를 씀
- ID가 다른 파일의 데이터셋과 겹치면 등록부 점검의 이름 충돌로 잡힘

11.10 마무리 점검표

- ☐ '종' 기준을 만족함(검증·오류·검수 기준 가운데 하나 이상이 기존 Skill과 다름)
- ☐ `origin`이 `inventory.csv`에 있고 `r_function`이 실제로 부르는 함수임
- ☐ 명세 파일 이름 = ID, `version`은 `0.1.0` 꼴, YAML 1.1 함정(따옴표) 확인
- ☐ 규칙의 열 키는 입력 인자 이름이고 규칙이 쓰는 오류 코드가 모두 `errors`에 있음
- ☐ wrapper가 `skill_error()`로 내는 코드도 `errors`에 있음
- ☐ 규칙·오류 메시지에 관측 범위·개별 값·식별자를 쓰지 않음. 범주 값은 `msg_levels()`로 붙임
- ☐ 결과표 이름·열이 명세 `outputs`와 문자 그대로 같고 표마다 200행 이하
- ☐ 그림의 수치가 표에도 있고 그림 문구는 영문
- ☐ 난수를 쓰면 시드를 고정하고 `notes`에 적음
- ☐ 테스트: 독립 기준값 대조 정상 사례 1개 이상, 오류 사례 2개 이상, 그림·경고 확인
- ☐ 전체 테스트 실패 0, 등록부 생성기 종료 코드 0과 "명세 점검: 문제 없음"
- ☐ R 실행 서비스와 agent를 모두 다시 띄움(또는 두 이미지를 다시 만듦)

12 버전과 재현성

- 같은 Skill에 같은 자료와 인자를 넣으면 언제 어디서 실행해도 같은 결과가 나와야 함
- 그래서 하는 일
  - 패키지 버전 고정
  - Skill 버전을 실행 요청과 결과에 실음
  - 실행마다 provenance를 남김

12.1 패키지 버전 고정

- R 실행 서비스의 패키지는 `r-executor/install_packages.R` 하나로 설치. Dockerfile도 이 스크립트를 부름
- 스크립트는 네 단계로 진행

1. CRAN 스냅샷
   - 2026-06-30 시점의 CRAN 스냅샷(Posit Package Manager)에서 실행 틀·API·분석 패키지를 설치
   - 기본값 `https://packagemanager.posit.co/cran/2026-06-30`. 환경변수 `CRAN_SNAPSHOT`으로 바꿀 수 있음
   - 기보유 패키지는 이 snapshot에서 jstable 1.3.25, jskm 0.6.3, jsmodule 2.0.1, splineplot 0.3.0이 됨
   - 컨테이너 build에서는 같은 날짜의 리눅스 바이너리 스냅샷(`…/cran/__linux__/noble/2026-06-30`)을 `CRAN_SNAPSHOT`으로 지정. 기반 이미지는 `rocker/r-ver:4.5.1`
2. 호환 버전 고정
   - commit을 고정한 GitHub 패키지와 맞는 버전을 먼저 설치
   - tune 1.1.2(1.2 이상에서 stove 오류), factoextra 1.0.7(2.0 이상에서 K-평균의 K를 잘못 고름), vip 0.4.6, lightgbm 3.3.5(stove가 이 버전을 요구)
   - 순서가 바뀌면 GitHub 패키지 설치가 최신 버전을 끌어와 실패
3. GitHub 커밋 고정
   - 커밋 SHA로 고정해 설치하는 것: statgarten 기보유 패키지 4종(stove, soroban, board, shiny.likert)과 그 의존 패키지(ggbiplot, treesnip), CRAN 판이 동작하지 않는 timeROC의 수정 포크
   - `remotes::install_github` 대신 커밋 고정 tarball을 직접 받아 설치
   - 이유: GitHub API의 비인증 호출 한도에 걸리지 않게, 패키지 `DESCRIPTION`의 `Remotes` 항목이 고정 버전을 개발판으로 덮어쓰지 않게
   - 설치하면서 `RemoteSha` 등 출처 필드를 `DESCRIPTION`에 적음

4. 기준 버전 확인
   - 마지막에 기보유 패키지 8종과 고정 버전 4종의 설치 버전을 기대값과 비교
   - 하나라도 다르면 `stopifnot()`으로 멈춤. 이미지 build도 여기서 실패

- 로컬 개발에서는 같은 패키지를 저장소의 `.r-lib` 폴더에 설치해 씀
  - `load.R`, `tests/testthat.R`, `serve.R`은 `.r-lib`이 있으면 라이브러리 경로 맨 앞에 둠
- 새 Skill이 새 패키지를 쓴다면 두 곳을 고침
  - `install_packages.R`의 설치 목록에 넣음 → 이미지에 들어감
  - `core.R`의 `package_versions()` 기본 목록에 넣음 → provenance에 버전이 기록됨
  - 스냅샷 날짜 이후 판이 필요하면 버전 고정(2단계)으로 넣고 이유를 주석에 적음
- 현재 provenance에 기록하는 패키지: 40개(jstable, jskm, jsmodule, splineplot, tableone, survival, survey, coxme, lme4, lmerTest, geepack, car, MASS, pROC, timeROC, MatchIt, survIDINRI, maxstat, riskRegression, cmprsk, ggpubr, GGally, rstatix, forestploter, mgcv, Hmisc, stove, soroban, board, shiny.likert, tune, parsnip, ranger, xgboost, lightgbm, meta, gsDesign, kappaSize, likert, partykit)

12.2 기보유 함수 목록(`inventory.csv`)

- `registry/inventory.csv`: Skill의 `origin`이 가리킬 수 있는 기보유 기능의 목록
- `registry/inventory.R`이 설치된 패키지에서 공개(export) 함수, 인자, 도움말 제목, 예제 유무를 뽑아 만듦
- 기준선: 2026-07-01 이전에 배포된 CRAN 버전, 그 이전의 GitHub 커밋, 사내 Shiny 앱 저장소의 그 이전 커밋
- 스크립트가 확인하는 것(맞지 않으면 경고)
  - CRAN 패키지의 배포일
  - GitHub 패키지 설치본의 `RemoteSha`가 기준 commit과 같은지
  - 앱 파일의 마지막 변경 커밋
- 현재 목록(생성 2026-10-10)의 구성:

패키지·앱	버전	출처	배포· 커밋 일	함수·앱 수	분석 함수(analysis)
jskm	0.6.3	CRAN	2026-05-27	2	2
jsmodule	2.0.1	CRAN	2025-12-18	72	4
jstable	1.3.25	CRAN	2026-03-30	36	15
splineplot	0.3.0	CRAN	2026-03-30	2	2
board	0.0.4	GitHub statgarten/board@625c52c	2023-11-15	9	7
openstat_app	89deb6c	git 저장소 커밋 89deb6c	2025-06-09	4	0
shiny.likert	0.0.1	GitHub statgarten/shiny.likert@3dd51f3	2023-02-13	1	0
soroban	0.0.9	GitHub statgarten/soroban@fc17f63	2023-11-28	10	0
stove	1.2.0	GitHub statgarten/stove@396f7a4	2023-12-14	25	25

- 함수마다 성격(`kind`)이 붙음

성격	뜻
`analysis`	직접 불러 결과를 얻는 계산 함수
`utility`	라벨·옵션 같은 보조 함수
`shiny_module`	분석 로직이 들어 있지만 Shiny 서버 안에서만 동작하는 모듈
`shiny_app`	앱 실행 함수나 사내 앱의 기능 단위 행

- jsmodule처럼 분석 로직이 모듈 안에 있으면 Skill은 모듈 코드를 읽어 같은 계산을 직접 호출로 옮기고 `origin`에는 그 모듈을 적음(섹션 3)
- `origin`에 적을 함수가 목록에 없으면 등록부 점검이 실패

- 목록을 다시 만들 때는 `.r-lib`에 기준 버전이 설치된 상태에서 `Rscript registry/inventory.R`을 실행

12.3 Skill 버전

- 명세의 `version`은 `주.부.수` 꼴 문자열
- 이 값이 쓰이는 세 곳

쓰이는 곳	동작
실행 요청	agent가 Skill을 실행할 때 `registry.json`의 버전을 요청에 실음(`{"skill_id": …, "version": "0.1.0", …}`). R 실행 서비스는 자기가 읽은 명세의 버전과 다르면 실행하지 않고 HTTP 409 `E_VERSION_MISMATCH`를 돌려줌. 요청에 버전이 없으면 비교하지 않음
결과의 provenance	`skill_version`으로 남음
시험 실행기	결과의 `skill_version`이 등록부의 버전과 같은지 확인

- 현재 등록부에서 `0.2.0`인 Skill은 `JST-SUB-COX`, `JST-SUB-GLM`이고 나머지는 `0.1.0`
  - 이 둘은 표 결과를 바꾸지 않고 forest plot 그림을 추가하면서(출력 `figures`와 `origin`에 jsmodule 모듈 추가) 부 버전을 올림
- 버전을 올리는 기준은 코드로 강제되지 않음. 이 문서의 권장:

바뀐 것	올릴 자리
결과표의 값이나 계산 방법	주 버전
입력 인자·출력 표나 그림·오류 정의처럼 명세의 계약(기존 결과값은 그대로)	부 버전
설명·안내 문구만	수 버전

- 어떤 경우든 버전을 바꾸면 생성기를 다시 돌려 `registry.json`을 갱신하고 R 실행 서비스와 agent를 함께 다시 띄움
  - 그렇지 않으면 agent의 요청이 `E_VERSION_MISMATCH`로 모두 거부됨

12.4 실행 기록(provenance)

- 모든 결과에는 provenance(섹션 7)가 붙음
  - 남는 것: Skill ID·버전, 연결 R 함수, 데이터셋, 인자 해시, R 버전, 고정 패키지 목록의 설치 버전(GitHub 패키지는 커밋 SHA 앞 7자리 포함), 실행 시각
  - agent는 이것을 Audit Trail과 trajectory에 저장
- 등록부의 `registry.xlsx` '생성 정보' 시트와 `REGISTRY.md` 머리에도 등록부를 만들 때의 R 버전과 패키지 버전이 기록됨
- 결과 자체의 재현성은 wrapper가 책임짐
  - 난수를 쓰는 계산은 시드를 고정(섹션 8)
  - 자료 전처리는 데이터셋 `loader()`에 모두 적음
- 품질 점검 기능은 실행한 분석을 같은 인자로 한 번 더 실행해 결과표가 같은지 비교 → 재현되지 않는 Skill은 여기서 드러남

13 기보유 패키지 특이점 대응

- Skill은 기보유 패키지(jstable, jskm, jsmodule, splineplot, statgarten 패키지, 사내 Shiny 앱)의 기능을 옮긴 것
- 옮기면서 기준 버전(jstable 1.3.25 등)의 함수를 직접 실행해 보니 결과값이 틀리는 경우, 특정 조건에서 실패하는 경우, 표기가 어긋나는 경우가 적지 않았음
- 모든 항목은 `docs/jstable-issues.md`에 재현 방법과 함께 정리되어 있음
- 등급별 건수(문서를 만들 때 그 파일의 표 행을 셈):

등급	뜻	건수
A	결과값이 틀리거나 틀릴 수 있음	27

등급	뜻	건수
B	오류로 실패	12
C	표기·형식 문제(jstable)	10
C(추가)	표기·형식 문제(jskm·jsmodule, 목록형)	9
S	statgarten·openstat_app 항목	14

- 새 Skill을 만들 때는 이 목록에서 쓰려는 함수의 항목을 먼저 찾아봄
  - 이미 알려진 문제: 같은 방식으로 우회
  - 새로 발견한 문제: 재현 자료와 정답 값을 함께 목록에 추가

13.1 대응 원칙

1. 결과가 틀리는 동작(A 등급)은 wrapper에서 고치고 테스트로 고정
   - 고친 결과는 기보유 패키지를 거치지 않은 독립 기준값과 대조
   - 버그가 드러나는 자료를 정상 사례에 넣어 두면 나중에 누가 우회 코드를 지워도 테스트가 실패
2. 실패하는 조건(B 등급)은 규칙으로 미리 막거나 실패하지 않는 호출 방식으로 바꿈
   - 사용자는 알 수 없는 R 오류(`E_RUNTIME`) 대신 이유와 대안이 있는 오류를 받아야 함
3. 표기 문제(C 등급)는 결과표에서 바로잡고 바로잡은 사실을 `notes`에 적음
4. 우회 방법은 계열 파일 머리말 주석에 적음
   - 적을 것: 기준 버전에서 직접 실행해 확인한 동작, 재현 자료, 대응
   - 적어 두면 다음 사람이 같은 함수를 쓸 때 다시 조사하지 않아도 됨(`tb1.R`, `cox.R`, `km.R`, `cor.R` 머리말 참고)
5. 고칠 수 없으면 그 기능을 Skill에 넣지 않음
   - 예: jsmodule의 ROC 모듈은 서베이·GEE 모형에서도 가중치와 군집을 무시하고 ROC를 계산 → 서베이·GEE ROC 변형은 만들지 않음

13.2 대표 사례

- 중복 검정을 끄면 실패하는 Table 1(B1)
  - `jstable::CreateTableOneJS`에 그룹(`strata`)을 주고 `test = FALSE`로 부르면 “subscript out of bounds”로 실패
  - 기술통계표와 SMD 표처럼 검정이 필요 없는 Skill은 검정을 켠 채 실행한 뒤 p값과 검정 열을 지움

```r
# jstable 1.3.25 CreateTableOneJS는 strata를 주고 test = FALSE로 부르면
# "subscript out of bounds"로 실패한다(mtcars로 재현됨). 검정을 켜서 실행한 뒤 p·test 열을 지운다.
tb1_run_without_test <- function(data, vars, group = NULL, ...) {
  if (is.null(group)) return(tb1_run(data, vars, NULL, test = FALSE, ...))
  run <- tb1_run(data, vars, group, test = TRUE, ...)
  run$table <- run$table[, setdiff(names(run$table), c("p", "test")), drop = FALSE]
  run
}
```

- 전역 자료를 이름으로 다시 찾는 표 함수(A1)
  - `coxme.display`와 다중상태 `cox2.display`는 단변량(crude) 모형을 다시 적합할 때 적합 객체의 `call`에 적힌 데이터 이름으로 자료를 찾음
  - 함수 안에서 적합한 모형을 넘기면 실패
  - 같은 이름의 전역 객체가 있으면 그 자료로 crude 값을 조용히 계산(lung 자료 혼합효과 Cox의 ph.ecog crude 값이 1.66이어야 하는데 1.96으로 나온 사례)
  - wrapper는 고정된 임시 이름으로 전역에 자료를 두고 끝나면 지움(섹션 8의 `cox_with_global_data()`)
  - 이 때문에 R 실행 서비스는 요청을 하나씩 처리
- 변수 이름 부분 일치로 틀린 interaction p값(A2)
  - subgroup 표 함수는 interaction 모형식을 문자열 치환(`gsub`)으로 만듦
  - 노출 변수 이름이 다른 변수 이름의 일부이면(노출 `rx`, 공변량 `agerx`) 엉뚱한 항이 바뀌어 다른 interaction의 p값(0.259, 정답 0.055)이 나옴
  - wrapper는 노출 변수를 자료의 어떤 이름과도 겹치지 않는 내부 이름으로 복사해 실행하고 결과의 이름을 원래대로 되돌림

```r
# 분석 자료를 만들고 노출 변수를 내부 이름으로 옮긴다(jstable 교호작용 gsub 버그 회피).
# 내부 이름은 자료의 어떤 변수명에도 들어 있지 않은 이름이라 gsub가 노출 항만 바꾼다
sub_prepare <- function(data, p, cols, binary_outcome = NULL) {
  d <- rm_prepare(data, cols, binary_outcome = binary_outcome)
  for (v in c(p$exposure, p$subgroups)) d[[v]] <- droplevels(factor(d[[v]]))
  tmp <- "sgexposure"
  while (any(grepl(tmp, c(names(d), "survival", "Surv"), fixed = TRUE))) tmp <- paste0(tmp,
"x")
  d[[tmp]] <- d[[p$exposure]]
  d[[p$exposure]] <- NULL
  list(data = d, exposure = tmp)
}
```

- 사건이 없는 칸이 정상값처럼 보이는 subgroup 표(A5)
  - subgroup × 노출 칸에 사건이 하나도 없으면 HR 0 (0, Inf), p = 1 같은 값이 정상 결과처럼 표에 나옴
  - subgroup Cox 명세는 같은 계열 규칙 `sg_cell_events`를 기준만 바꿔 두 번 겪: 사건이 1건 미만인 칸이 있으면 `E_SPARSE_SUBGROUP`으로 막고 5건 미만이면 `W_FEW_SUBGROUP_EVENTS`로 경고
  - 하나의 규칙으로 차단 기준과 경고 기준을 나눠 두는 방식은 다른 Skill에도 쓸 수 있음
- 수렴 실패를 조용히 결과로 내는 GEE 표(A10)
  - `geeglm.display`는 반복 추정이 수렴하지 않아도 경고 없이 표를 냄
  - wrapper가 적합 객체의 오류 코드를 확인해 `E_NOT_CONVERGED`로 막음(섹션 6)
  - 같은 함수는 자료가 군집 ID로 정렬되지 않으면 군집을 잘못 나누면서 표의 군집 수는 원래 값으로 보여 줌(A9) → wrapper는 적합 전에 ID(와 시간) 순으로 정렬
- 고정된 1.96(A13)
  - `glmshow.display`는 선형 회귀를 포함해 신뢰구간에 1.96을 고정해 씀
  - 반올림 경계에서 표준 Wald 신뢰구간과 끝자리가 달라질 수 있음
  - 결과를 바꾸면 기보유 기능과 달라짐 → 동작은 유지하고 명세의 검수 기준에 z = 1.96을 명시해 기준값도 같은 방식으로 계산
  - 이처럼 "틀린 것은 아니지만 관례와 다른" 동작은 명세와 `notes`에 드러내는 쪽을 택함
- 의존 패키지 버전에 따라 달라지는 동작(S1, S2)
  - statgarten의 stove는 tune 1.2 이상에서 오류가 나고 factoextra 2.0 이상과 함께 쓰면 K-평균의 군집 수를 하나 작게 고름
  - 코드로 우회하기보다 호환되는 버전을 고정(섹션 12)

14 R 실행 서비스 API

- R 실행 서비스는 plumber로 만든 HTTP API(`r-executor/plumber.R`)
- agent만 부르는 내부 서비스
  - 배포 구성(`deploy/compose.yml`)에서는 외부로 포트를 열지 않고 내부 네트워크에만 둠
- 요청에는 Skill ID, 인자, 데이터셋 ID만 받음. R 코드는 받지 않음
- 기동: 저장소 루트에서 `Rscript r-executor/serve.R`
  - 주소: `R_EXECUTOR_HOST`(기본 `127.0.0.1`, 컨테이너는 `0.0.0.0`)
  - 포트: `R_EXECUTOR_PORT`(기본 8001)
- endpoint 목록은 `plumber.R`의 주석에서 읽음

메서드	경로	설명
GET	`/health`	상태 확인
GET	`/skills`	등록 Skill 목록
GET	`/datasets/{ref}/schema`	데이터셋 구조(변수명·형식·범주·요약통계, 행 단위 값 없음)
POST	`/run`	Skill 실행
POST	`/datasets/{ref}/check`	데이터 검증(집계·판정만, 데이터 값 없음)
POST	`/validate`	실행 없이 입력·데이터 검증

메서드	경로	설명
GET	`/figures/{file}`	Skill이 만든 그림
POST	`/uploads/register`	업로드 자료 등록(가명처리·재식별 위험 점검)

- 이 문서의 범위: Skill과 관련된 `/health`, `/skills`, `/datasets/{ref}/schema`, `/run`, `/validate`, `/figures/{file}`
- 데이터 검증(`/datasets/{ref}/check`)과 업로드 등록(`/uploads/register`)은 문서 ②·③에서 다룸
- 아래 응답 예시는 2026-10-12에 저장소 루트에서 `R_EXECUTOR_PORT=18001 Rscript r-executor/serve.R`로 서비스를 로컬에 띄우고 HTTP로 요청해 받은 실제 응답
  - 요청과 저장: `techdocs/skill-standard/_examples/capture_http.py`
  - 오류 응답의 provenance는 생략

14.1 `POST /run`

- 요청 본문은 JSON 객체

필드	필수	내용
`skill_id`	예	등록된 Skill ID
`dataset_ref`	예	허용 목록의 데이터셋 ID 또는 업로드 자료 ID(`up_…`)
`params`	아니오	인자 객체. 없으면 빈 객체
`version`	아니오	요청자가 아는 Skill 버전. 있으면 등록된 버전과 비교

```
curl -s -X POST localhost:8001/run -H 'Content-Type: application/json' \
  -d '{"skill_id": "JST-TB1-TTEST", "version": "0.1.0", "dataset_ref": "colon_recur",
       "params": {"vars": ["age", "nodes"], "group": "sex"}}'
```

- 성공하면 HTTP 200과 섹션 7의 결과 JSON이 돌아옴(이 요청의 표 부분은 그 장의 두 번째 예시와 같음)
- 기본 설정에서는 Skill이 별도 R 프로세스에서 실행됨(섹션 2)
- 실패하면 상태 코드가 아래처럼 정해짐

경우	HTTP	오류 코드
본문을 읽을 수 없거나 `skill_id`·`dataset_ref`가 없음	400	`E_BAD_REQUEST`
`version`이 등록된 버전과 다름	409	`E_VERSION_MISMATCH`
Skill 실행 중 예상하지 못한 R 오류	500	`E_RUNTIME`
그 밖의 모든 Skill 오류(등록되지 않은 Skill·데이터셋, 입력 검증, 규칙, wrapper가 지정한 코드, timeout)	422	해당 코드

규칙 위반(3그룹 변수로 2그룹 비교):

```
HTTP 422
{
  "status": "error",
  "skill_id": "JST-TB1-TTEST",
  "error": {
    "code": "E_GROUP_COUNT",
    "action": "ask_user",
    "message": "그룹 범주가 2개가 아님: rx의 범주 수 3 (최대 2)",
```

```
    "recovery_hint": "3그룹 이상은 JST-TB1-ANOVA를 사용"
  }
}
(provenance 생략)
```

허용 목록에 없는 데이터셋(경로를 넣은 경우):

```
HTTP 422
{
  "status": "error",
  "skill_id": "JST-TB1-TTEST",
  "error": {
    "code": "E_UNKNOWN_DATASET",
    "action": "block",
    "message": "허용 목록에 없는 데이터셋: /etc/passwd",
    "recovery_hint": "등록된 데이터셋만 사용 가능"
  }
}
(provenance 생략)
```

버전 불일치와 필수 필드 누락(Skill 실행 전에 발생 → `action`과 `recovery_hint`가 없음):

```
HTTP 409
{
  "status": "error",
  "skill_id": "JST-TB1-TTEST",
  "error": {
    "code": "E_VERSION_MISMATCH",
    "message": "요청 버전 0.0.9, 등록 버전 0.1.0"
  }
}
```

```
HTTP 400
{
  "status": "error",
  "error": {
    "code": "E_BAD_REQUEST",
    "message": "skill_id와 dataset_ref가 필요함"
  }
}
```

- agent의 클라이언트(`executor.py`)는 422·409·500 응답도 본문을 그대로 결과로 받아 오류 처리로 넘김

14.2 `POST /validate`

- 본문은 `/run`과 같음(`version`은 보지 않음)
- 입력 검증, 데이터 검증 규칙, 실행 조건까지만 실행. wrapper는 부르지 않음
- 통과하면 HTTP 200과 경고 목록, 막히면 HTTP 422와 오류를 돌려줌
- 이 경로는 별도 프로세스와 timeout 없이 서비스 프로세스에서 실행됨

```
HTTP 200
{
  "status": "ok",
  "skill_id": "JSM-COR-PEARSON",
  "warnings": [
    "W_NONNORMAL: Shapiro-Wilk p < 0.05인 변수: bili(p = 1.16e-25), albumin(p = 0.00023)"
  ]
}
```

```
HTTP 422
{
  "status": "error",
  "skill_id": "JSM-COR-PEARSON",
  "error": {
    "code": "E_TYPE_MISMATCH",
    "action": "ask_user",
    "message": "var1·var2는 숫자형, group은 범주형이어야 함: var2: sex은(는) numeric 형식이 아님",
    "recovery_hint": "범주형 변수와 연속변수의 관계는 JSM-VIS-BOX 또는 JST-TB1 계열을 사용"
  }
}
```

- 경고가 하나여도 `warnings`는 배열(섹션 7)

14.3 조회 endpoint

- `GET /health`
  - `{"status": "ok", "skills": <등록 Skill 수>, "packages": {…}}`를 돌려줌
  - `packages`는 provenance와 같은 패키지 버전 목록
  - 위의 로컬 실행에서는 `skills`가 106
- `GET /skills`
  - 등록 Skill마다 `id`, `name`, `method`, `category`, `description`, `version`, `inputs`를 담은 배열을 돌려줌
  - 규칙·오류·출력 정의는 담지 않음
  - agent는 이 endpoint가 아니라 `registry.json`으로 도구를 만듦
- `GET /datasets/{ref}/schema`: 데이터셋 구조를 돌려줌
  - 최상위: `dataset`, `source`, `description`, `n_rows`와 변수 목록 `variables`
  - 변수마다 `name`, `class`, `n_missing`. 숫자형이면 요약통계(`summary`: min, q1, median, q3, max, mean)와 서로 다른 값 수(`n_unique`), 범주형이면 수준 목록(`levels`, 범주가 하나여도 배열. 30개를 넘으면 개수 `n_levels`만)
  - 행 단위 값은 없음. 업로드 자료는 최솟값·최댓값을 빼고, 5행 미만에만 있는 범주는 `levels`에서 빼고 그 개수를 `n_rare_levels`로, 날짜·시각 열은 개수 `n_levels`만 줌
  - agent는 이것을 LLM에 데이터셋 구조로 넘김 → 데이터셋 `description`과 범주 수준 이름이 LLM이 자료를 이해하는 근거
  - 허용 목록에 없는 ID는 404
- `GET /figures/{file}`: Skill이 저장한 그림 PNG를 돌려줌(`Content-Type: image/png`)
  - `file`이 64자리 16진수 + `.png` 형식이 아니면 400, 파일이 없으면 404
  - 위 로컬 실행에서 `JSM-COR-PEARSON` 결과의 `file`로 받은 PNG의 SHA-256은 결과 메타데이터의 `sha256`과 같았음

14.4 동시 처리

- plumber는 요청을 한 번에 하나씩 처리
- 일부 wrapper가 임시 전역 객체를 쓰기 때문에 같은 프로세스에서 동시 실행하지 않도록 한 설계(섹션 13)
- 처리량이 더 필요하면 서비스 인스턴스를 늘림

15 출력 명세와 기계 판정

- 시험 실행기 `eval/kaic_suite.py`는 서버 내부를 보지 않고 공개 API만으로 Skill을 시험
- Python 표준 라이브러리만 씀 → 다른 PC에서 그대로 돌릴 수 있음
- 판정 기준은 사람이 정하지 않고 등록부 명세에서 나옴 → 명세를 어떻게 쓰느냐가 곧 시험에서 어떻게 판정되느냐
- 아래는 실행기가 명세의 어느 부분을 어떻게 쓰는지만 정리

15.1 실행과 출력 형식 판정

- 실행기는 명세의 예제(`examples`)마다 그 인자로 분석계획을 제출하고 실행한 뒤 결과 JSON을 받아 아래를 확인

확인 항목	기준
실행 성공	결과의 `status`가 `ok`이고 `error`가 없음

확인 항목	기준
표 형식	결과의 표 이름이 모두 명세 `outputs.tables`에 있음. 표마다 행의 열 집합이 같음. 명세 `columns`의 필수 열이 모두 있고 명세에 없는 열이 없음
그림	명세에 `figures`가 있거나 결과에 그림이 있으면 받은 파일이 PNG 서명으로 시작하고 2,000바이트보다 크며 SHA-256이 산출물 목록의 해시와 같음
출처 기록	provenance의 `skill_id`가 같음. `skill_version`이 등록부 버전과 같음. `r_function·packages·params_hash·r_version`이 있음

- 표 형식 판정은 명세의 열 표기를 정규식으로 바꿔서 함(섹션 4.4)
  - `<설명>`은 `.+`가 됨
  - `A | B` 꼴로 나눈 대안은 그중 하나와 맞으면 됨
  - 끝에 `?`가 붙은 열은 없어도 됨
- 행이 없는 빈 표와 `columns`가 없는 표는 열을 확인하지 않음
- 명세의 표 가운데 일부가 결과에 없는 것은 문제 삼지 않음(표가 하나도 없으면 부적합)
- 명세에 없는 이름의 표가 결과에 있으면 부적합

15.2 입력 오류 판정

- 실행기는 명세의 입력 정의와 첫 번째 예제 인자에서 오류 입력을 자동으로 만듦
- 만들 수 있는 것만 만듦

종류	만드는 방법	기대 코드
없는 변수	예제 인자에 있는 첫 열 인자(`column·column_list`)의 값을 `no_such_var`로 바꿈(목록이면 첫 원소)	`E_COL_NOT_FOUND`
필수 인자 누락	예제 인자에 있는 첫 필수 인자를 뺌	`E_MISSING_PARAM`
범위 밖 숫자	`min`이나 `max`가 있는 첫 `integer·number` 인자에 `min - 1`(없으면 `max + 1`)을 넣음	`E_INVALID_VALUE`
허용 밖 선택값	첫 `enum` 인자에 `__not_allowed__`를 넣음	`E_INVALID_VALUE`
형식 불일치	예제 인자에 있는 첫 `dtype: numeric column` 인자에 같은 데이터셋에서 아직 쓰지 않은 범주형(factor·문자) 변수를 넣음	`E_TYPE_MISMATCH`

- 기대 action은 그 코드의 명세 `errors` 정의에서 가져옴. 없으면 실행 틀의 공통 정의에서 가져옴
- 실행기가 확인하는 것
  - 오류 입력으로 계획을 제출해 HTTP 422로 거부되는지
  - 분석별 오류의 코드와 action이 기대와 같은지
  - `message`와 `recovery_hint`가 비어 있지 않은지
  - 그다음 같은 프로젝트에 예제 인자 그대로 다시 제출해 접수되는지(복구)

15.3 명세 작성 시 유의점

이 판정 방식에서 바로 나오는 규칙:

- 결과표의 열 이름은 명세 `columns`와 문자 그대로 같아야 함
  - 열을 더하거나 이름을 바꾸면 명세도 함께 고침

- 첫 예제는 성공해야 하고 필수 인자와 열 인자를 포함해야 오류 입력 시험이 만들어짐(섹션 4.7)
- 숫자 인자의 허용 범위는 규칙보다 `min·max`로 적음
  - 입력 검증에서 `E_INVALID_VALUE`로 막힘
  - LLM 도구 정의에도 `minimum·maximum`으로 들어감
  - 범위 밖 숫자 시험도 자동으로 만들어짐
- 입력 검증 공통 코드(`E_COL_NOT_FOUND`, `E_MISSING_PARAM`, `E_INVALID_VALUE`, `E_TYPE_MISMATCH`)의 action을 명세에서 바꾸면 기대 action도 그것을 따름
  - 바꿀 특별한 이유가 없으면 공통 정의와 같은 `ask_user`로 둠

16 부록

16.1 Skill 전체 목록

- 등록부(`registry.json`, 생성 2026-10-27 13:58:45 UTC)의 106종을 분석 범주별로 나눔. 범주 안에서는 ID 순
- 각 Skill의 입력·규칙·오류·검수 기준·예제 전체: `registry/registry.xlsx`의 '등록부' 시트나 `registry/skills/<ID>.yaml`

16.1.a 생존분석 (12종)

ID	기능	R 함수
JSK-KM-CIF	경쟁위험 누적발생함수(CIF)와 Gray 검정	jskm::jskm, survival::survfit, cmprsk::cuminc
JSK-KM-LANDMARK	Landmark 생존분석	jskm::jskm, survival::survfit, survival::survdiff
JSK-KM-LOGRANK	Kaplan-Meier 생존곡선과 log-rank 검정	jskm::jskm, survival::survfit, survival::survdiff
JSK-KM-SVY	표본설계·IPTW 가중 Kaplan-Meier 생존곡선	jskm::svyjskm, survey::svykm, survey::svylogrank
JSM-SURV-CUTOFF	생존 기준 최적 절단값(최대 선택 log-rank 통계)	maxstat::maxstat.test, jskm::jskm, survival::survfit, survival::survdiff
JSM-SURV-FINEGRAY	Fine-Gray 경쟁위험 회귀(하위분포 위험비) 결과표	survival::finegray, survival::coxph, jstable::cox2.display
JST-COX-CLUSTER	군집 robust SE Cox 회귀 결과표	jstable::cox2.display
JST-COX-COXME	혼합효과 Cox 회귀 결과표	jstable::coxme.display
JST-COX-FRAILTY	감마 frailty Cox 회귀 결과표	jstable::cox2.display
JST-COX-MSM	다중상태(경쟁위험) Cox 회귀 결과표	jstable::cox2.display
JST-COX-PH	Cox 비례위험 회귀 결과표	jstable::cox2.display
JST-COX-STRATA	층화 Cox 회귀 결과표	jstable::cox2.display

16.1.b 기술통계·그룹비교 (11종)

ID	기능	R 함수
JST-TB1-ANOVA	연속변수 3그룹 이상 비교표	jstable::CreateTableOneJS
JST-TB1-CHISQ	범주형 변수 그룹 비교표(카이제곱)	jstable::CreateTableOneJS
JST-TB1-DESC	기술통계표	jstable::CreateTableOneJS
JST-TB1-FISHER	범주형 변수 그룹 비교표(Fisher exact)	jstable::CreateTableOneJS

ID	기능	R 함수
JST-TB1-KW	비정규 연속변수 3그룹 이상 비교표	jstable::CreateTableOneJS
JST-TB1-MWU	비정규 연속변수 2그룹 비교표	jstable::CreateTableOneJS
JST-TB1-NORMALITY	정규성 검정 기반 그룹 비교표	jstable::CreateTableOneJS
JST-TB1-PAIRWISE	그룹 쌍별 사후비교표(어느 그룹끼리 다른지)	jstable::CreateTableOneJS
JST-TB1-SMD	표준화 평균차(SMD) 균형표	jstable::CreateTableOneJS
JST-TB1-STRATA2	2단 층화 그룹 비교표	jstable::CreateTableOneJS
JST-TB1-TTEST	연속변수 2그룹 비교표	jstable::CreateTableOneJS

16.1.c 서베이·표본설계 (10종)

ID	기능	R 함수
JST-SVY-CAT	서베이 범주형 변수 그룹 비교표	jstable::svyCreateTableOneJS
JST-SVY-CONT	서베이 연속변수 그룹 비교표	jstable::svyCreateTableOneJS
JST-SVY-COX	서베이 가중 Cox 회귀표	jstable::svycox.display
JST-SVY-DESC	서베이 가중 기술통계표	jstable::svyCreateTableOneJS
JST-SVY-LINEAR	서베이 선형 회귀표	jstable::svyregress.display
JST-SVY-LOGISTIC	서베이 로지스틱 회귀표	jstable::svyregress.display
JST-SVY-PAIRWISE	서베이 그룹 쌍별 사후비교표(어느 그룹끼리 다른지)	jstable::svyCreateTableOneJS
JST-SVY-RANK	서베이 비정규 연속변수 그룹 비교표(중앙값·순위 검정)	jstable::svyCreateTableOneJS
JST-SVY-SMD	서베이 가중 표준화 평균차(SMD) 표	jstable::svyCreateTableOneJS
JST-SVY-SUBCOX	서베이 가중 Cox 서브그룹 분석표	jstable::TableSubgroupMultiCox, jstable::TableSubgroupCox

16.1.d 머신러닝(분류) (9종)

ID	기능	R 함수
STV-CLS-KNN	K-최근접이웃 분류	stove::KNN, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-LGBM	LightGBM 분류	stove::lightGbm, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-LOGISTIC	벌점 로지스틱 회귀 분류(머신러닝)	stove::logisticRegression, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-MLP	신경망(다층 퍼셉트론) 분류	stove::MLP, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-NB	나이브 베이즈 분류	stove::naiveBayes, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-RF	랜덤포레스트 분류	stove::randomForest, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix

ID	기능	R 함수
STV-CLS-SVM	선형 서포트 벡터 머신 분류	stove::SVMLinear, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-TREE	의사결정나무 분류(CART)	stove::decisionTree, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix
STV-CLS-XGB	XGBoost 분류	stove::xgBoost, stove::trainTestSplit, stove::prepForCV, stove::rocCurve, stove::confusionMatrix

16.1.e 메타분석 (8종)

ID	기능	R 함수
OSA-META-BIAS	출판 편향 평가(funnel plot, Egger 검정)	meta::metabias, meta::funnel.meta
OSA-META-BIN	이분형 결과 메타분석(사건 수·N)	meta::metabin, meta::forest.meta
OSA-META-CONT	연속형 결과 메타분석(평균·SD)	meta::metacont, meta::forest.meta
OSA-META-DRAPERY	Drapery plot(p값 함수)	meta::metabin, meta::metacont, meta::metagen, meta::drapery
OSA-META-GEN	효과크기·95% CI 통합 메타분석(일반 역분산)	meta::metagen, meta::forest.meta
OSA-META-HET	메타분석 이질성 평가(Cochran Q·I^2·τ^2·예측구간·영향 분석)	meta::metabin, meta::metacont, meta::metagen, meta::metainf, metafor::baujat
OSA-META-PROP	단일 비율 메타분석(유병률·발생 비율)	meta::metaprop, meta::forest.meta
OSA-META-TRIMFILL	Trim-and-fill 출판 편향 보정	meta::trimfill.meta, meta::funnel.meta

16.1.f 머신러닝(회귀) (7종)

ID	기능	R 함수
STV-REG-KNN	K-최근접이웃 회귀	stove::KNN, stove::trainTestSplit, stove::prepForCV
STV-REG-LGBM	LightGBM 회귀	stove::lightGbm, stove::trainTestSplit, stove::prepForCV
STV-REG-LINEAR	벌점 선형 회귀(머신러닝)	stove::linearRegression, stove::trainTestSplit, stove::prepForCV
STV-REG-MLP	신경망(다층 퍼셉트론) 회귀	stove::MLP, stove::trainTestSplit, stove::prepForCV
STV-REG-RF	랜덤포레스트 회귀	stove::randomForest, stove::trainTestSplit, stove::prepForCV
STV-REG-TREE	의사결정나무 회귀(CART)	stove::decisionTree, stove::trainTestSplit, stove::prepForCV
STV-REG-XGB	XGBoost 회귀	stove::xgBoost, stove::trainTestSplit, stove::prepForCV

16.1.g 진단·예측성능 (7종)

ID	기능	R 함수
JSM-ROC-AUC	ROC 곡선과 AUC	pROC::roc, pROC::ci.auc, pROC::ggroc, stats::glm

ID	기능	R 함수
JSM-ROC-COMPARE	모형 간 AUC 비교(DeLong 검정)	pROC::roc, pROC::ci.auc, pROC::roc.test, pROC::ggroc, stats::glm
JSM-ROC-CUTOFF	최적 절단값과 진단정확도(민감도·특이도·PPV·NPV)	pROC::roc, pROC::coords, pROC::ci.auc, stats::binom.test
JSM-ROC-IDI	통합판별개선지수(IDI)	Hmisc::improveProb, stats::glm
JSM-ROC-NRI	순재분류지수(NRI)	Hmisc::improveProb, stats::glm
JSM-SURV-IDINRI	생존모형 재분류 지표(IDI·NRI)	survIDINRI::Est.HR.FUN, survIDINRI::IDI.FUN, survIDINRI::PTB.IDI.FUN, survival::coxph
JSM-SURV-TIMEROC	시간의존 ROC·AUC와 생존 예측모형 성능 비교	riskRegression::Score, timeROC::timeROC, survival::concordance, survival::coxph

16.1.h 표본수 계산 (5종)

ID	기능	R 함수
OSA-SS-CRCTBIN	군집 무작위배정 임상시험 표본수(이분형 결과)	stats::qnorm
OSA-SS-CRCTCONT	군집 무작위배정 임상시험 표본수(연속형 결과)	stats::qnorm
OSA-SS-KAPPACI	관찰자 간 일치도(kappa) 표본수(신뢰구간 폭 기반)	kappaSize::CIBinary, kappaSize::CI3Cats, kappaSize::CI4Cats, kappaSize::CI5Cats
OSA-SS-KAPPAPOW	관찰자 간 일치도(kappa) 표본수(검정력 기반)	kappaSize::PowerBinary, kappaSize::Power3Cats, kappaSize::Power4Cats, kappaSize::Power5Cats
OSA-SS-SURV	생존분석 표본수(로그순위 검정, Lachin-Foulkes)	gsDesign::nSurvival

16.1.i 시각화 (4종)

ID	기능	R 함수
JSM-VIS-BAR	그룹별 평균 막대그래프	ggpubr::ggbarplot, ggpubr::stat_compare_means, ggpubr::geom_pwc
JSM-VIS-BOX	그룹별 상자그림	ggpubr::ggboxplot, ggpubr::stat_compare_means, ggpubr::geom_pwc
JSM-VIS-HIST	히스토그램(분포)	ggpubr::gghistogram
JSM-VIS-LINE	시점별 선그래프	ggpubr::ggline, ggpubr::stat_compare_means

16.1.j 탐색적 자료분석 (4종)

ID	기능	R 함수
BRD-EDA-DIST	단일 변수 분포 탐색(히스토그램·요약·이상치·빈도)	board::distribute, board::describe, board::outlier, board::ggpie
BRD-EDA-NORMAL	Shapiro-Wilk 정규성 판정	board::isNormal, stats::shapiro.test

ID	기능	R 함수
BRD-EDA-OUTLIER	IQR 기반 이상치 탐지	board::outlier
BRD-EDA-SUMMARY	데이터 프로파일링(변수 요약·데이터 품질)	board::brief, board::describe, board::dominate

16.1.k 회귀분석 (4종)

ID	기능	R 함수
JST-REG-LINEAR	선형 회귀 결과표	jstable::glmshow.display
JST-REG-LOGISTIC	로지스틱 회귀 결과표	jstable::glmshow.display
JST-REG-POISSON	포아송 회귀 결과표	jstable::glmshow.display
SRB-REG-STEP	단계적 선택 다중 선형회귀(AIC)	stats::step, caret::varImp

16.1.l 다변량 분석 (3종)

ID	기능	R 함수
SRB-STAT-PCA	주성분분석(PCA)과 biplot	stats::prcomp, ggbiplot::ggbiplot
SRB-TREE-CLS	분류나무(조건부 추론 나무, 범주형 목적변수)	partykit::ctree, ggparty::ggparty
SRB-TREE-REG	회귀나무(모형 기반 재귀 분할, 연속형 목적변수)	partykit::lmtree, ggparty::ggparty

16.1.m 반복측정·GEE (3종)

ID	기능	R 함수
JST-GEE-LINEAR	GEE 선형 회귀 결과표	jstable::geeglm.display
JST-GEE-LOGISTIC	GEE 로지스틱 회귀 결과표	jstable::geeglm.display
JST-GEE-POISSON	GEE 포아송 회귀 결과표	jstable::geeglm.display

16.1.n 반복측정·혼합모형 (3종)

ID	기능	R 함수
JST-MIX-GLMM	로지스틱 혼합모형 결과표	jstable::lmer.display
JST-MIX-LMM	선형 혼합모형 결과표	jstable::lmer.display
JST-MIX-POISSON	포아송 혼합모형 결과표	jstable::lmer.display

16.1.o 상관분석 (3종)

ID	기능	R 함수
JSM-COR-MATRIX	상관행렬·산점도 행렬	GGally::ggpairs, stats::cor.test
JSM-COR-PEARSON	Pearson 상관분석(산점도)	ggpubr::ggscatter, stats::cor.test
JSM-COR-SPEARMAN	Spearman 순위상관분석(산점도)	ggpubr::ggscatter, stats::cor.test

16.1.p 서브그룹 분석 (3종)

ID	기능	R 함수
JST-SUB-COX	Cox 서브그룹 분석표	jstable::TableSubgroupMultiCox
JST-SUB-GLM	GLM 서브그룹 분석표	jstable::TableSubgroupMultiGLM

ID	기능	R 함수
JST-SUB-GLMM	혼합모형 서브그룹 분석표	jstable::TableSubgroupGLM

16.1.q 스플라인·용량-반응 (3종)

ID	기능	R 함수
SPL-SPL-COX	제한 입방 스플라인 Cox 용량-반응 곡선	survival::coxph, splines::ns, splineplot::splineplot
SPL-SPL-GAM	일반화 가법 모형(GAM) 평활 곡선	mgcv::gam, splineplot::splineplot
SPL-SPL-GLM	제한 입방 스플라인 GLM 용량-반응 곡선	stats::glm, splines::ns, splineplot::splineplot

16.1.r CDISC 연계 (2종)

ID	기능	R 함수
CDS-ADSL-BASELINE	CDISC ADaM ADSL 분석집단별 기저 특성표	jstable::CreateTableOneJS
CDS-ADTTE-SURV	CDISC ADaM ADTTE 사건까지 시간 분석(KM·log-rank·Cox)	jskm::jskm, survival::survfit, survival::survdiff, survival::coxph, jstable::cox2.display

16.1.s 인과성분석 (2종)

ID	기능	R 함수
JSM-PS-IPTW	역확률 가중(IPTW)과 가중 전후 균형	stats::glm, jstable::CreateTableOneJS, jstable::svyCreateTableOneJS, survey::svydesign
JSM-PS-MATCH	성향점수 매칭과 매칭 전후 균형	MatchIt::matchit, jstable::CreateTableOneJS, jstable::svyCreateTableOneJS

16.1.t 기술통계 (1종)

ID	기능	R 함수
SRB-STAT-GROUP	그룹 조합별 요약 통계	dplyr::group_by, dplyr::summarise

16.1.u 머신러닝(군집) (1종)

ID	기능	R 함수
STV-CLU-KMEANS	K-평균 군집분석(실루엣 기준 군집 수 선택)	stove::kMeansClustering, stats::kmeans

16.1.v 설문 분석 (1종)

ID	기능	R 함수
LKT-VIS-LIKERT	Likert 응답 분포 차트	likert::likert, likert::likert.bar.plot

16.2 오류 코드

- 명세에 정의된 오류 코드와 action, 그 정의를 가진 Skill 수
- 같은 코드가 Skill마다 다른 action으로 정의되어 있으면 action별로 따로 셈
- 전체 1,301개 정의, 서로 다른 코드 137종

오류 코드	action	쓰는 Skill 수
E_COL_NOT_FOUND	ask_user	101
E_ARG_CONFLICT	ask_user	100
E_TYPE_MISMATCH	ask_user	99
W_MISSING_HIGH	warn	90
E_INVALID_VALUE	ask_user	86
E_CONSTANT_VARIABLE	block	77
E_TOO_FEW_ROWS	block	67
E_RUNTIME	block	59
E_TOO_MANY_LEVELS	ask_user	48
E_MISSING_PARAM	ask_user	41
E_GROUP_COUNT	ask_user	32
E_TOO_FEW_EVENTS	block	27
W_LOW_EPV	warn	19
E_SMALL_GROUP	block	17
W_NO_CONVERGENCE	warn	17
E_TIMEOUT	block	16
W_SMALL_SAMPLE	warn	16
E_EVENT_LEVEL_NOT_FOUND	ask_user	15
E_CATEGORICAL_NOT_COVARIATE	ask_user	14
E_EVENT_CODING	ask_user	14
E_NONPOSITIVE_TIME	ask_user	14
E_BAD_WEIGHTS	ask_user	11
E_DESIGN_MISSING	block	11
E_LONELY_PSU	block	11
E_PSU_NOT_NESTED	ask_user	11
E_DESIGN_DF	block	10
E_TARGET_NOT_CONTINUOUS	ask_user	9
E_TOO_FEW_STUDIES	block	8
E_FEW_CLUSTERS	block	7
E_FEW_EVENTS	block	7
W_CONSTANT_PREDICTION	warn	7
W_FEW_STUDIES	warn	7
W_SEPARATION	warn	7
E_GROUP_LEVELS	ask_user	6
E_INVALID_COUNTS	block	6
E_DUPLICATE_MODEL	ask_user	5

오류 코드	action	쓰는 Skill 수
E_INVALID_CI	block	5
E_INVALID_SD	block	5
E_SINGLE_GROUP	ask_user	5
E_UNKNOWN_PARAM	block	5
W_GROUP_NO_EVENT	warn	5
E_CATEGORICAL_NOT_PREDICTOR	ask_user	4
E_NO_EFFECT	ask_user	4
E_NO_EVENTS	block	4
E_NO_REPEATED	block	4
E_NONPOSITIVE_TIME	block	4
E_OUTCOME_FAMILY	ask_user	4
W_FEW_CLUSTERS	warn	4
W_SMALL_SUBGROUP	warn	4
W_TEST_FAILED	warn	4
E_CENSOR_NOT_FOUND	ask_user	3
E_DUPLICATE_TIME	ask_user	3
E_EXPOSURE_LEVELS	ask_user	3
E_NOT_CONVERGED	block	3
E_REF_RANGE	ask_user	3
E_SPARSE_SUBGROUP	ask_user	3
E_TIME_NOT_INTEGER	ask_user	3
E_TIME_REQUIRED	ask_user	3
E_TOO_FEW_CLUSTERS	block	3
E_TOO_FEW_PAIRS	block	3
E_TOO_MANY_ROWS	block	3
E_TOO_MANY_STATES	ask_user	3
W_CURVE_MISMATCH	warn	3
W_LOW_OBS_PER_PARAM	warn	3
W_NO_REPEATED_CLUSTERS	warn	3
W_NOT_CONVERGED	warn	3
W_PH_TEST_FAILED	warn	3
W_PH_VIOLATION	warn	3
W_SINGULAR_FIT	warn	3
W_SMALL_GROUP	warn	3
E_ADAM_FLAG	ask_user	2
E_ADAM_STRUCTURE	ask_user	2

오류 코드	action	쓰는 Skill 수
E_EVENT_NOT_01	ask_user	2
E_EVENT_NOT_FOUND	ask_user	2
E_EVENTRATE_TIME	ask_user	2
E_FEW_CASES_CONTROLS	ask_user	2
E_INVALID_PREVALENCE	ask_user	2
E_KNOTS_TIED	ask_user	2
E_NO_COMPETING_EVENT	ask_user	2
E_NOT_BINARY_01	ask_user	2
E_NOT_COUNT	ask_user	2
E_TIME_POINT_RANGE	ask_user	2
E_TOO_FEW_VALUES	ask_user	2
E_TOO_MANY_PARAMS	block	2
E_TREATED_LEVEL_NOT_FOUND	ask_user	2
W_IMBALANCE	warn	2
W_NO_SPLIT	warn	2
W_OVERDISPERSION	warn	2
W_PAIRWISE_SKIPPED	warn	2
W_POOR_OVERLAP	warn	2
E_ACCRUAL_TOO_LONG	ask_user	1
E_ADAM_PARAM	ask_user	1
E_CUTOFF_OUT_OF_RANGE	ask_user	1
E_FEW_UNIQUE	ask_user	1
E_INVALID_CUTOFFS	ask_user	1
E_LANDMARK_NO_EVENTS	ask_user	1
E_LANDMARK_NO_RISK	ask_user	1
E_LANDMARK_RANGE	ask_user	1
E_LEVEL_MISMATCH	ask_user	1
E_LEVELS_REQUIRED	ask_user	1
E_NEGATIVE_TIME	ask_user	1
E_NO_CUTPOINT	ask_user	1
E_NO_MATCH	block	1
E_NONPOSITIVE_OFFSET	ask_user	1
E_OFFSET_NOT_POSITIVE	ask_user	1
E_OUTCOME_CODING	ask_user	1
E_OUTCOME_NOT_CONTINUOUS	ask_user	1
E_OUTCOME_NOT_COUNT	ask_user	1

오류 코드	action	쓰는 Skill 수
E_SAMPLE_SIZE_RANGE	block	1
E_SEPARATION	block	1
E_SINGLE_COVARIATE	ask_user	1
E_SLOPE_NOT_COVARIATE	ask_user	1
E_SPARSE_CELL	ask_user	1
E_STATE_COUNT	ask_user	1
E_STATE_NOT_FOUND	ask_user	1
E_STRATA_LEVELS	ask_user	1
E_SUBGROUP_LEVELS	ask_user	1
E_SUBGROUP_NO_EVENT	ask_user	1
E_TIME_LEVELS	ask_user	1
E_TOO_MANY_STRATA	ask_user	1
W_ADDED_NOT_SIGNIFICANT	warn	1
W_AUC_BELOW_HALF	warn	1
W_CUTOFF_TIES	warn	1
W_DEGENERATE_CUTOFF	warn	1
W_EXTREME_WEIGHTS	warn	1
W_FEW_CONTROLS	warn	1
W_FEW_SUBGROUP_EVENTS	warn	1
W_FEW_VALUES	warn	1
W_LANDMARK_FEW_AT_RISK	warn	1
W_MANY_LEVELS	warn	1
W_MANY_TIES	warn	1
W_NO_PREDICTOR	warn	1
W_NONNORMAL	warn	1
W_SMALL_EXPECTED	warn	1
W_SPARSE_CELLS	warn	1
W_STRATUM_NO_EVENT	warn	1
W_SUBGROUP_FEW_EVENTS	warn	1
W_UNMATCHED_TREATED	warn	1

16.3 예제 데이터셋

- 데이터셋 허용 목록(`r-executor/R/datasets.R`, `datasets_*.R`)의 29개와 명세 예제에서 각 데이터셋을 쓰는 Skill 수
- 모두 R 패키지에 들어 있는 공개 자료. 전처리는 정의 파일의 `loader()`에 있음

데이터셋 ID	출처	정의 파일	예제로 쓰는 Skill 수
colon_recur	survival::colon	datasets.R	16

데이터셋 ID	출처	정의 파일	예제로 쓰는 Skill 수
lung	survival::lung	datasets.R	10
pbc	survival::pbc	datasets.R	11
cdisc_adsl	safetyData::adam_adsl (CDISC Pilot 01 ADaM ADSL)	datasets_cdisc.R	1
cdisc_adtte	safetyData::adam_adtte (CDISC Pilot 01 ADaM ADTTE)	datasets_cdisc.R	1
sleepstudy	lme4::sleepstudy	datasets_longitudinal.R	1
respiratory	geepack::respiratory	datasets_longitudinal.R	3
dietox	geepack::dietox	datasets_longitudinal.R	2
epil	MASS::epil	datasets_longitudinal.R	2
ml_pima	MASS::Pima.tr + MASS::Pima.te	datasets_ml.R	14
ml_birthwt	MASS::birthwt	datasets_ml.R	8
ml_biopsy	MASS::biopsy	datasets_ml.R	2
meta_aspirin	meta::Fleiss1993bin	datasets_ml.R	1
meta_thrombolysis	meta::Olkin1995	datasets_ml.R	2
meta_mental	meta::Fleiss1993cont	datasets_ml.R	1
meta_bcg	metadat::dat.bcg (2×2 표를 사건 수·총 수로 변환)	datasets_ml.R	2
meta_cll_hr	metadat::dat.steurer2006	datasets_ml.R	1
meta_depression_mi	metadat::dat.feng2019	datasets_ml.R	1
likert_mass	likert::mass (열 이름 정리)	datasets_ml.R	1
no_data	(없음)	datasets_ml.R	5
asah	pROC::aSAH	datasets_roc.R	5
lalonde	MatchIt::lalonde	datasets_roc.R	2
svy_nhanes	survey::nhanes	datasets_survey.R	4
svy_apistrat	survey::apistrat	datasets_survey.R	1
svy_pbc	survival::pbc (survey::svycoxph 도움말 예제의 표본추출 모형)	datasets_survey.R	6
retinopathy	survival::retinopathy	datasets_survival.R	1
rats	survival::rats	datasets_survival.R	1
cgd_gap	survival::cgd	datasets_survival.R	0
cgd_count	survival::cgd (환자 단위 집계)	datasets_survival.R	1